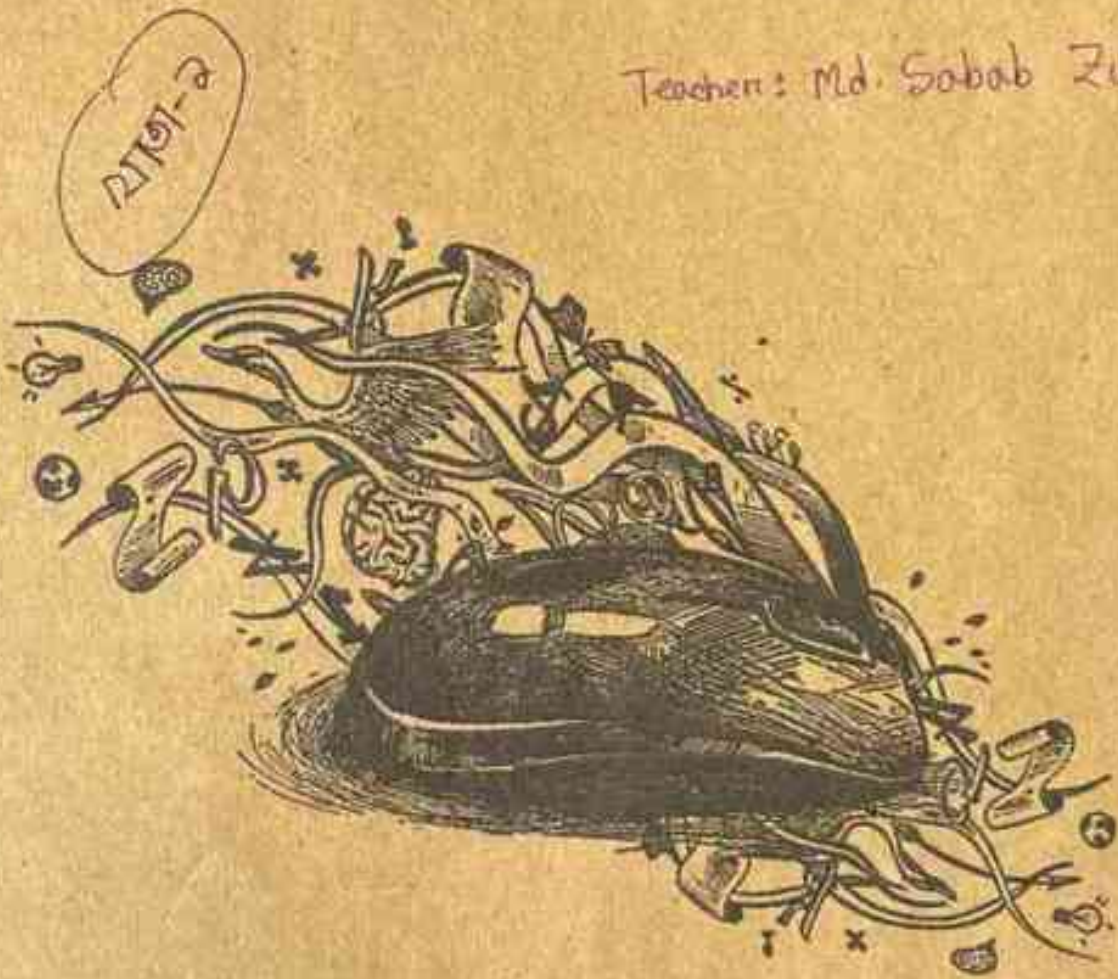


CLASSMATE

5th sem.

Course Code: CSE3105
Compiler Design

Teacher: Md. Sabab Zulfikar



Bashundhara
Exercise Book
Write Your Future

Ensan Mahmud
Compiler Design

Mid-01

→ 19

Language Processing System, phase of
compiler design, CFG, Regular Expression

Mid-02

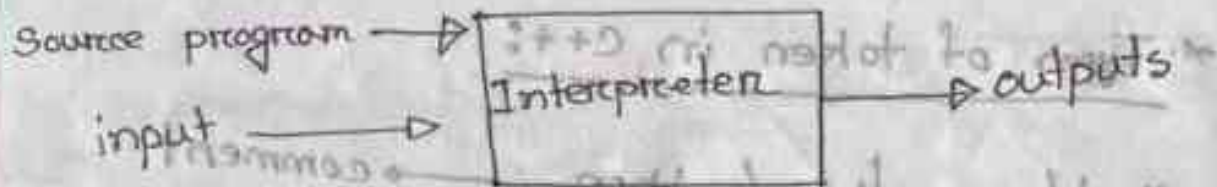
24

20-5-2025 (Sabab sir)

[cvxrzhge]

Slide name: Language processing system
compiler.

interpreter: [statement by statement convert]
[time for error]



* Compiler and interpreter:

* phase of compiler:

* The Structure of a compiler:

Syntactical error → Syntax error

Semantical error → `int a = 5.00`

* phase of compiler: 6 phases
int is float value

- Lexical analysis
- Syntax analysis
- Semantic "
- Intermediate code Generation
- Code optimization
- Final code generation

another slide of 13 page

* The Role of the Lexical Analyzer:

অনেকসময় extra white space remove করে থাকে।

Token → অর্থাত্মিক বাইট

How it works:

Lexeme: a specific instance of a token.

* Types of token in C++:

!/usr/bin/python → comment

var 1 = "Hello world"

var 2 = "Compiler Design"

print ("var 1:", var 1)

print ("var 2 [1:5]:", var 2[1:5])

Code (যদি token সূচনা করে)

21-05-2025

→ slide: phase of a compiler

* phase of a compiler:

$$\text{position} = \text{initial} + \text{rate} * 60$$

① Lexical Analysis: white space & comment remove

{ token name, attribute value }

→ token

position = initial + rate * 60

Lexical Analysis
Lexical error is caught

$\langle \text{id}, 1 \rangle \langle = \rangle \langle \text{id}, 2 \rangle \langle + \rangle \langle \text{id}, 3 \rangle \langle * \rangle \langle 60 \rangle$

tokens

② Syntactic Analysis (Syntax analysis): ②

[parse tree बनाता]

statement: position = initial + rate * 60

$\langle \text{id}, 1 \rangle \langle = \rangle \langle \text{id}, 2 \rangle \langle + \rangle \langle \text{id}, 3 \rangle \langle * \rangle \langle 60 \rangle$

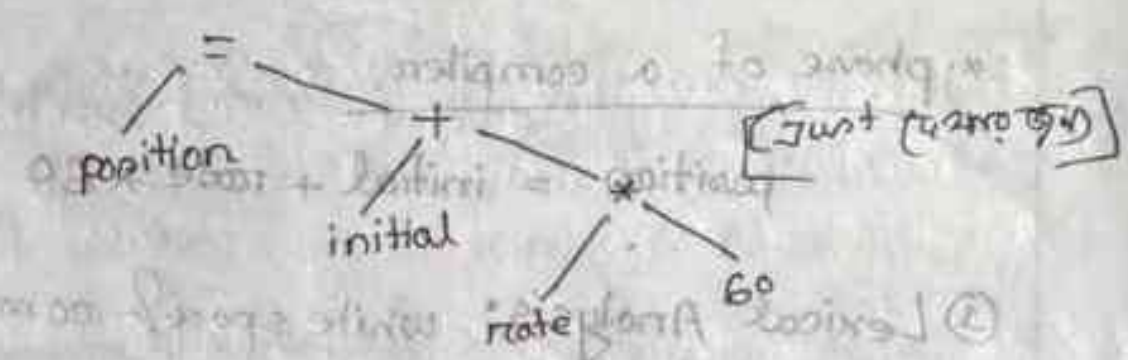
↓ convert

token of (id)

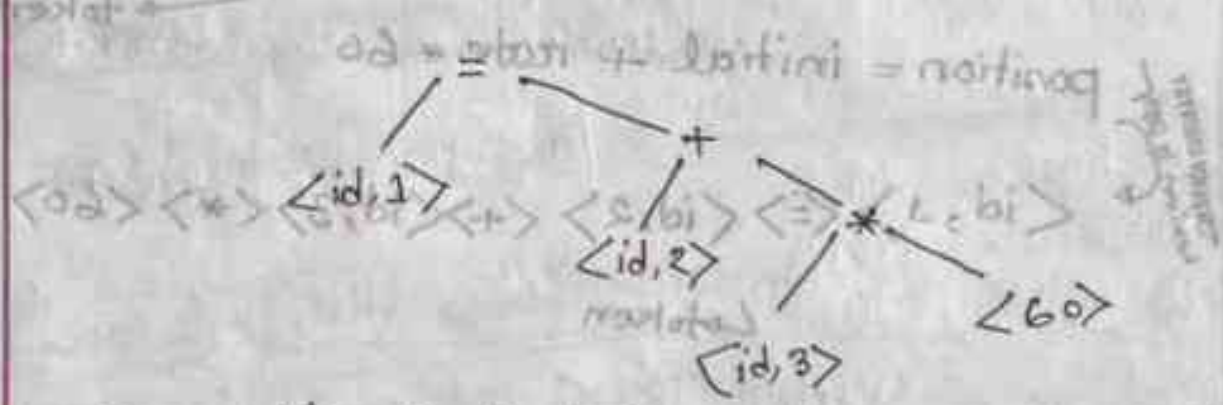
$\langle \text{id}, 1 \rangle$

$\langle \text{id} \rangle$

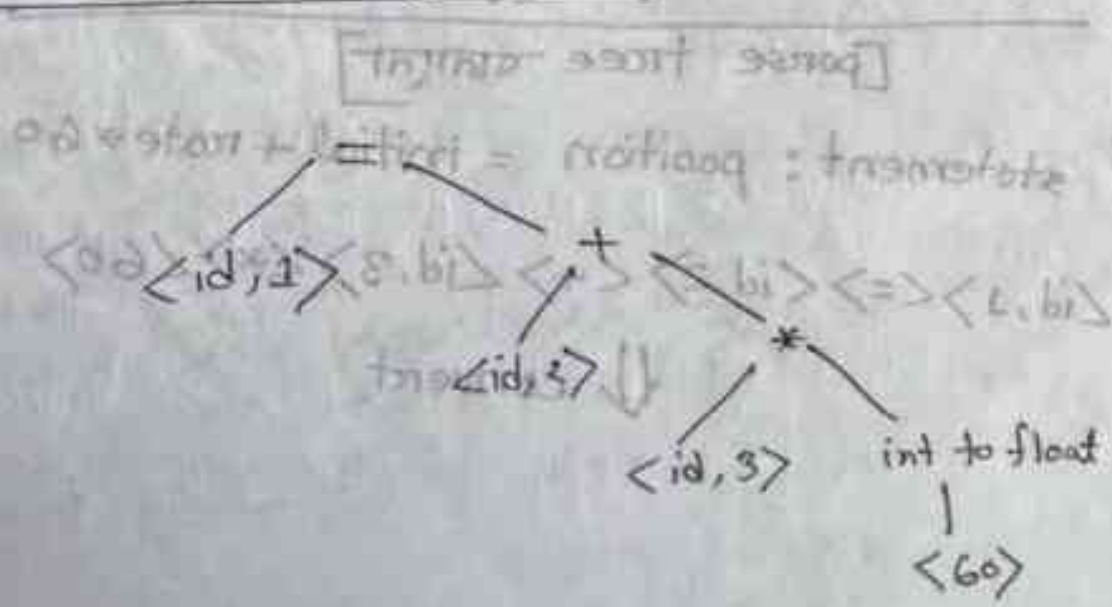
21-05-2022



↓
tokens name, attribute value

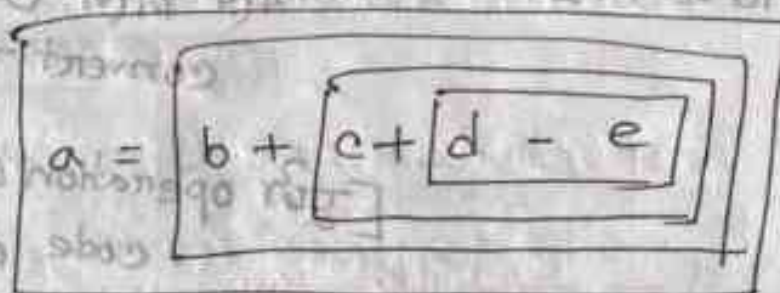


③ Semantic Analysis:



④ Intermediate Code Generation: ③

[3-Address Code] $x \text{ op } y = z$



$$\begin{array}{l|l} t_1 = d - e & t_3 = b + t_2 \\ t_2 = c + t_1 & a = t_3 \end{array} \rightarrow \text{3-Address Code}$$

previous tree (ex):

$$t_1 = \text{int to float}(60)$$

$$t_2 = id_3 * t_1$$

$$t_3 = id_2 + t_2$$

$$id_1 = t_3$$

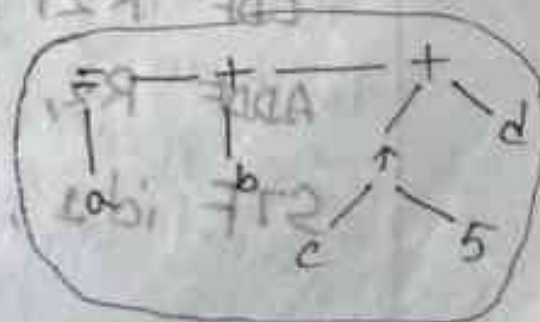
$$c^{\uparrow 5} = c^{\wedge 5} = c^5$$

example:

$$a = b + c^{\uparrow 5} + d$$

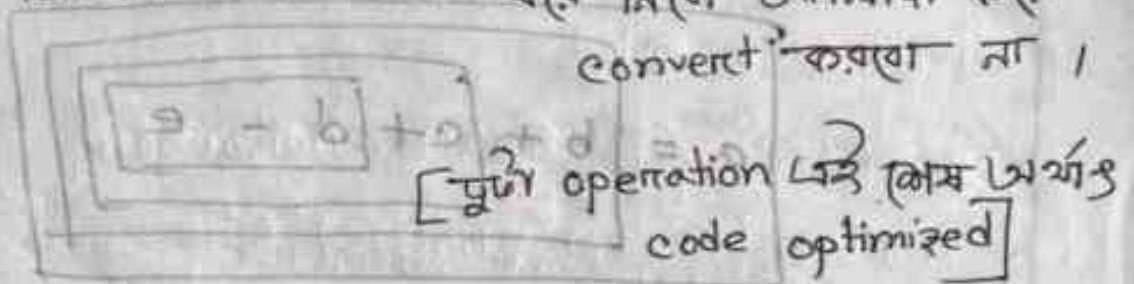
$$\Rightarrow a = b + c * c * c * c * c + d$$

convert into 1, 2, 3, 4 step



⑤ Code Optimization:

$t_1 = id_3 \times 60.0$ → direct int 60 কে float হিসেবে
 $id_1 = id_2 + t_1$ - বিয়ে নিয়ে আলাদা করে convert করতে না।



⑥ Final Code Generation:

POWF [power এর জন্য]

LDF → load factor
 ADDF → যোগ
 MULFi → গুণ
 DIVFi → ভাগ
 STFi → store
 POWF → পাওয়ার

LDF R1, id3

MULF R1, R1, #60.0 [# কনস্ট্যান্ট constant]

LDF R2, id2 [R2 = id2]

ADDF R2, R2, R1 [R2 = R2 + R1]

STF id1, R2 [id1 = R2]

* Error Handling : কোড লিখার সময় কোন
error দেখা দিলে সেটা control করে। সবসময়
আমরা error handler যুক্ত থাকি।

1. Example:

$a = b + c * 50$

[Statement to 6th phase is
pass করা হয়েছে]



$a = b + c * 50$ [lexical analysis & extra
space remove করে দিও]

* Types of Error:

- Lexical
- Syntactical
- Semantical
- Logical

[Slide: Token, Pattern
and Lexemes]

* Error Recovery Strategies:

→ panic Mode, 1/0

→ statement error: পারসার নিজ (থাক) error
দূর কর চেষ্টা।

error productions: user যখন error করে

- পারসার অনুসারে একটি function set

- ব্যবসায়,

global correction: parser nearest possible.

panic mode: is a simple & commonly used error recovery strategy in compiler design, particularly within the parsing phase. When a syntax error is detected, the parser enters "panic mode" and discards input symbols one by one until it encounters a "synchronizing token".

Global correction mode: in compiler design is a theoretical error recovery strategy that aims to minimize the changes required to transform an incorrect input string into a syntactically correct one.

int age = 25;
Lexeme: Actual text from the source program
int, age, =, 25, ;
tokens: Symbolic representation: <int>, <age>

Youtube class

* Introduction to compiler:

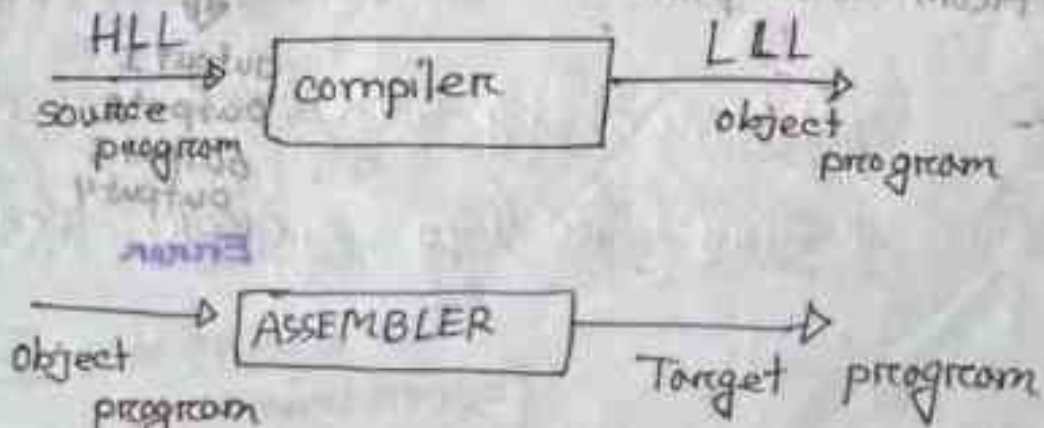
→ A compiler is a translator that converts the High Level Language into the machine language.

[HLL → developer/programmer]

[Machine L → processor]

→ Compiler is used to show errors to the programmer

→ The main purpose of compiler is to change the code written in one language without changing the meaning of the program.



[High Level Language: c, c++, java]

run faster

→ programming language
← Compiler [c, c++]

Takes all the code in one go

↓
Converts Code in Machine Lang. (ML)

↓
Gives to System for processing

↓
shows Output with users

slow, python

← Interpreter

Takes one instruction at time

↓
Converts First instruction to ML

↓
Gives to system for processing & generate output

↓
Converts Second instruction

* In Case of Error - Interpreter:

If Error

↓
Stop Execution
From that point

program

Line 1

Line 2

Line 3

Line 4

Error

↓

output 1

output 2

output 3

output 4

Error

compiler

ASSEMBLER

target

* In Case of Error - Compiler:

If Error

program won't
get executed

program

line 1

line 2

line 3

line 4

Error

No Output

[If there is an error in a program, the program will not run.]

* Hi.cpp

```
void main()
{
    cout << "hi";
}
```

source file

Compiler

101010100 Hi.exe

```
1101110011
01100110
00011111
11000111
```

Machine code

```
* main()
{
    printf("Hello World");
}
```

Ans.

0101110011100

111001110111

[This code is ML4
convert to ML4]

* Interpreter: এটিও compiler এর মতো HLL ভাষাকে
 মেশিন ভাষায় রূপান্তর করে। তবে কম্পাইলার যেমন প্রথমে
 সোর্স প্রোগ্রামকে তাৎক্ষণিক প্রোগ্রাম রূপান্তর করে এবং
 তারপর ফলাফল প্রদান করে কিন্তু interpreter সোর্স
 প্রোগ্রামকে তাৎক্ষণিক প্রোগ্রাম রূপান্তর করে না, ~~এ~~
 interpreter লাইন নির্ধার করে এবং আন্তর্ভুক্তিকালীন
 প্রদর্শন করে।

[লাইন বই লাইন execute করে]

Source Program $\xrightarrow{\text{Interpreter}}$ Result

* main() {

> 0101001001010101

printf("Hello World");
 01100110
 > 01010111011101101101
 1110011

main() {
 printf("Hello World");
 }

> 010101

[statement by statement convert করে]

[অমসু বসি কিন্তু error করে]

definition:

An interpreter translates code written in a high-level programming language into machine code line-by-line as the code runs.

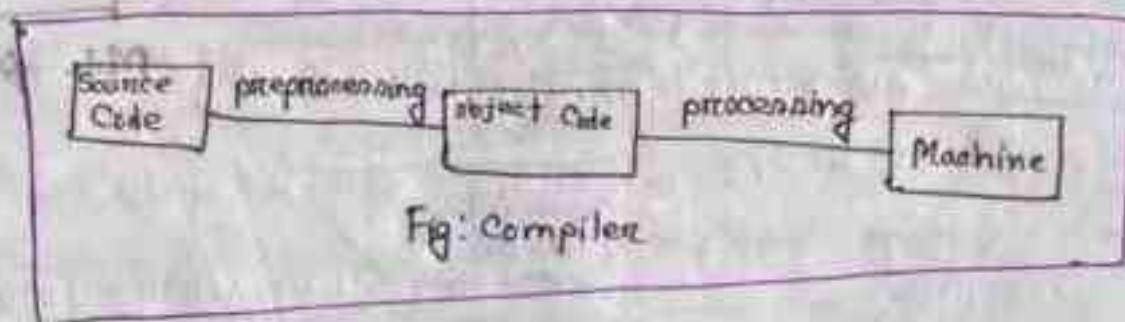
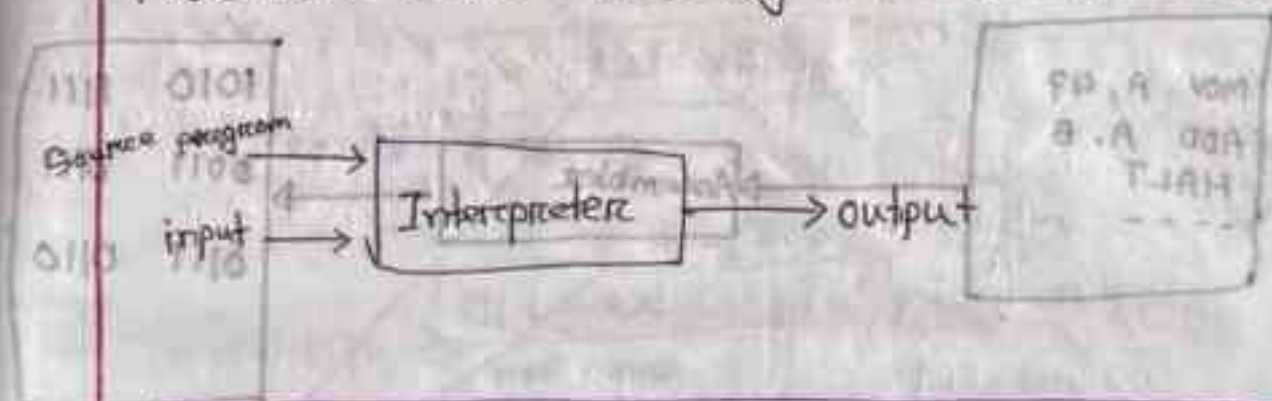


Fig: Compiler

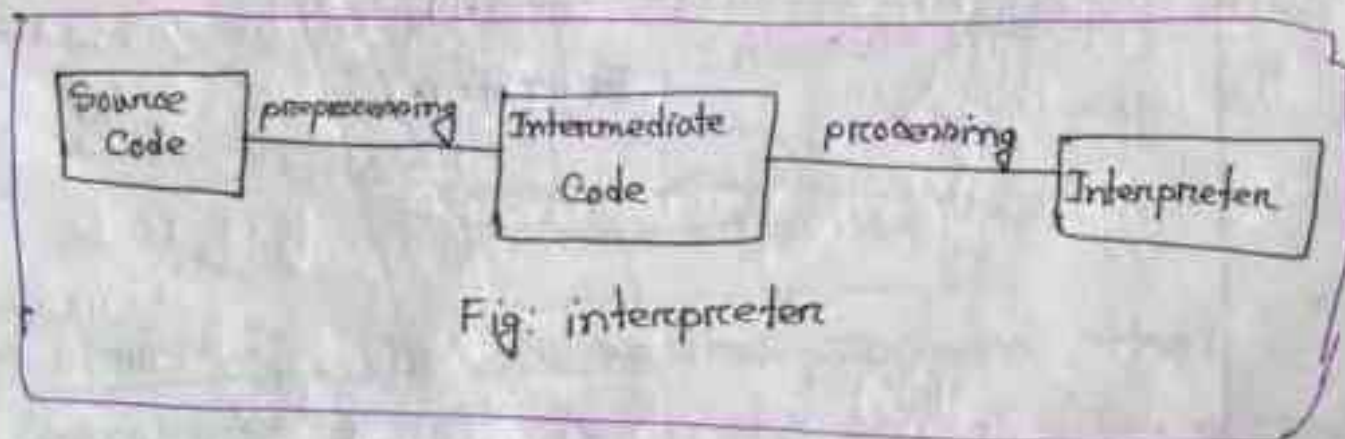


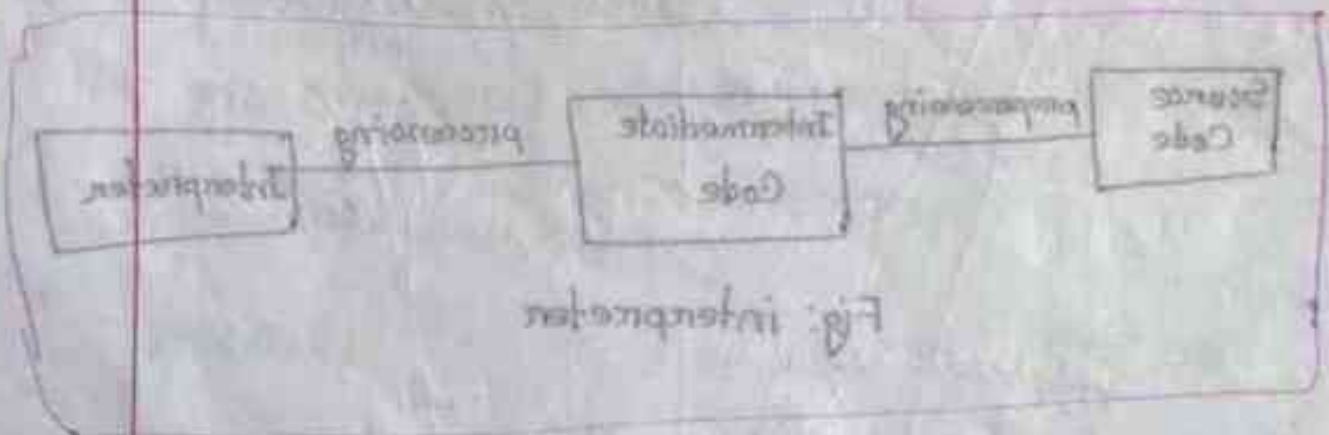
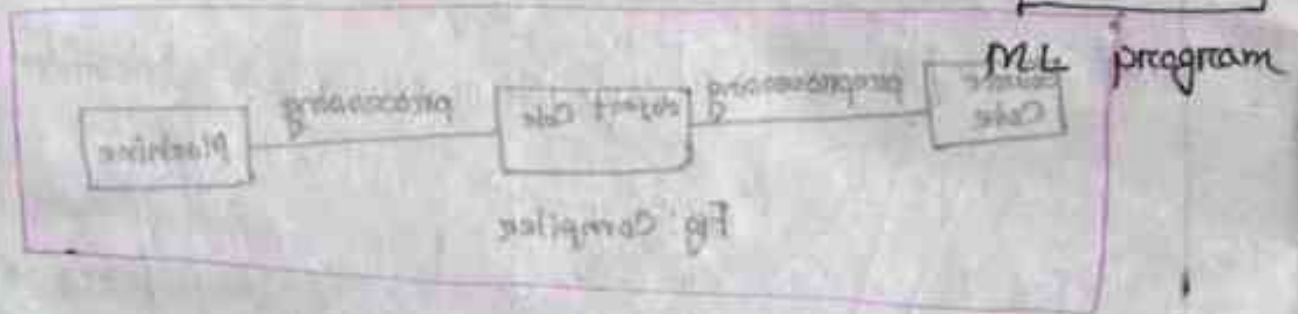
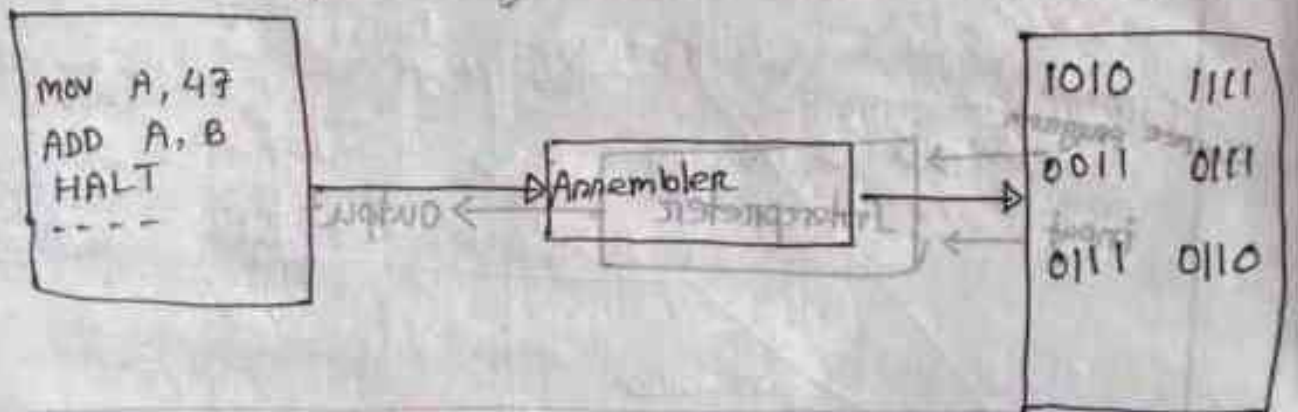
Fig: interpreter

→ Compiler is used in programming languages like C, C++, C#, Scala etc.
[C# = C sharp]

→ Interpreter used: PHP, Ruby, python, perl etc.

Assembler: ଉଚ୍ଚତର ସ୍ତରୀୟ ଲିଖିତ ଆକ୍ରମଣ ଲିଖିତ ଆକ୍ରମଣ
 ଅନୁବାଦ କର ।

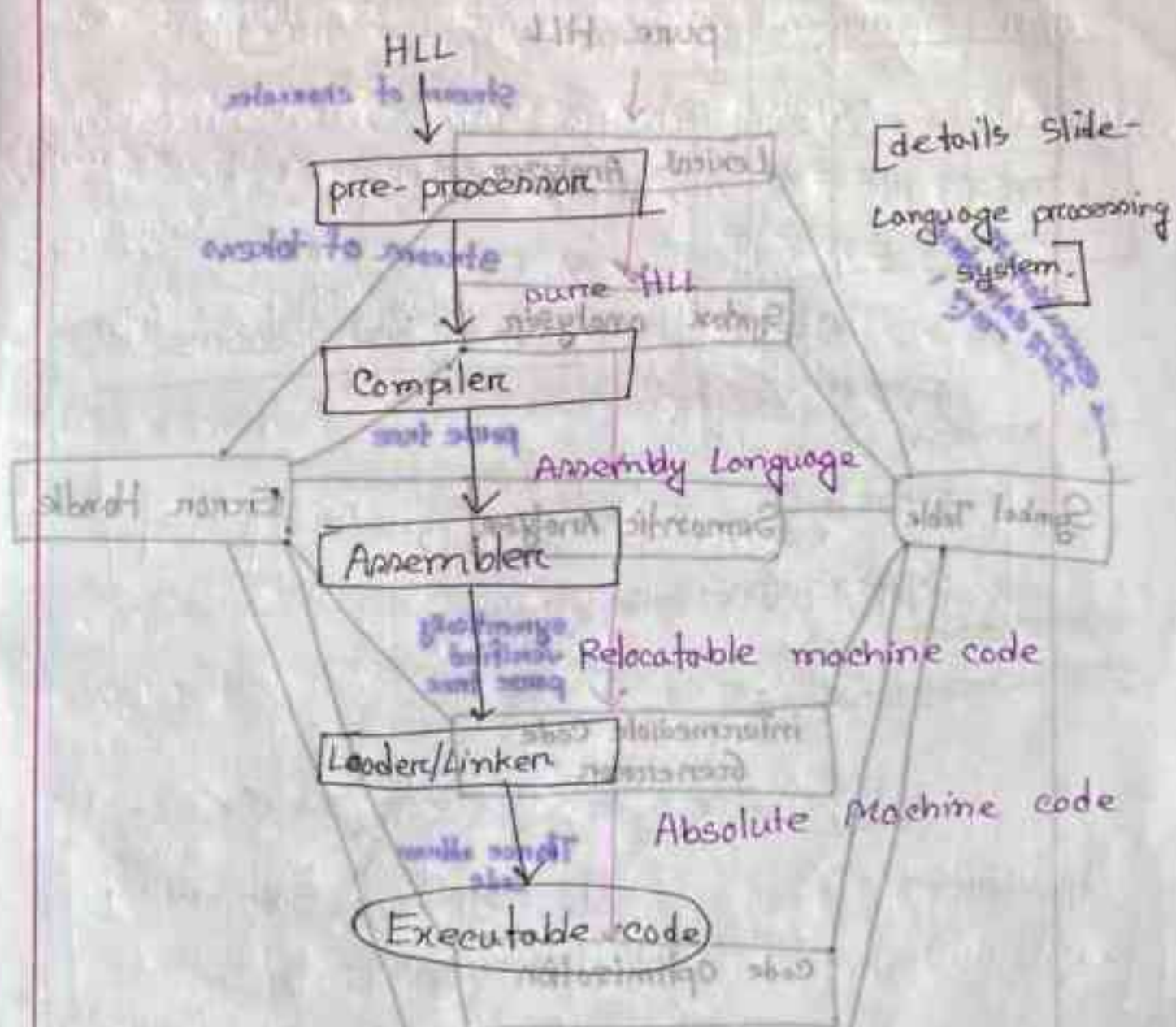
Assembly language



Compiler is used to convert high level language code into machine code.
 [e.g. C, C++, etc.]

Interpreter is used to convert high level language code into machine code.
 [e.g. Python, Java, etc.]

* Language processing System:

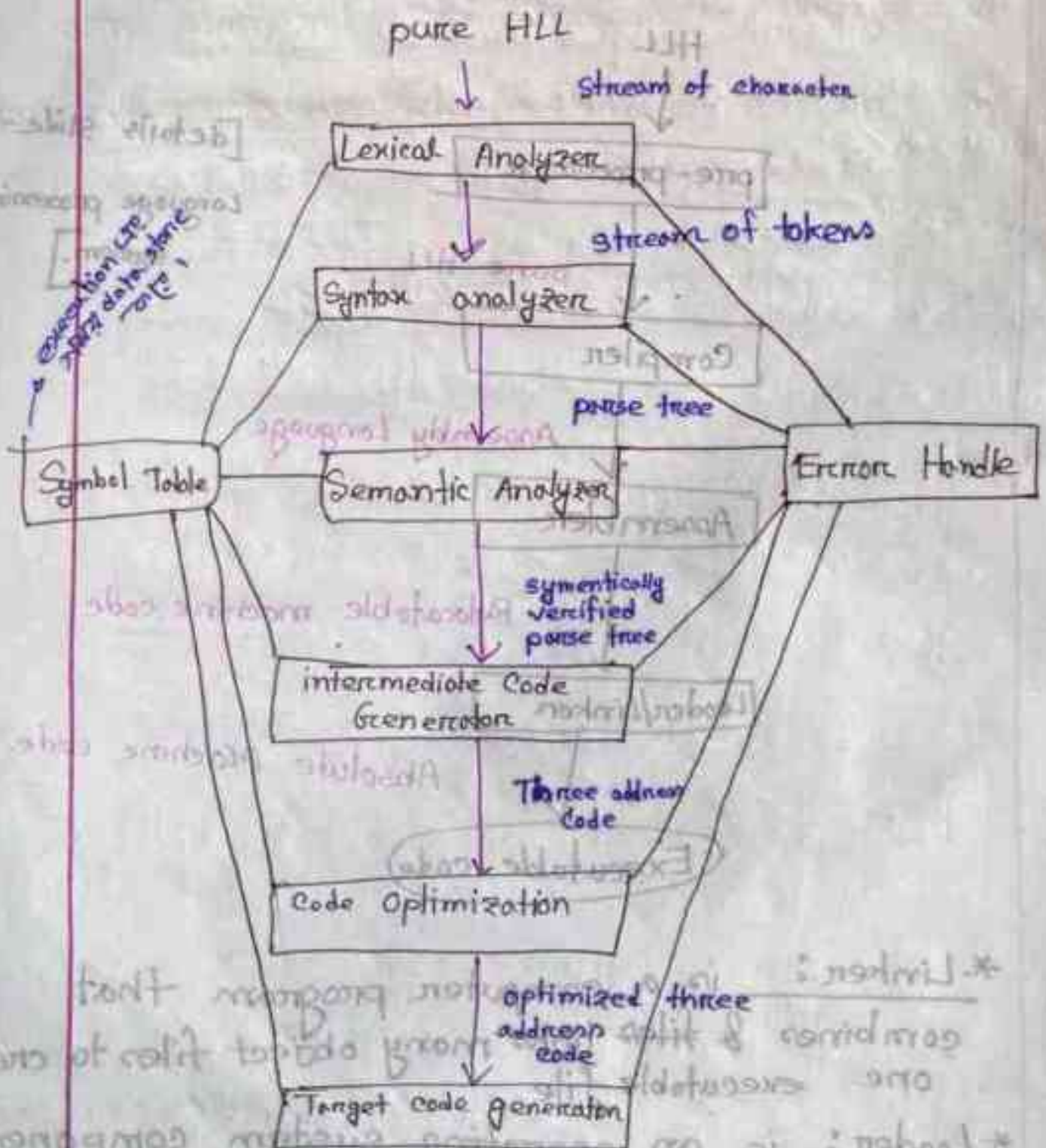


* Linker: is a computer program that combines & files links many object files to create one executable file.

* Loader: is an operating system component that loads & executes executable files into the main memory.

Farhan Hassan

phases of compiler:



* pure HLL:

int x, a, b, c;

x = a + b * c;

Symbol Table

S.No	Variable	Type
1	x	int
2	a	int
3	b	int
4	c	int

① Lexical Analyzer:

→ Lexeme: a specific instance of a token.

→ Return करके tokens

→ जिसका extra space था उसे remove करके छोड़ देना

$\langle id, 1 \rangle = \langle id, 2 \rangle + \langle id, 3 \rangle * \langle id, 4 \rangle$

x: identifier ①

→ tokens.

= : Assignment operator

a: identifier ②

+ : Addition operator

b: identifier ③

*: Multiplication operator

c: identifier ④

② Syntactic Analyzer (Syntax Analyzer):

Symbol	Value	Index
id	x	1
operator	+	2
Grammar:		
$S \rightarrow id = E$		
$E \rightarrow E + T \mid T$		

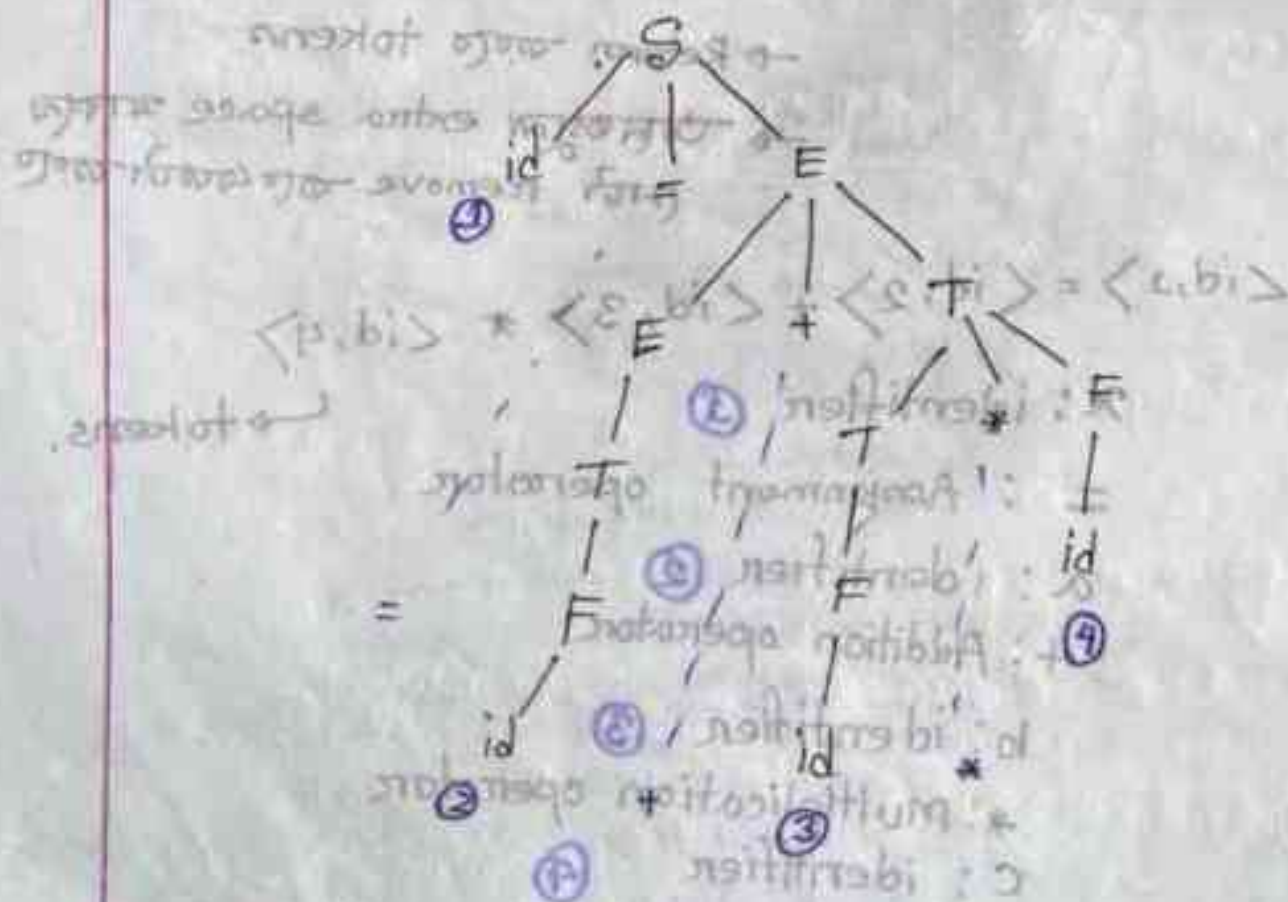
$T \rightarrow T * F \mid F$
 $F \rightarrow id$

[Return a parse tree]

$\langle id, 1 \rangle = \langle id, 2 \rangle + \langle id, 3 \rangle * \langle id, 4 \rangle$

$$x = a + b * c$$

Lexical Analyzer ①

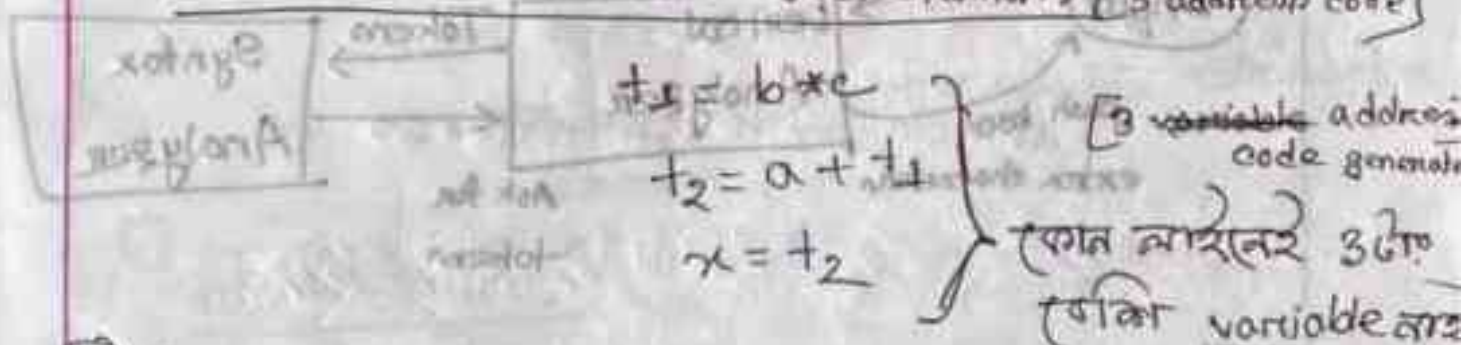


③ Symantic Analyzer:

Symantically verified
parse tree

[type অনুযায়ী declaration check, একটি variable কে আগে declare করতে দিবে না এবং - চাঞ্চল্য, অর্থাৎ previous tree গুলি declare করে।]

④ Intermediate Code Generation: [3 address code]



⑤ Code Optimization:

$t_1 = b * c$
 $x = a + t_1$ } optimized 3-address code

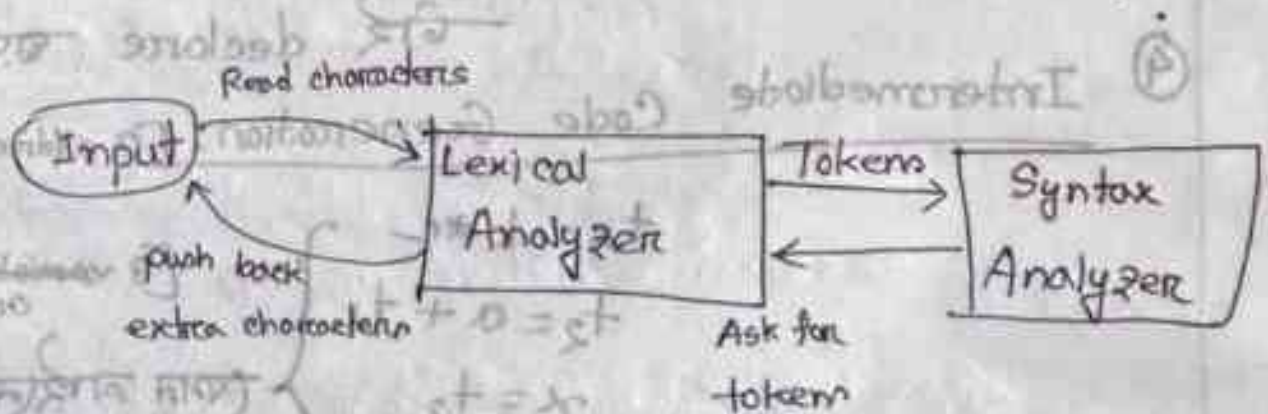
⑥ Target Code Generation:

```
MUL AL, BL
ADD CL, AL
MOV DL, CL
```

Lec-4:

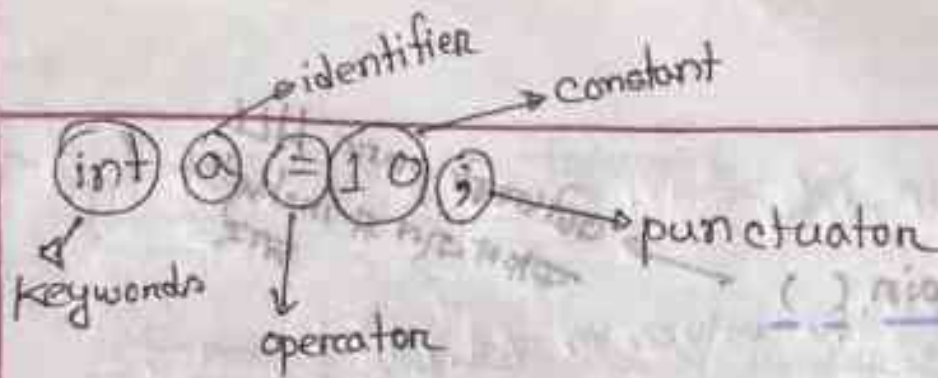
* Lexical Analyzer: It is the first phase of compiler also known as scanner. It converts pure HLL into a sequence of tokens.

Lexical Analyzer can be implemented with the Deterministic finite Automata (DFA)



Token: is a sequence of characters that can be treated as a unit in the grammar of the programming language.

⇒ keywords [int, if, else] predetermined	⇒ Constants
⇒ Identifier [user define] variable, constant	⇒ Punctuators [brackets, comma]
⇒ Operator	



blank, ~~tab~~, ~~new~~

Steps: blank, new line, comments → ~~token~~

How works Lexical analysis:

① Input Preprocessing:

white space ~~remove~~, comments
~~delete~~ extra character
 remove.

② Tokenization:

→ Sequence of tokens a
 convert.

③ Token classification:

④ Token validation:

⑤ Error Detection:

```
int main ( )
{
```

```
// Variable declaration
```

```
int a, b;
```

Q = 10 ;

return 0

7

example:

$$\text{position} = \text{initial} + \text{rate} * 60$$

$\langle id, 1 \rangle \times \langle id, 2 \rangle \langle + \rangle \langle id, 3 \rangle \langle * \rangle \langle 60 \rangle$ (14)

Tokens

(iii) token classification:

(iv) taken voluntarily

⑦ Error Detection:

* Count Number of tokens:

```

① int main()
{
    int A = 10, B = 20, sum;
    sum = A + B;
    printf("%d", sum);
    return 0;
}

```

tokens: 16

total: 33 tokens

```

② int primenumber(int number)
{
    int i;
    for(i = 2; i <= num/2; i++)
    {
        if(num % i != 0)
            continue;
        else
            return 1;
    }
    return 0;
}

int main()
{
    int num;
    num = 7;
    res = primenumber(num);
    if(res == 0)
        printf("%d", num);
    else
        printf("%d not prime", num);
}

```

tokens: 48

→ int main () → 4

→ printf ("the values in %d", i) → 7 (1)

→ int a = 20; → 5

→ int b = 10; → 5

→ int a = 5; → 5

→ int a = 5; → 6 [i.e. int]

→ char arr a = 'j'; → 6

→ char a = "xyz"; → 6

→ int (*Farchan *) a = 100; → 5

→ int (*Farchan *) a = 100; → 6

→ int (*abc *) xyz; → 5

→ A = B; → 5

→ A = B + + + + C; → 8

→ A = B - - - - C; → 8

→ A = B * * * * C; → 10



token error:

printf("xyz

→ लाइन कोम्प्लेशन के दौरान
अवधि के बाद फिर भी
complete अवधि error

~~printf("xyz~~

!ab

DFA: int



[Sabab sir are lecture (21-05-2024) ko convert
firar nahi.]

Slide

27-05-2025 (Lab) - Turing

27-05-2025

Compiler



There are two parts of compiler:

1. Analysis part:

2. Synthesis part (code)

Phases of a compiler:

Lexical Analyzer: - converts lab symbols into tokens.

- Token is single logical unit that has a meaning in the code.

* A Role of Lexical Analyzer:

Problem:

$$R = 559 \times 334 + 9 : 1129.55$$

Let (R) be constant time + extra type convert into

into convert and print

Ans

111
559

→ keyword, operator,

identifier, etc.

Under
identifiers:

Valid: sum, sum_, sum\$, num10, 210sum, 10sum

Invalid: 10sum, sum\$

problem:

"My = 555 + 334 + 9";

↳ constant print

"int num = 10;

string name = "ABC";

↳ identifier
↳ keyword

int n = 5; (41)

→ monitor
[In 3rd sem. taken class and 1st Home Task]
[2. rule file (5)]

* Context

Context Free Grammar

$$\{V, E, S\} = V \cup E$$

$$\{V, E\} = \Sigma$$

* Context Free Grammar (CFG): $G = (V, E, P, S)$

properties

- A set of Non-terminals (V)
(Generate other symbols)
- A set of tokens, known as terminals, (E)
- A set of Productions (P)
- A start symbol (S)

* Example of CFG:

$$Q \rightarrow Z \mid Q \rightarrow N \mid Q \rightarrow E$$

$$Z \rightarrow 000$$

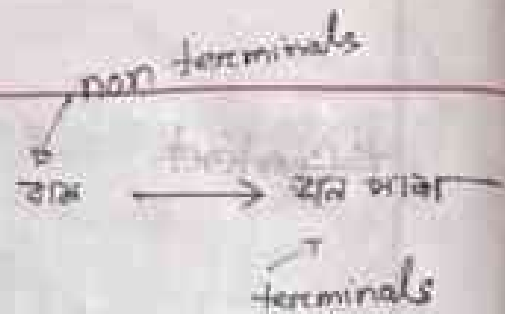
$$N \rightarrow 1Q1$$

$$E \rightarrow \text{Kaan}$$

Give the formal definition of the above CFG.

Soln:

Formal Definition:



$$* V = \{Q, Z, N\} \rightarrow \text{non terminals}$$

$$* \Sigma = \{0, 1\} \rightarrow \text{terminal terminals}$$

Set of Production Rule, $P = \{Q \rightarrow Z, Q \rightarrow N, Q \rightarrow \epsilon,$

$$Z \rightarrow 0Q0, N \rightarrow 1Q1\}$$

Start Symbol, $s = Q$

non terminal Q
 Q $\rightarrow$ Z, N $\rightarrow$ $0Q0, 1Q1$

Q $\rightarrow$ Z, N $\rightarrow$ $0Q0, 1Q1$

(3) Z, N $\rightarrow$ $0Q0, 1Q1$

Another example:

$$E \rightarrow E + T$$

$$T \rightarrow (T)$$

$$T \rightarrow - T \mid 1 \mid 0 \mid \epsilon$$

$$T \rightarrow id$$

Give the formal definition of the given CFG.

Soln:

$$V = \{E, T\}$$

$$\Sigma = \{+, (,), -, id\}$$

[id is terminal symbol
i.e. it cannot be further
divided into smaller parts]

$$P = \{E \rightarrow E + T, T \rightarrow (T), T \rightarrow -T, T \rightarrow id\}$$

$$S = E$$

$\hookrightarrow E$ (non-terminal symbol)

*.

$$\begin{array}{l} E \rightarrow E + T \\ T \rightarrow (T) \\ T \rightarrow -T \\ T \rightarrow id \end{array} \Rightarrow \begin{array}{l} E \Rightarrow E + T \\ T \Rightarrow (T) \\ T \Rightarrow -T \\ T \Rightarrow id \end{array} \Rightarrow \begin{array}{l} E \Rightarrow E + T \\ T \Rightarrow (T) \\ T \Rightarrow -T \\ T \Rightarrow id \end{array} \Rightarrow E \Rightarrow E + T \Rightarrow (E) + T \Rightarrow ((-T)) + id$$

Example format for the above derivation.

*. Derivations : 2 types :

- sequence of production rules, in order
- Left Most Derivation (LMD)
- Right Most Derivation (RMD)

Semantic Frame: $\rightarrow$ offset
20 no. of slide

Example:

$E \rightarrow E + E$
 $\rightarrow E * E$
 $\rightarrow (E)$
 $\rightarrow -E$
 $\rightarrow id$

$(id * id) + id$

$\hookrightarrow$ Derive the input string

Soln:

LMD: $\rightarrow$ $\rightarrow$

parse trees:

$E \rightarrow E + E$

$\rightarrow E * E + E$

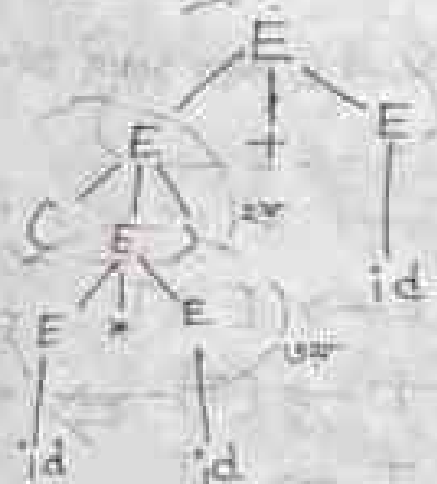
$\rightarrow (E) + E$

$\rightarrow (E * E) + E$

$\rightarrow (id * E) + E$

$\rightarrow (id * id) + E$

$\rightarrow (id * id) + id$



$E \rightarrow (E)$

CFL = parse tree Left to Right

RMD:

$E \rightarrow E + E$
 $\rightarrow E + id$
 $\rightarrow (E) + id$
 $\rightarrow (E * E) + id$
 $\rightarrow (E * id) + id$
 $\rightarrow (id * id) + id$

parse tree



* Another example:

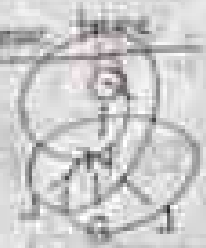
$Q \rightarrow 2$
 $\rightarrow N$
 $\rightarrow E$
 $2 \rightarrow 0Q0$
 $N \rightarrow 1Q1$

Derive 11100111

Sol:

LMD:

$Q \rightarrow N$
 $\rightarrow 1Q1$
 $\rightarrow 11Q11$
 $\rightarrow 111Q111$



$Q \rightarrow N$

$\rightarrow 1Q1$ [$N \rightarrow 1Q1$]

$\rightarrow 1N1$

$\rightarrow 11Q11$

$\rightarrow 11N11$

$\rightarrow 111Q111$

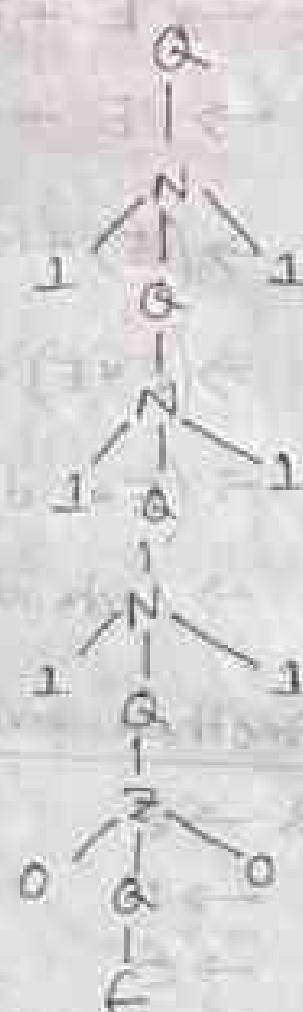
$\rightarrow 1112111$

$\rightarrow 1110Q0111$ [$2 \rightarrow 0Q0$]

$\rightarrow 1110E0111$

$\rightarrow 11100111$

parse tree



LMD (અથવા ડાબા parse tree) અને RMD (અથવા
 ડાબા parse tree some, જાહેર કરવામાં આવેલ
 વાસ્તવમાં પણ દર્શાવે રાખે .

Another example:

$$S \rightarrow SS | (S) | \epsilon$$

Derive: $()()$

Soln:

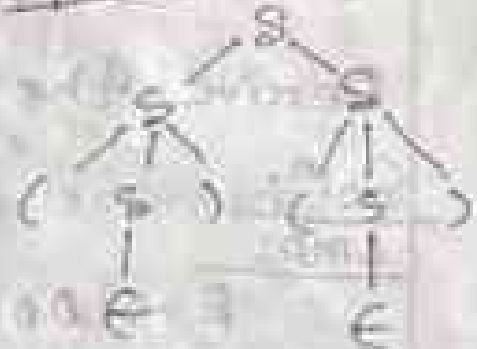
LRD:

$$\begin{aligned} S &\rightarrow SS \\ &\rightarrow (S) S \quad [\text{LRD}] \\ &\rightarrow (\epsilon) S \\ &\rightarrow () S \\ &\rightarrow () (S) \quad [\text{LRD}] \\ &\rightarrow () (\epsilon) \quad [\text{LRD}] \\ &\rightarrow () () \end{aligned}$$

RMD:

$$\begin{aligned} S &\rightarrow SS \\ &\rightarrow S(S) \\ &\rightarrow S(\epsilon) \\ &\rightarrow S() \\ &\rightarrow (S)() \\ &\rightarrow (\epsilon)() \\ &\rightarrow ()() \end{aligned}$$

parse tree



parse tree

x. Another example. ^(***)

$$E \rightarrow DD$$

$$D \rightarrow FJ | CF | aD | cD$$

$$F \rightarrow ab | ad | J$$

$$J \rightarrow \pi a | \pi c | \epsilon$$

Derive the string "abracadra".

Solⁿ:

LDND:

$$E \rightarrow DD$$

$$\rightarrow abD$$

$$\rightarrow FJD [D \rightarrow FJ]$$

$$\rightarrow abJD [F \rightarrow ab]$$

$$\rightarrow abraD [J \rightarrow \pi a]$$

$$\rightarrow abra cD [D \rightarrow cD]$$

$$\rightarrow abra cD [D \rightarrow aD]$$

$$\rightarrow abra cFJ [D \rightarrow FJ]$$

$$\rightarrow abra cadJ [F \rightarrow ad]$$

$$\rightarrow abra cadra [J \rightarrow \pi a]$$

hence,
 $J = J$
work

RMD:

$E \rightarrow DD$

$\rightarrow DeD$

$\rightarrow DeFJ$

$\rightarrow DeFca$

$\rightarrow Deadca$

$\rightarrow FJcadca$

$\rightarrow Frcadca$

$\rightarrow abra cadca$

H-W

problem:

$S \rightarrow ABA/aBA$

$A \rightarrow aBa/bA$

$B \rightarrow bBB/ba$

Derive the string: "abaabbabababaa"

problem:

$S \rightarrow aB/bA$

$A \rightarrow a/aS/bAA$

$B \rightarrow b/bS/aBB$

Derive string baabbabba

→ Parse Tree is a graphical representation of a derivation sequence of a sentential form.

Youtube class - 06

Fachon Hammad

What is grammar?

⇒ In compiler design, grammar refers to the formal rules that define the syntax of a programming language.

It defines the structure of valid sentences or statements in the language.

int a;, a = 100;

⇒ Grammar consists of a set of production rules that describe how valid language constructs can be formed.

A grammar has 4-tuples:

$$G = (V, T, P, S) \quad \text{[Context Free Grammar]}$$

where,

$V \rightarrow$ Variables / Non terminals

$T \rightarrow$ Terminals

$P \rightarrow$ Productions

$S \rightarrow$ Start Symbol.

ex:

$S \rightarrow AB$

$A \rightarrow a$

$B \rightarrow b$

$V = \{S, A, B\}$

$T = \{a, b\}$

$P = \left\{ \begin{array}{l} S \rightarrow AB \\ A \rightarrow a \\ B \rightarrow b \end{array} \right\}$

$S = \{S\}$

Let 03:

What is Language?

→ In compiler design the language of a grammar refers to the set of all possible strings that can be generated by that grammar.

Grammar (G) is defined by

$G = (V, T, P, S)$

where,

$V \rightarrow$ Variables / Non terminals

$T \rightarrow$

$P \rightarrow$

$S \rightarrow$

Language ($L(G)$): The language of a grammar G is the set of all strings composed the terminal symbols that can be derived from the start symbol S by applying the production rules in P .

Two types language: ~~inte~~ ~~finite~~ ~~infinite~~

→ finite
→ infinite

ex: [finite]

$S \rightarrow AB$

$A \rightarrow a$

$B \rightarrow b$

$G = (V, T, P, S)$

$V = \{S, A, B\}$

$S = \{S\}$

$P = \left\{ \begin{array}{l} S \rightarrow AB \\ A \rightarrow a \\ B \rightarrow b \end{array} \right\}$

$L(G) = \{ab\}$

$L(G)$ mon. $\{ab\}$ possible $\{a\}$ $\{b\}$ generate

Ex: [infinite]

Grammar

$$S \rightarrow aS/e$$

$$L(\infty): \{ \epsilon, a, aa, aaa, \dots \}$$

$$\{ \epsilon, a, a^2, a^3, \dots, a^n \}$$

↳ એકમ દ્વારા બનેલ શબ્દ

• Leftmost Derivation (LMD): In a leftmost derivation at each step, the left most non-terminal in the current string is replaced one of the production rules.

Ex:

$$E \rightarrow E + T \mid T$$

$$T \rightarrow T * F \mid F$$

$$F \rightarrow (E) \mid id$$

Deriving the string $id + id * id$
using the leftmost derivation

Scanned with
CamScanner

Deriving the string id + id * id
using the rightmost derivation.

$$\begin{aligned}
 F &\rightarrow (E) / id \\
 T &\rightarrow T * F / F \\
 F &\rightarrow F + T / T
 \end{aligned}$$

ex:

Right most Derivation [RMD]

$$\begin{aligned}
 &F \rightarrow E + T \\
 &\rightarrow T + T \\
 &\rightarrow F + T [T \Rightarrow F] \\
 &\rightarrow id + T [T \Rightarrow id] \\
 &\rightarrow id + T * F [T \Rightarrow T * F] \\
 &\rightarrow id + F + F [T \Rightarrow F] \\
 &\rightarrow id + id * id
 \end{aligned}$$

Soln

Solⁿ:

$$E \rightarrow E + T$$

$$\rightarrow E + T * F$$

$$\rightarrow E + T * id$$

$$\rightarrow E + id * id$$

$$\rightarrow id + id * id$$

Ex:

$$E \rightarrow E + E \mid E * E \mid id$$

deriving the string: $id + id * id$, using RMD.

Solⁿ:

$$E \rightarrow E + E$$

$$\rightarrow E + E * E$$

$$\rightarrow E + E * id$$

$$\rightarrow E + id * id$$

$$\rightarrow id + id * id$$

Example:

$$E \rightarrow E + T \mid E - T \mid T$$

$$T \rightarrow T * F \mid T / F \mid F$$

$$F \rightarrow (E) \mid id$$

Deriving the string: $id * (id + id)$ using

LMD & RMD

L.M.D

$$E \Rightarrow T \quad [E \Rightarrow T]$$

$$\rightarrow T * F \quad [T \Rightarrow T * F]$$

$$\rightarrow F * F \quad [T \Rightarrow F]$$

$$\rightarrow id * F \quad [F \Rightarrow id]$$

$$\rightarrow id * (F) \quad [F \Rightarrow E]$$

$$\rightarrow id * (E + T) \quad [E \Rightarrow E + T]$$

$$\rightarrow id * (T + T) \quad [E \Rightarrow T]$$

$$\rightarrow id * (F + T)$$

$$\rightarrow id * (id + T)$$

$$\rightarrow id * (id + F)$$

$$\rightarrow id + (id + id)$$

R.M.D. L.M.D. are same (only name diff)

$$E \Rightarrow T$$

$$\rightarrow T * F \quad [E \Rightarrow T * F]$$

$$\rightarrow T * (E) \quad [F \Rightarrow (E)]$$

$$\rightarrow T * (E + T) \quad [E \Rightarrow E + T]$$

$$\rightarrow T * (E + id)$$

$$\rightarrow T * (T + id)$$

$$\rightarrow T * (F + id)$$

$$\rightarrow T + (id + id)$$

$$(Ans) \rightarrow id + (id + id)$$

• Parse Tree

ex: $E \rightarrow E + E \mid E * E \mid id$

String: $id + id * id$

Solⁿ:

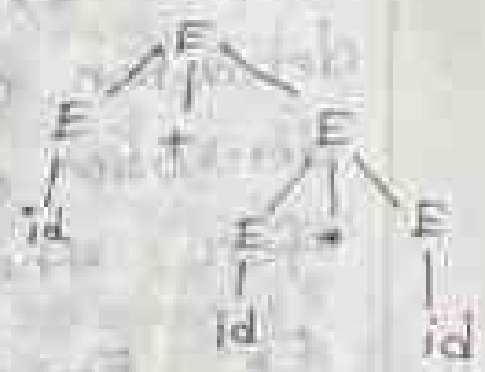
L.M.D: $E \rightarrow E + E$

$\rightarrow id + E$

$\rightarrow id + E * E$

$\rightarrow id + id * E$

$\rightarrow id + id * id$



R.M.D:

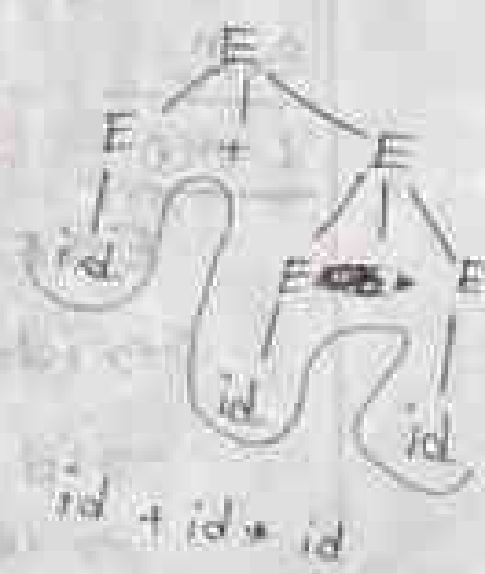
$E \rightarrow E + E$

$\rightarrow E + E * E$

$\rightarrow E + E * id$

$\rightarrow E + id * id$

$\rightarrow id + id * id$



* Ambiguous Grammar:

A Grammar is said to be ambiguous if

there exist more than one leftmost derivation or more than one rightmost

derivation or more than one parse tree

for the input string.

Ex: $E \rightarrow E + E \mid E * E \mid id$

String: $id + id * id$

Soln
L.M.D

$E \rightarrow E + E$	$E \rightarrow E * E$
$\rightarrow id + E$	$\rightarrow id + id * E$
$\rightarrow id + id * E$	$\rightarrow id + id * id$
$\rightarrow id + id * id$	

LMD (Leftmost Derivation) is given string (Left to Right) and
RMD (Rightmost Derivation) is given string (Right to Left)

OR,
P.D.O. (another way to check)

$E \rightarrow E + E$	$E \rightarrow E * E$
$\rightarrow E + E + E$	$\rightarrow E * id$
$\rightarrow E + E * id$	$\rightarrow E + E * id$
$\rightarrow E + id * id$	$\rightarrow E + id + id$
$\rightarrow id + id + id$	$\rightarrow id + id * id$

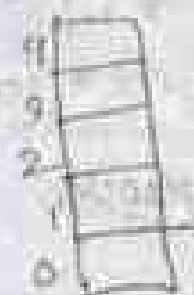
So, This grammar is ambiguous.

24-06-2025 (Lab)

parenthesis

Balance

stack



Balance hai

→ odd length hai

→ opening aur closing

hai aur closing mila kar

$n = \text{stack}(\text{str})$

if $(n \% 2 == 1) \parallel \text{str}[0] == '(' \parallel$

$\text{str}[0] == '[' \parallel \text{str}[n-1] == ')' \parallel$

unbalance

else

$\{ \text{for } (i = 0; i < n; i++) \{ \text{if } (\text{str}[i] == '(' \parallel \text{str}[i] == '[') \{ \text{push}(\text{str}[i]); \} \}$

$\text{pop}(); \}$

$\text{if } (\text{str}[i] == ')' \parallel \text{str}[i] == ']') \{ \text{if } (\text{stack}() == 0) \{ \text{return } 0; \}$

comments ko diya hai kya consider

karke hai, kya code

02-07-2025

Ambiguous Grammar / Ambiguity: एक ही LMD/parse

• द्वारा represent कर सकते / एक ही वाक्य के parse tree

$E \rightarrow E + E$

$\rightarrow E * E$

$\rightarrow (E)$

$\rightarrow id$

$\rightarrow id * (E)$

LMD-01

$E \rightarrow E + E$

$\rightarrow id + E$

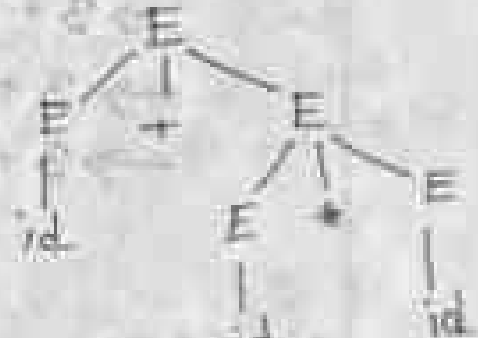
$\rightarrow id + E * E$

$\rightarrow id + id * E$

$\rightarrow id + id * id$

Derive the following string:

$id + id * id$



LMD-02

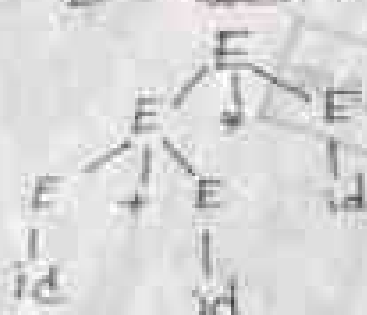
$E \rightarrow E * E$

$\rightarrow E + E * E$

$\rightarrow id + E * E$

$\rightarrow id + id * E$

$\rightarrow id + id * id$



ambiguous grammar no problem for?

* $E \rightarrow E + E$

$\rightarrow E * E$

$\rightarrow (E)$

$\rightarrow -E$

$\rightarrow id$

$id \rightarrow 3 \mid 2$

LMD 01:

$E \rightarrow E + E$

$\rightarrow id + E$

$\rightarrow 3 + E + E$

$\rightarrow 3 + id * E$

$\rightarrow 3 + 2 * id$

$\rightarrow 3 + 2 * 2$

LMD 02:

$E \rightarrow E * E$

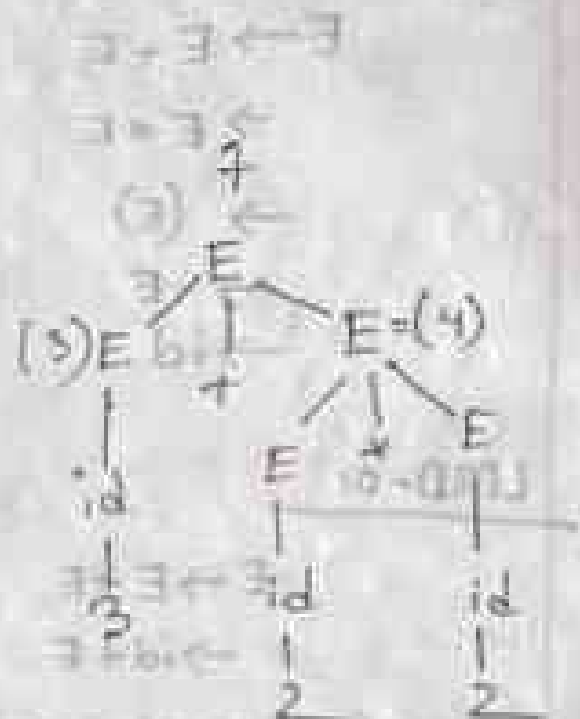
$\rightarrow E + E$

$\rightarrow id + E * E$

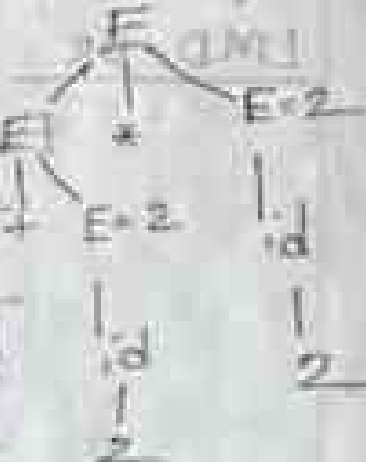
$\rightarrow 3 + id * E$

$\rightarrow 3 + 2 * id$

$\rightarrow 3 + 2 * 2$



(10) X



LMD 01 & LMD 02

(by giving priority or precedence in operators also we can use parenthesis) → to convert unambiguous from ambiguous

To make unambiguous, just give priority.

Unambiguous Grammar:

slide

ex: (previous ex (not solve))



$$E \rightarrow E + T \leftarrow A$$

$$\rightarrow T + T$$

$$\rightarrow F + T$$

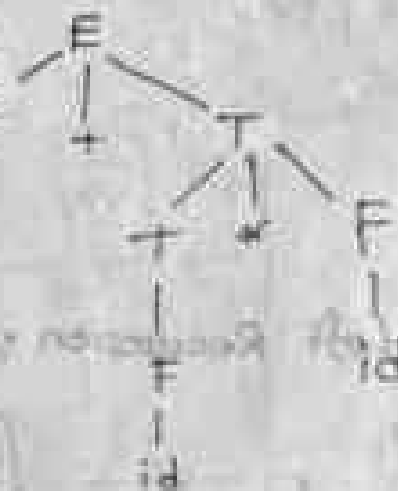
$$\rightarrow id + T$$

$$\rightarrow id + T * F$$

$$\rightarrow id + F * F$$

$$\rightarrow id + id * F$$

$$\rightarrow id + id * id$$



$$A \leftarrow A$$

$$A \leftarrow A$$

$$A \leftarrow A$$

Left Recursion:

→ infinite looping problem.

↳ left & right left recursion

$$A \rightarrow A\alpha \mid \beta$$

$$[A \rightarrow A\alpha \text{ वा } A \rightarrow \beta]$$

$$A \rightarrow A\alpha$$

$$A \rightarrow \beta + \epsilon \leftarrow \epsilon$$

$$T + T \leftarrow$$

$$T + \epsilon \leftarrow$$

$$T + b \leftarrow$$

$$T + b \leftarrow$$

$$T + b \leftarrow$$

$$T + b \leftarrow$$

$$T + b \leftarrow$$

$$\begin{aligned} A &\rightarrow \alpha A \mid \beta \\ \cancel{A &\rightarrow \alpha A} \\ A &\rightarrow \alpha A \\ A &\rightarrow \beta \end{aligned}$$

↳ left recursive

ना हटाया जाये

↳ हटाया जाये left

recursion

ex:

$$A \rightarrow A\alpha \mid \beta$$

After eliminating Left Recursion:

$$A \rightarrow \beta A'$$

$$A' \rightarrow \alpha A' \mid \epsilon$$

$$[\beta \text{ प्रत्येक left recursion को हटाने के लिए } A' \text{ का उपयोग करें}]$$

recursion को

हटाने के लिए

उपयोग करें

उपयोग करें

formula

$$\begin{aligned}
 A &\rightarrow A\alpha_1 \mid A\alpha_2 \mid B_1 \mid B_2 \\
 A &\rightarrow B_1 A' \mid B_2 A' \\
 A &\rightarrow \alpha_1 A' \mid \alpha_2 A' \mid \epsilon \mid b_1 \mid b_2
 \end{aligned}$$

Immediate Left Recursion Elimination:

Example:

$$\begin{aligned}
 E &\rightarrow E + T \mid T \\
 T &\rightarrow T * F \mid F \\
 F &\rightarrow (E) \mid id
 \end{aligned}$$

$$\frac{E}{A} \rightarrow \frac{E}{A} + \frac{T}{A} \mid \frac{T}{A}$$

$$\begin{aligned}
 E &\rightarrow TE' \\
 E' &\rightarrow +TE' \mid \epsilon
 \end{aligned}$$

$$\frac{T}{A} \rightarrow \frac{T}{A} * \frac{F}{A} \mid \frac{F}{A}$$

$$\begin{aligned}
 T &\rightarrow FT' \\
 T' &\rightarrow *FT' \mid \epsilon
 \end{aligned}$$

$$F \rightarrow (E) \mid id$$

More than one step Left Recursion Elimination:

$$S \rightarrow Af | b \quad [\text{Left Recursion}]$$

$$A \rightarrow Ae | Sd | Be \quad [\text{Left Recursion}]$$

$$B \rightarrow Ag | Sh | k$$

Initial Step: $B \rightarrow Ag \rightarrow BAg$ $B \rightarrow Sh \rightarrow BSh$

$$S \rightarrow Af | b$$

$$A \rightarrow Ae | Sd | Be$$

$$A \rightarrow Ae | (Af | b)d | Be \quad \leftarrow$$

$$A \rightarrow Ae | Afd | bd | Be$$

$$A \rightarrow bdA' | BeA'$$

$$A' \rightarrow eA' | fdA' \quad \leftarrow$$

$$B \rightarrow Ag | Sh | k$$

$$B \rightarrow (bdA' | BeA')g | (Af | b)h | k$$

$$B \rightarrow bdA'g | BeA'g | Afh | bh | k$$

$$B \rightarrow bdA'g | BeA'g | (bdA' | BeA')fh | bh | k$$

$$B \rightarrow \underline{bdA'g} \mid \underline{BeA'g} \mid \underline{bdA'fh} \mid \underline{BeA'fh} \mid \underline{bh} \mid \underline{k}$$

P_1 P_2 P_3 P_4

□ मिल या left
recursion करे

$$B \rightarrow bdA'g B' \mid bdA'fh B' \mid bh B' \mid k B'$$

$$B' \rightarrow eA'g B' \mid eA'fh B' \mid \epsilon$$

H.W:

Slide are not math

3/10/2025 (Lab)

CFG $\rightarrow$ context free grammar

CFG

pattern

pattern

$(ab)^+ bba$

home task

valid string

abba, bbba, abbbba, aobbaabba

8-07-2025

Left Factoring: ~~mis-match~~ $A \rightarrow \alpha \mid \beta$ $A \rightarrow A$

$$\textcircled{1} A \rightarrow \alpha \beta_1 \mid \alpha \beta_2 \left\{ \begin{array}{l} \alpha \mid \beta \leftarrow A \\ \alpha \mid \beta \mid \beta \leftarrow A \end{array} \right.$$

$$\left\{ \begin{array}{l} A = \alpha A' \\ A' = \beta_1 \mid \beta_2 \end{array} \right.$$

$$\textcircled{2} A \rightarrow \alpha_1 \alpha_2 \alpha_3 \alpha_4 A \mid \alpha_1 \alpha_2 \alpha_3 \alpha_4 \beta \leftarrow A$$

$$\left\{ \begin{array}{l} A \rightarrow \alpha_1 \alpha_2 A' \\ A' \rightarrow \alpha_3 \alpha_4 \beta \mid \alpha_3 \alpha_4 \beta \end{array} \right.$$

$$\textcircled{3} A \rightarrow \text{if expr then stmt else stmt} \mid \text{if expr then stmt}$$

$$A \rightarrow \text{if expr. then stmt } A'$$

$$A' \rightarrow \text{else stmt} \mid \epsilon$$

$$\textcircled{4} A \rightarrow \beta_1 \alpha_1 \alpha_2 \mid \beta_2 \alpha_1 \alpha_2$$

$\hookrightarrow$ (can't be left factored.)
mismatch

$$(7) A \rightarrow abB | aB | cdg | cdeB | edfB$$

$$\left\{ \begin{array}{l} A \rightarrow aA' | cdA'' \\ A' \rightarrow bB | B \\ A'' \rightarrow g | eB | fB \end{array} \right. \begin{array}{l} \text{Group 1: } A \rightarrow aA' \text{ and } A' \rightarrow bB | B \\ \text{Group 2: } A \rightarrow cdA'' \text{ and } A'' \rightarrow g | eB | fB \end{array}$$

$$(8) A \rightarrow ad | a | ab | abc | b$$

$$A \rightarrow aA' | b \quad A' \rightarrow d | \epsilon | b | bc \quad A'' \rightarrow \epsilon | c$$

$$A' \rightarrow d | \epsilon | b | bc$$

$$A' \rightarrow d | \epsilon | bA''$$

$$A'' \rightarrow \epsilon | c$$

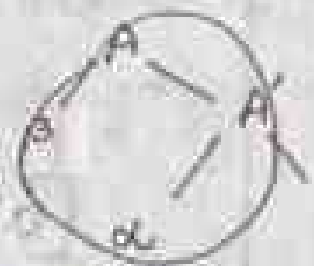
$$\left\{ \begin{array}{l} A \rightarrow aA' | b \\ A' \rightarrow d | \epsilon | bA'' \\ A'' \rightarrow \epsilon | c \end{array} \right.$$

$$(9) A \rightarrow abB | aB | cdg | cdeB | edfB$$

* $A \rightarrow A\alpha / \beta$ is called **Right Recursive**

$A \rightarrow \beta A'$

$A' \rightarrow \alpha A' / \epsilon$



Left Recursion is a serious problem:

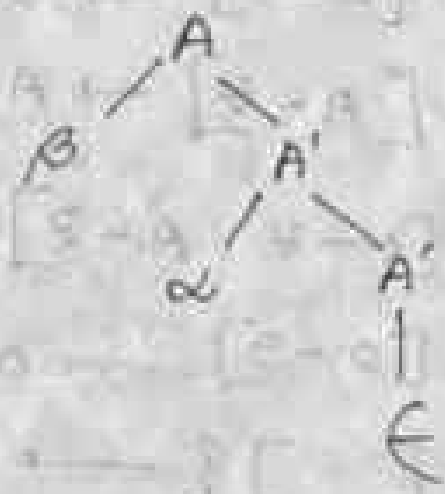
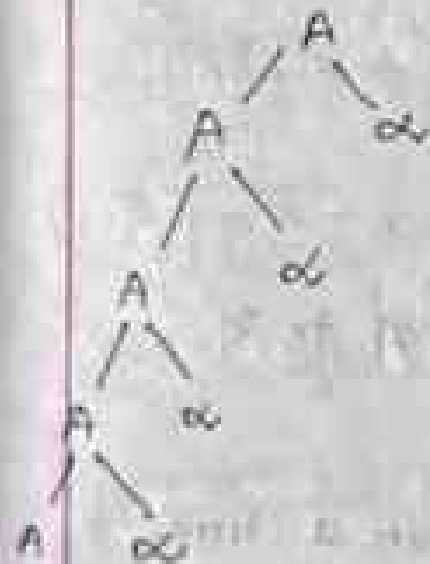
[Important for exam]

* $A \rightarrow A\alpha / \beta$

$A \rightarrow \beta A'$

$A' \rightarrow \alpha A' / \epsilon$

$\beta\alpha$
Left-recursive



[Left Recursion is]

[Infinite loop]

15-03-2025 (Lab) 2/2A → A

1. Regular Expression (REGEX):
 $xy \rightarrow A$
 $xy \rightarrow A$

Tokenization for lexical analysis

[abc] → a, b or c

[a|b|c]

[^abc] → any character except a, b or c

[a-b] → any character

[a-z] → a to z

[A-Z] → A to Z

[a-zA-Z] → a to z, A to Z

[0-9] → 0 to 9

[] → Occurs 0 or 1 times

[+] → Occurs at least 1 or more times

[*] → Occurs 0 or more times

[{n}] → Occurs atleast n or more times



$[] \{ \}$ → at least 1 times but less than 2 times.

problem:

You have to generate a mobile number, this will be started 8 or 9 and total digit will be 10.

Soln:

$[8\ 9] [0-9] \{9\}$

8 or 9 0 to 9 9 times repeat of

problem: You have to generate a gp number.

Soln:

$[0] [1] [2\ 3] [0-9] \{8\}$

0, 1 fixed

problem: + character, alphabet + digit + and special character (\$)

character (\$)

password - character

Soln

$[0-9] [0-9] [0-9] [0-9] [0-9]$

$[0-9] [0-9] \{7\} [0-9]$

[୨୨-୨]

[ମାଗେଣ୍ଡା ଲାଇଂ ଓ ଇମ୍ ୧୦୨]

୧ ଲାଇଂ ଓ ଇମ୍ ୧୦୨ ଯେ ୨ ମାଟ

ମାଗେଣ୍ଡା ଲାଇଂ ଓ ଇମ୍ ୧୦୨ ଯେ ୨ ମାଟ

୧୦୨ ଲାଇଂ ଓ ଇମ୍ ୧୦୨

ମାଗେଣ୍ଡା ଲାଇଂ ଓ ଇମ୍ ୧୦୨ ଯେ ୨ ମାଟ

[ମାଗେଣ୍ଡା ଲାଇଂ] [୧] [୨] [୩] [୪] [୫] [୬] [୭] [୮] ୬

୧୦୨ ଲାଇଂ ଓ ଇମ୍ ୧୦୨

[୧] [୨] [୩] [୪] [୫] [୬] [୭] [୮]

[ମାଗେଣ୍ଡା ଲାଇଂ] [୧] + [୨] [୩] [୪] [୫] [୬] [୭] [୮]

୧୦୨ ଲାଇଂ ଓ ଇମ୍ ୧୦୨

୧୦୨ ଲାଇଂ ଓ ଇମ୍ ୧୦୨

[ମାଗେଣ୍ଡା ଲାଇଂ] [୧] + [୨] [୩] [୪] [୫] [୬] [୭] [୮]

[ମାଗେଣ୍ଡା ଲାଇଂ] [୧] + [୨] [୩] [୪] [୫] [୬] [୭] [୮]

୧୦୨ ଲାଇଂ ଓ ଇମ୍ ୧୦୨

୧୦୨ ଲାଇଂ ଓ ଇମ୍ ୧୦୨

୧୦୨ ଲାଇଂ ଓ ଇମ୍ ୧୦୨

$(000)_2 (000)_2$ [one in one]

is the only way to get 000
 and if all inputs are 0, then output is 0

$(001)_2 (011)_2 (111)_2$ - these are the only cases where output is 1

[$(001)_2$ is 1, $(011)_2$ is 1, $(111)_2$ is 1]
 (00) - these are 0

(00) $(00)_2$ - these are 0
 (00) $(01)_2$ - these are 0

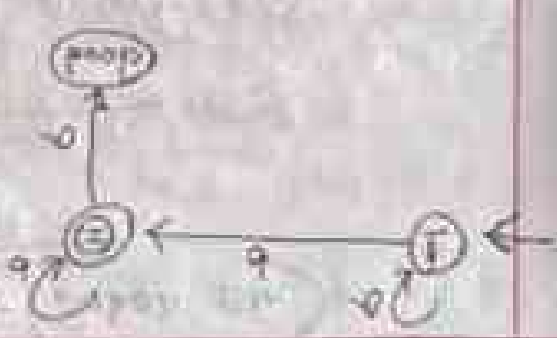
A circuit



if both are 0, then output is 0
 if both are 1, then output is 1

+ 9 * 0 for non-zero values

1. This is a circuit for a 2-bit adder.
 2. This is a circuit for a 2-bit adder.
 3. This is a circuit for a 2-bit adder.
 4. This is a circuit for a 2-bit adder.



15-07-2025 (Sabab Sir)

* Regular Expressions:

Regex:

* $\rightarrow$ zero or more

+ $\rightarrow$ one or more

? $\rightarrow$ zero or one

* examples

① $(aa)^*(bb)^*$ $\rightarrow$ even number of a & b
[But qth & 2nd amir slide 6]

$(aa)^*$ $\rightarrow$ even no. of a
 $a(aa)^*$ $\rightarrow$ odd no. of a

* $(aa)(a|b)^*(bb)$ $\rightarrow$ 2nd aa 4th (a|b) bb line
 $\rightarrow$ 2nd line

* Set of all strings having even number of
aa and even number of bb.
 $= (aaaa)^*(bbbb)^*$ [qth ans slide 6]

$$= P_2(0, (0, 1))$$

$$\leftarrow *((9|3)0)9 \square$$

1. ଶେଷର ୧ କିମ୍ବା ୩ ଅଙ୍କର ଗୁଡ଼ିକ

$$\leftarrow *0900900900 \square$$

1. ଶେଷର ୧ ଅଙ୍କର କେବଳ ୧ ଅଙ୍କର

କିମ୍ବା ୩ ଅଙ୍କର ଗୁଡ଼ିକ ଶେଷ ୧ ୦

$$\leftarrow *0*((900900)900 \square$$

1. ଶେଷର ୧ ଗୁଡ଼ିକ ଫଳାଫଳ

$$\leftarrow *((9|0)9 * ((9|0)9 * (9|0) \square$$

1. ଶେଷର ୩ ଅଙ୍କର ଫଳାଫଳ ଫଳାଫଳ ଫଳାଫଳ

$$\leftarrow ((9|0) ((9|0)0 * (9|0) \square$$

$\boxed{0}$ for strings having even number of 0's

$$*((00) =$$

Set of all strings having even number of 0's

00 for strings having odd number of 0's

$$\leftarrow *((00)00 \square$$

Set of all strings having odd number of 0's

* get-gmail address format

a.b@gmail.com

$(L^+D^+).(L^+D^+)^+ @ gmail.com$

Sabab 31-2ul 32-1 kean @ gmail.com

$L = a - z$

$D = 0 - 9$

$(L^+D^+)^+ (. (L^+D^+)^+)^? @ gmail.com$

Letter pangut maw

digit pangut maw

mas pangut maw

ut maw pangut maw

one atleast 2ul

mul pangut maw

ut maw pangut maw pangut maw pangut maw

ut maw pangut maw pangut maw pangut maw

* All strings of 'a' & 'b' that do not contain the substring 'abb'.

$= b^* (a (a|b))^*$

* All strings of 'a' & 'b' that contain at most two b's.

$= a^* b^? a^* b^? a^*$

* All strings of 'a' & 'b' that do not contain the sub sequence 'abb'.

$a \boxed{a} b \boxed{a} b \boxed{a} a \rightarrow$ $b^* a^* b^* a^*$

16-07-2025

Comments, consisting of a string surrounded by /* and */ without an intervening /*, unless it is inside double-quotes (").

Ans:

$L \rightarrow [a-zA-Z0-9]$

$C \rightarrow " \text{ } " /$

comment $\rightarrow /* (L^* C^*)^* */$

Numeric literals in Pascal

- Definition of the token unsigned-number.

digit $\rightarrow 0|1|2|3|4|5|6|7|8|9$

unsigned-integer = digit⁺

= digit digit digit

* example: All strings of 'a, b, c' that starts with atleast one 'a' or 'b' which is followed by two 'c'. the appearance of two 'c' and then the strings then. even number of 'b'.

Solⁿ

$$(a|b)^+ cc (bb)^* \text{ or } (a^+ | b^+) cc (bb)^*$$

Ans

(A24) ...

...

...

...

...

22-07-2024

* Lexical Analysis:

Lexical Analysis Vs Parsers.

* Buffer point: $\rightarrow$ $\rightarrow$ $\rightarrow$

v	a	n	d	=	m	+	e	o	f
---	---	---	---	---	---	---	---	---	---

$\uparrow$ $\uparrow \rightarrow$
 l_b f_w

End of file
 ref is not there
 kha hai nahi hai

Lexeme $\rightarrow$ $\rightarrow$

* AKA "FINITE Finite State Machines", "Finite Automata", "FA".

Two types: DFA, NFA

* Non Deterministic Finite Automata (NFA):

Mathematical model that consists

[formal definition]

$$S = \{0, 1, 2, 3\}$$

$$S_0 = \text{start}$$

$$F = \{3\}$$

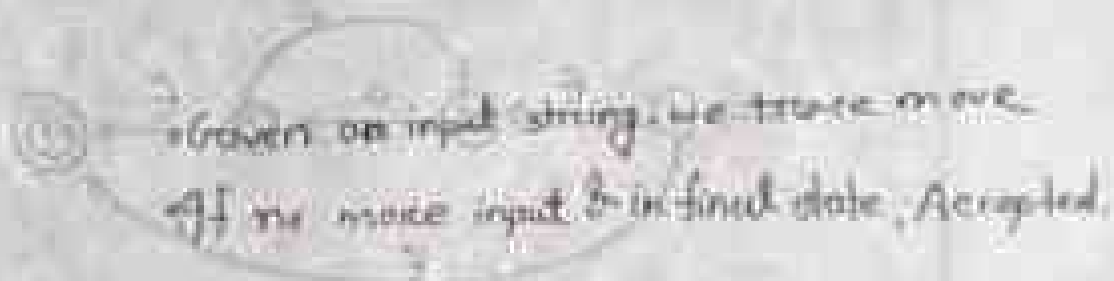


transition table:

	a	b
0	{0,1}	{0}
1	{2}	
2		{3}

Final state is 2

* How Does An NFA Work?



Given an input string, we trace move.
If no more input & in final state, Accepted.

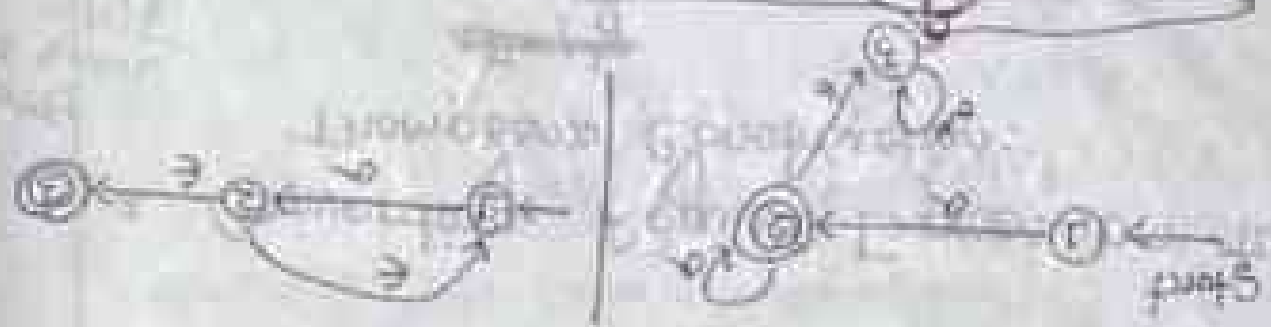
* Handling Undefined Transitions:

[death state]

* Converting Regular Expression to NFAs:

Thompson's Construction:





State 1 is the start state

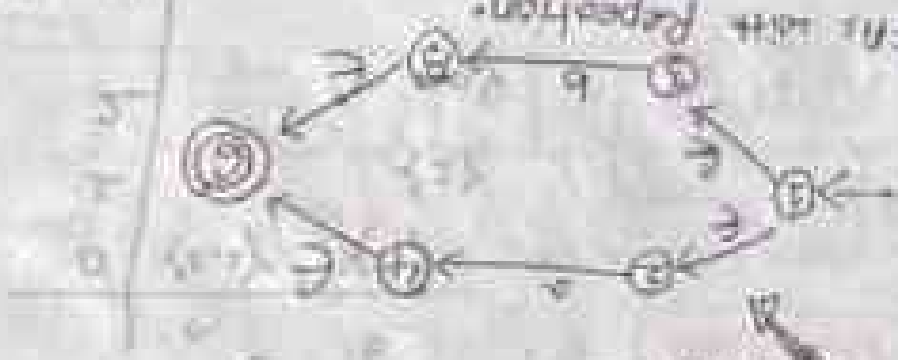
→ NFA draw for a^+b^+

2. DFA for a^+b^+
 1. 2. 3. 4. 5. 6. 7. 8. 9. 10. 11. 12. 13. 14. 15. 16. 17. 18. 19. 20. 21. 22. 23. 24. 25. 26. 27. 28. 29. 30. 31. 32. 33. 34. 35. 36. 37. 38. 39. 40. 41. 42. 43. 44. 45. 46. 47. 48. 49. 50. 51. 52. 53. 54. 55. 56. 57. 58. 59. 60. 61. 62. 63. 64. 65. 66. 67. 68. 69. 70. 71. 72. 73. 74. 75. 76. 77. 78. 79. 80. 81. 82. 83. 84. 85. 86. 87. 88. 89. 90. 91. 92. 93. 94. 95. 96. 97. 98. 99. 100.



State 1 is the start state

→ If A is a regular expression with NFA N_A



Thompson's rule (union)

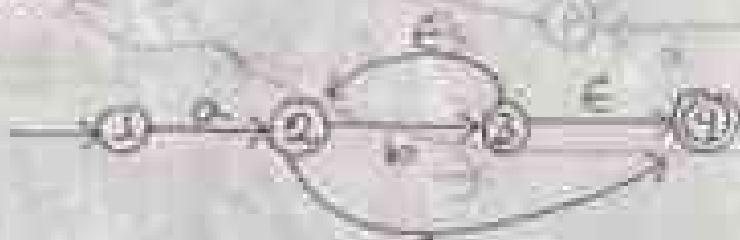


→ a/b

$\rightarrow a^3$



$\rightarrow a^+b^+$



$+ (ab^+ | a^+b)^+$

ab:



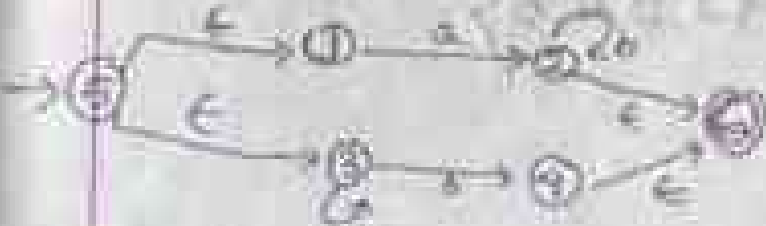
a^+b:



a^+b:

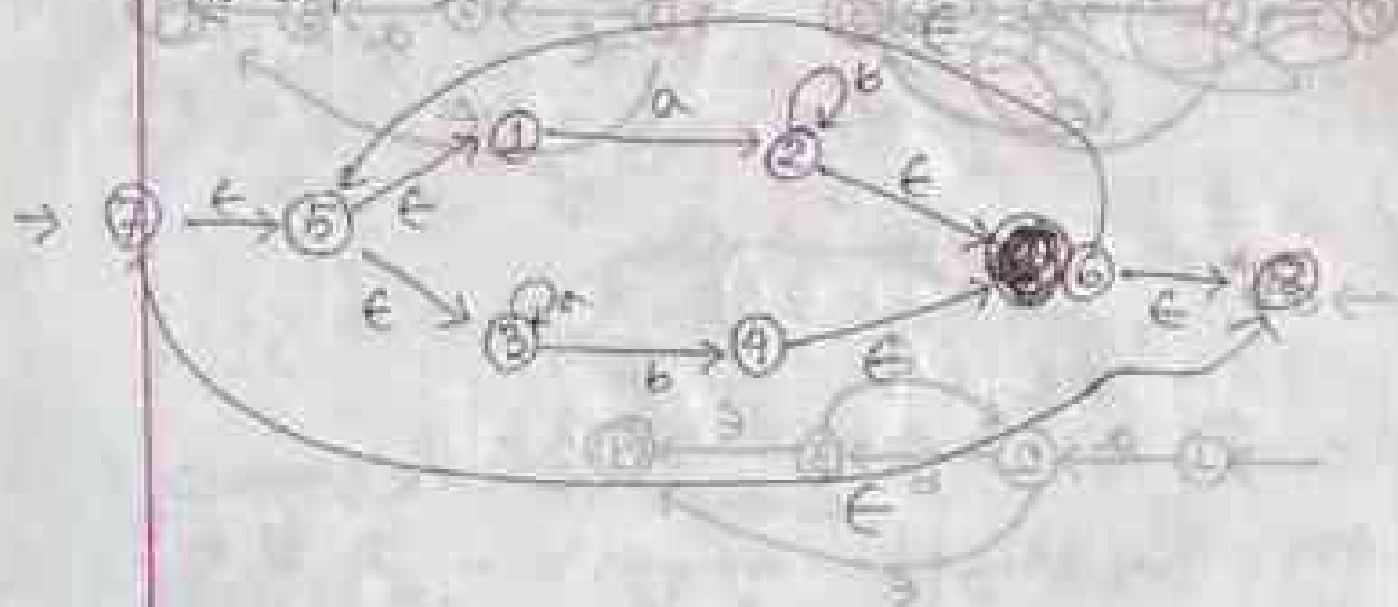


$ab^+ | a^+b$:



(a,b)

$(ab^* / a^*b)^*$:



23-07-2025

~~State~~

ϵ -closure:

ϵ -closure(T)

= T + All states

$(a^*b / a^*b)^*$

$$\epsilon\text{-closure}(\{1\}) = \{1\}$$

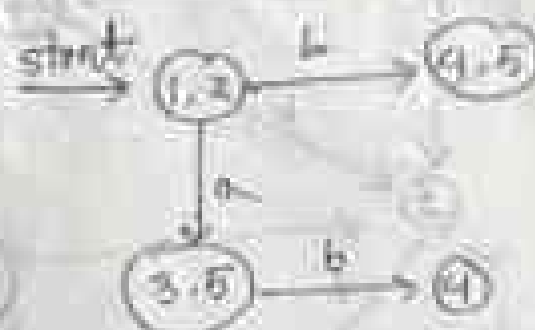
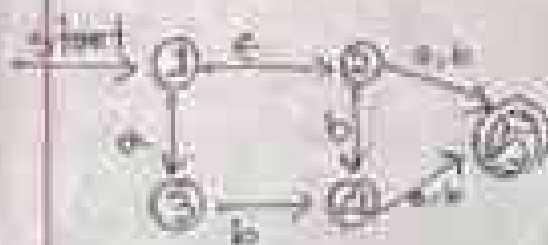
$$\epsilon\text{-closure}(\{2\}) = \{1, 3, 4\}$$

$$\epsilon\text{-closure}(\{4\}) = \{1, 4\}$$

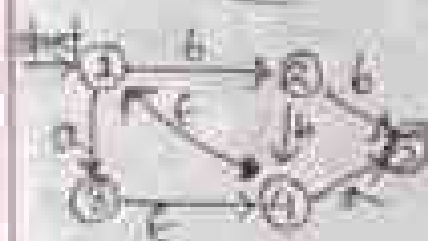
$$\epsilon\text{-closure}(\{1, 3, 5\}) = \{1, 3, 4, 5\}$$

Example 2: Subset Construction

NFA:



2. NFA: (example-3)



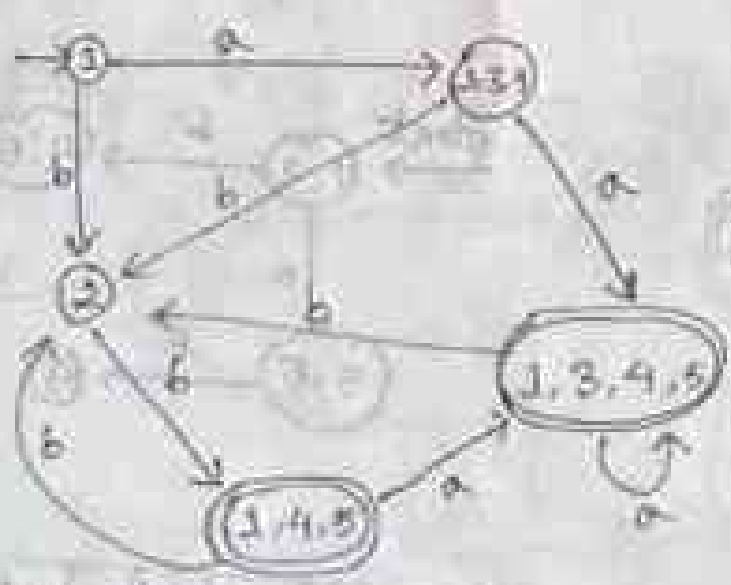
ϵ -closure($\{1\}$) = $\{1, 2, 4\}$
 ϵ -closure($\{2\}$) = $\{1, 3, 4\}$
 ϵ -closure($\{3\}$) = $\{3, 4\}$
 ϵ -closure($\{4\}$) = $\{4\}$
 ϵ -closure($\{5\}$) = $\{5\}$



ϵ -closure($\{1\}$) = $\{1, 2, 4\}$
 ϵ -closure($\{2\}$) = $\{1, 3, 4\}$
 ϵ -closure($\{3\}$) = $\{3, 4\}$
 ϵ -closure($\{4\}$) = $\{4\}$
 ϵ -closure($\{5\}$) = $\{5\}$

| State | a | b |
|---------------|---------------------|---------------|
| $\{1\}$ | $\{1, 2, 4\}$ | $\{2\}$ |
| $\{2\}$ | $\{1, 3, 4\}$ | $\{4, 5\}$ |
| $\{3\}$ | $\{3, 4\}$ | $\{4\}$ |
| $\{4\}$ | $\{4\}$ | $\{4\}$ |
| $\{5\}$ | $\{5\}$ | $\{5\}$ |
| $\{1, 2, 4\}$ | $\{1, 2, 3, 4, 5\}$ | $\{2, 4, 5\}$ |
| $\{1, 3, 4\}$ | $\{1, 2, 3, 4, 5\}$ | $\{2, 4, 5\}$ |
| $\{3, 4\}$ | $\{3, 4\}$ | $\{4\}$ |
| $\{4\}$ | $\{4\}$ | $\{4\}$ |
| $\{5\}$ | $\{5\}$ | $\{5\}$ |

Equivalent DFA:



Example 4:

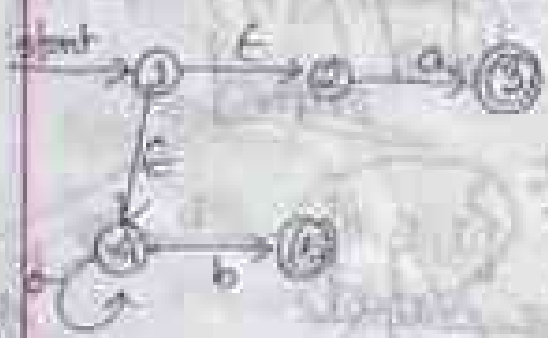


Table:

| | a | b |
|-----------|-----------|--------|
| {1} | {2, 3, 4} | {3, 5} |
| {2, 3, 4} | {2, 3, 4} | {5} |
| {3, 5} | | |
| {4} | {4} | {5} |
| {5} | {5} | {5} |

4. Example: DFA to REs

CLASSMATE

Course Code: CSE3105
Compiler Design

Unit-2



Bashundhara
Exercise Book
Write Your Future

Ehsan Mahmud

Compiler Design

$$x. P \rightarrow \pi P P s s P \mid \overbrace{\pi P P s P \pi}^{\text{Mid}} \mid \underline{s P s} \mid \pi \mid \pi X$$

Solⁿ:

$$\boxed{P \rightarrow \pi P' \mid s P s}$$

$$P' \rightarrow P P s s P \mid P P s P \pi \mid \epsilon \mid X$$

$$\hookrightarrow \boxed{P' \rightarrow P P s P' \mid \epsilon \mid X}$$

$$\boxed{P'' \rightarrow s P \mid P \pi}$$

$$x. X \rightarrow p q X \mid p X q \mid p q \pi X \mid p$$

Solⁿ:

$$\boxed{X \rightarrow p X'}$$

$$X' \rightarrow q X \mid X q \mid q \pi X \mid \epsilon$$

$$\hookrightarrow \boxed{X' \rightarrow q X' \mid X q \mid \epsilon}$$

$$\boxed{X'' \rightarrow X \mid \pi X}$$

mid-2

LL(a) CS 2.10

13-08-25 (W)

examined at Ambiguity in CFGs - oneso academy

0/2/2 ← [previous note covered]

$$j \rightarrow 194 \leftarrow 9 \quad \text{plan} \leftarrow A \quad \text{do } [id + id + id] \rightarrow$$

Ambiguity in CFGs - solve Problems :

Q1: Determine whether the following grammar is suitable for predictive parsing.

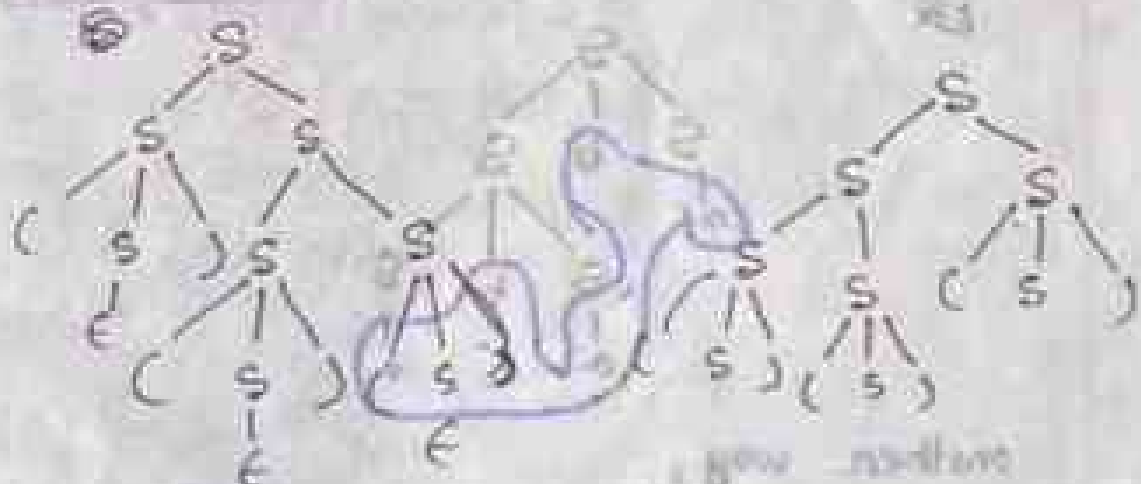
(a) $\mathcal{E}(ss) / (s) \rightarrow s$

 $\mathbb{S}_0/\mathbb{N}_0$

ambiguous and suitable all

Portia Tree

Derivation: $(\cup) (\cap) (\cap)$



We give 4 receive 4 different parse tree from the derivation } So it is ambiguous.

Q2: Consider the following two Grammars,

$G_1: S \rightarrow SbS/a$

$G_2: S \rightarrow aB/ab, A \rightarrow AB/a, B \rightarrow ABb/b$

Which of the following option is correct?

(A) Only G_1 is ambiguous

(B) Only G_2 is ambiguous

(C) Both G_1 & G_2 are ambiguous

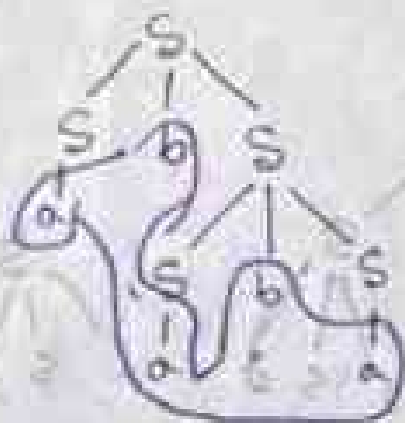
(D) Both G_1 & G_2 are unambiguous

Soln:

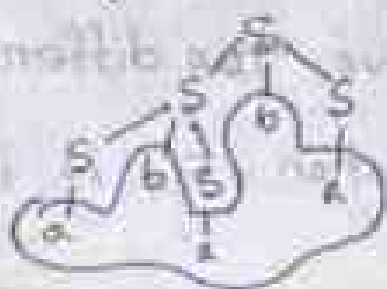
$G_1: S \rightarrow SbS/a$

Let's derive:
ababa

Ex



another way,



So, G_1 is
ambiguous
because more than
one parse tree

Gr2: $S \rightarrow aB|ab, A \rightarrow AB|a, B \rightarrow ABb|b$ (A)



Let's try to derive string: ab

another way,



So, Gr2 is also

ambiguous.

Q3: Which of the following grammars are ambiguous?

(A) $S \rightarrow SS|aSb|bSa|\lambda$

(B) $S \rightarrow aSbS|bSaS|\lambda$

(C) $S \rightarrow aAB, A \rightarrow bBb, B \rightarrow A|\lambda$

where λ denotes empty string.

1. (A) & (B) only

2. (B) only

3. (B) & (C) only

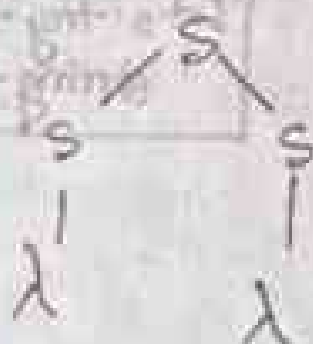
4. (A) & (B) only

Solⁿ:



(A) $S \rightarrow \epsilon \mid aSb \mid bSa \mid \lambda$

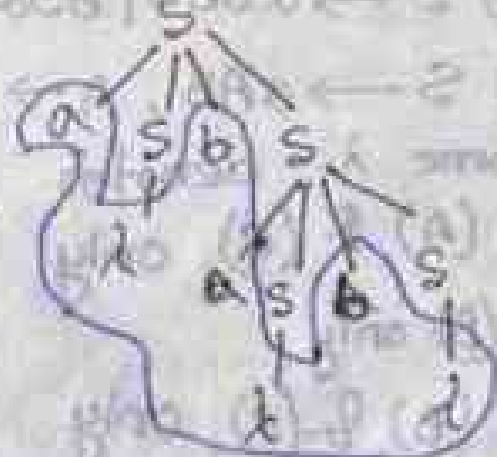
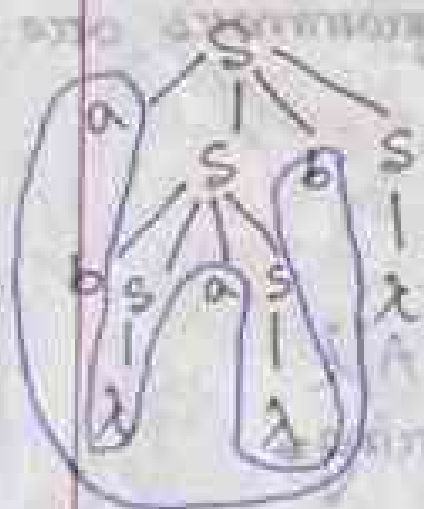
derive,
 λ



So, ambiguous

(B) $S \rightarrow aSbS \mid bSaS \mid \lambda$

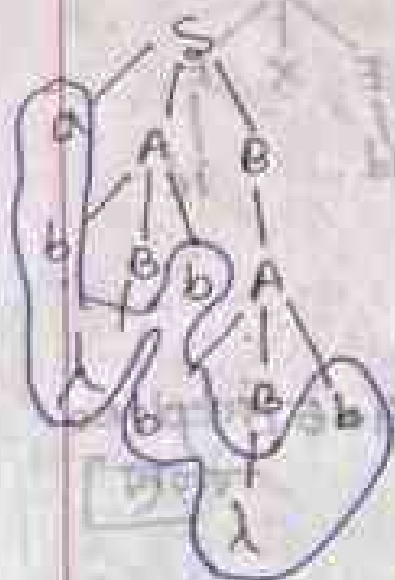
derive
abab



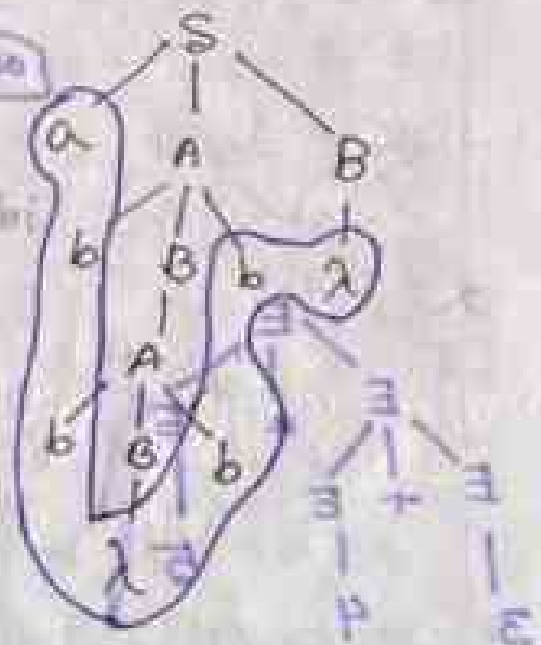
So, ambiguous.

(c) $S \rightarrow aAB, A \rightarrow bBb, B \rightarrow A|\lambda$

Let's derive:
abbbb



[Example]



So, ambiguous.

Lec-17:

Problems of Ambiguity in CFGs

problem:

1. Associativity Property Violation.

2. Precedence Property Violation.

Context Free Grammar:

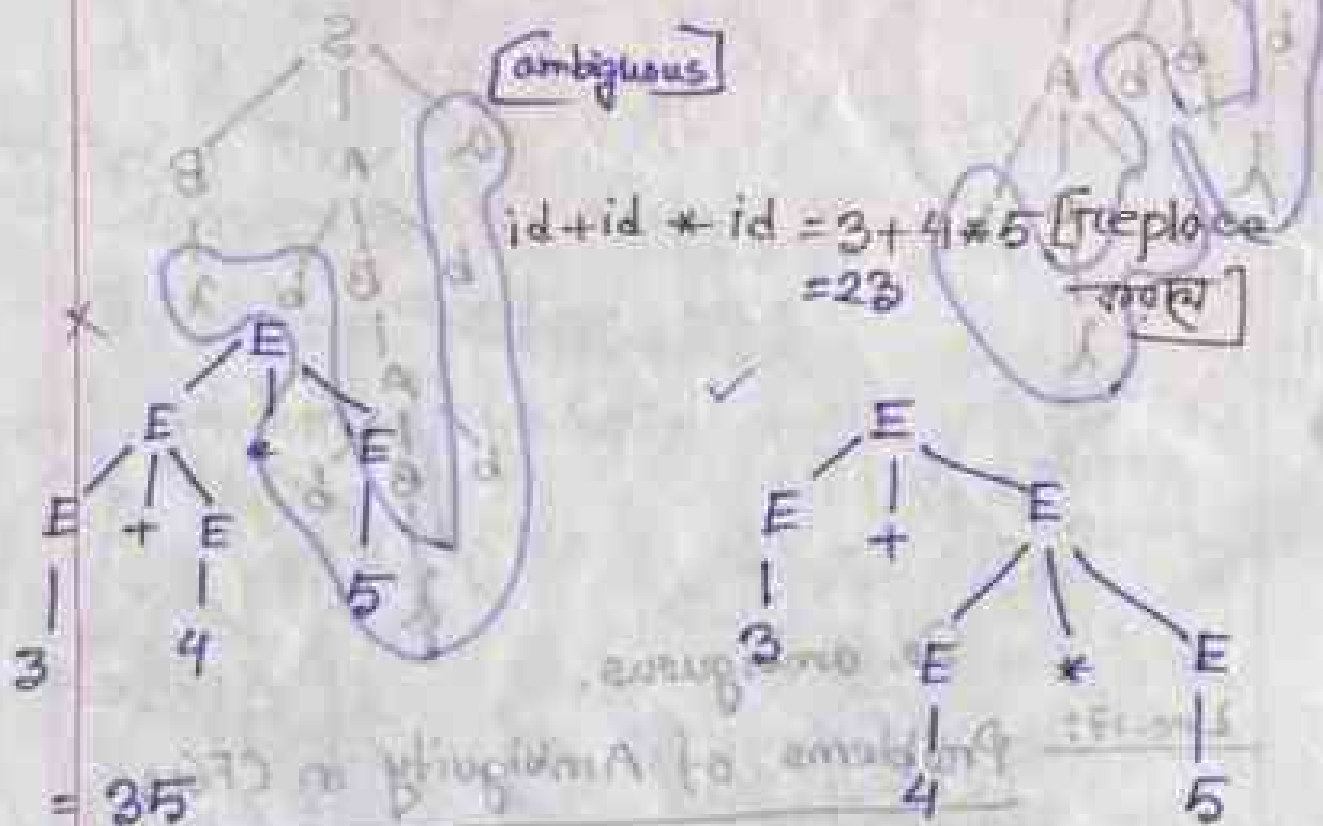
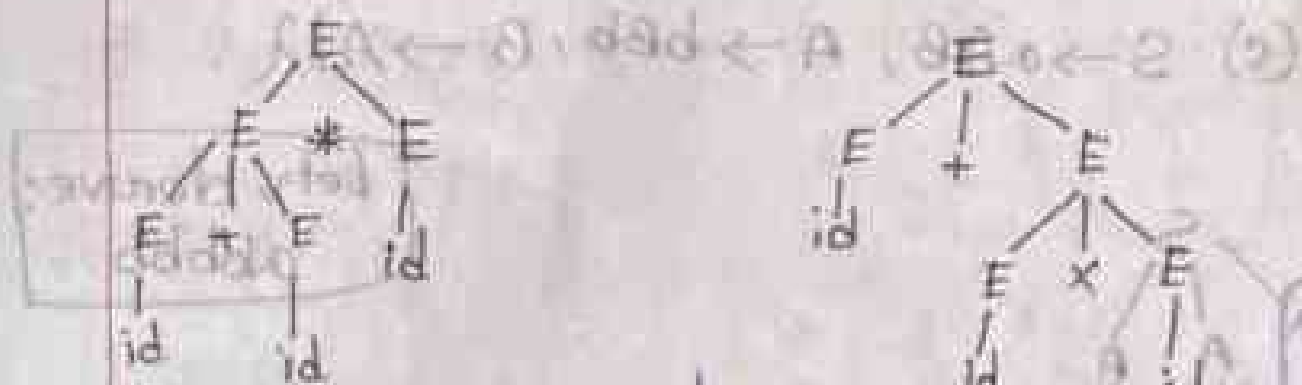
1. $E \rightarrow E + E$

2. $E \rightarrow E * E$

3. $E \rightarrow id$

id + id * id

Let's derive



For the same expression we find two different values. This is the problem.

Property violation.

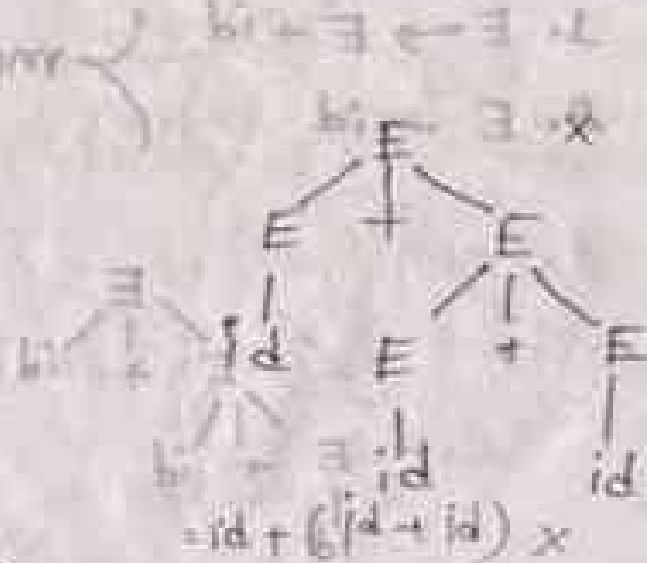
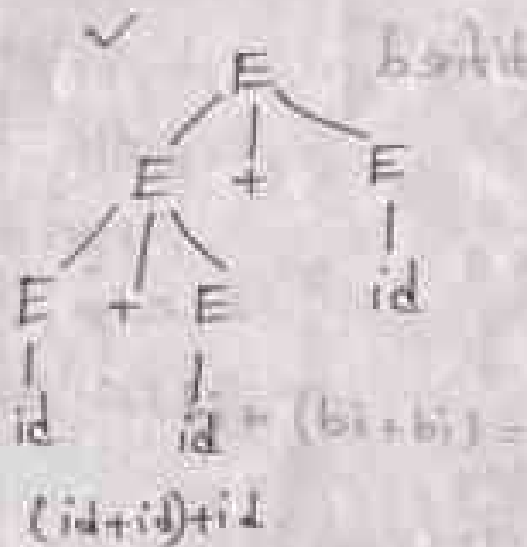
Again,

1. $E \rightarrow E + E$
2. $E \rightarrow E * E$
3. $E \rightarrow id$

Let's derive,

$id + id + id$

it's actually mean, $(id + id) + id$



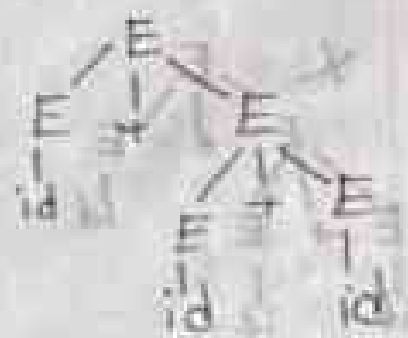
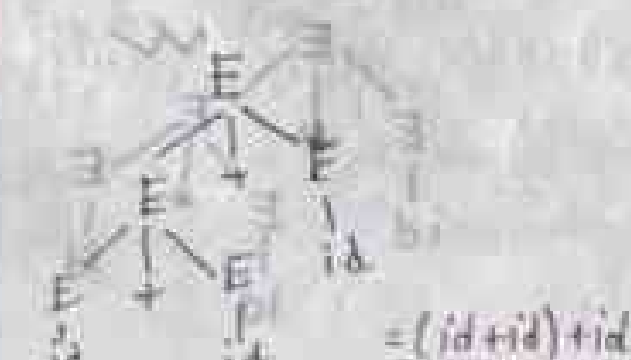
[Associativity Property Violation]

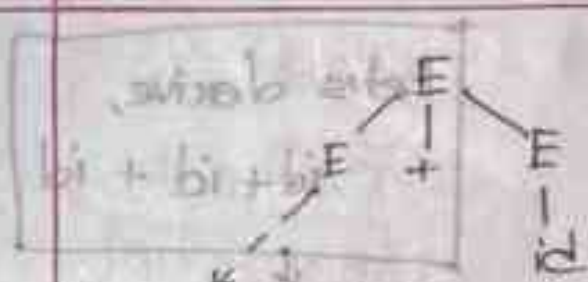
* Associativity Violation and Solution in CFGs:

1. $E \rightarrow E + E$

2. $E \rightarrow id$

$id + id + id$

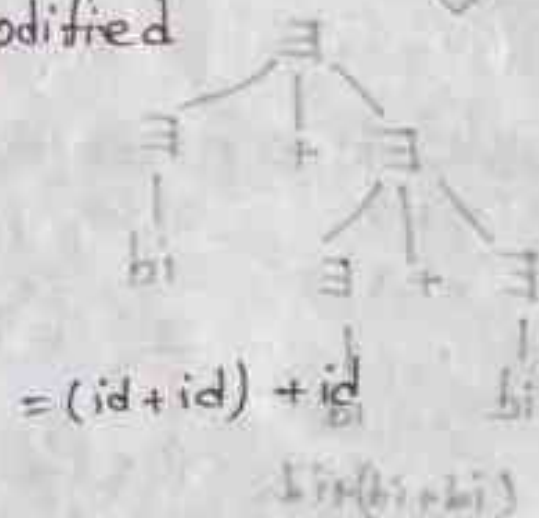
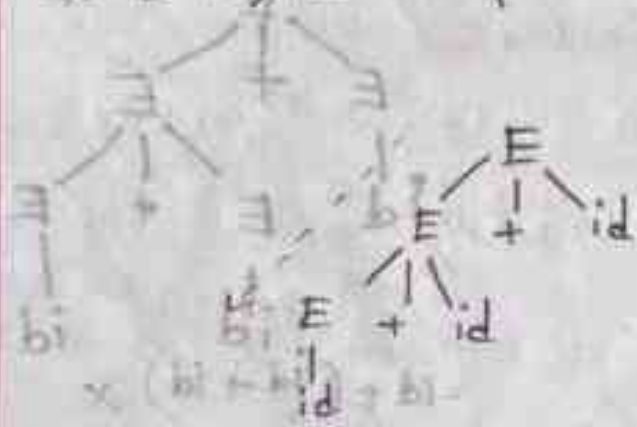




3. $E \rightarrow id$
 2. $E \rightarrow E * E$
 1. $E \rightarrow E + E$

Now, (mean) $id + id$

1. $E \rightarrow E + id$
 2. $E \rightarrow id$
- } modified



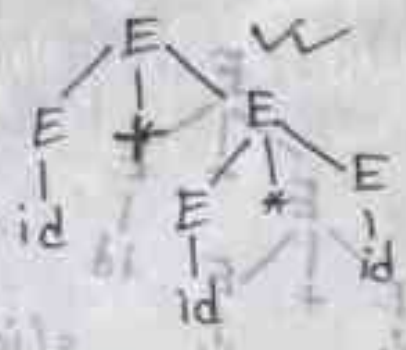
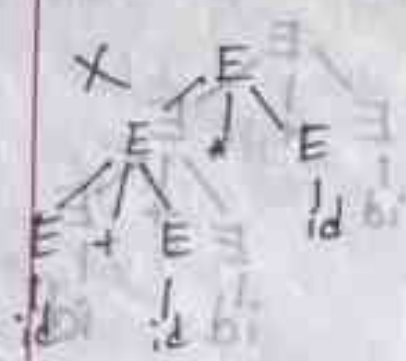
[Violation of associativity]

[Solved the associativity problem]

Lec-19: Precedence Violation and Solution in CFG

$E \rightarrow E + E \mid E * E \mid id$

$id + id * id$



[+ & * both * have higher precedence. ~~and + has~~
~~lowest precedence~~]

$$1. E \rightarrow E + E \Rightarrow E \rightarrow E + T \mid F$$

$$2. T \rightarrow T * T \Rightarrow T \rightarrow T * F \mid F$$

$$3. F \rightarrow id$$

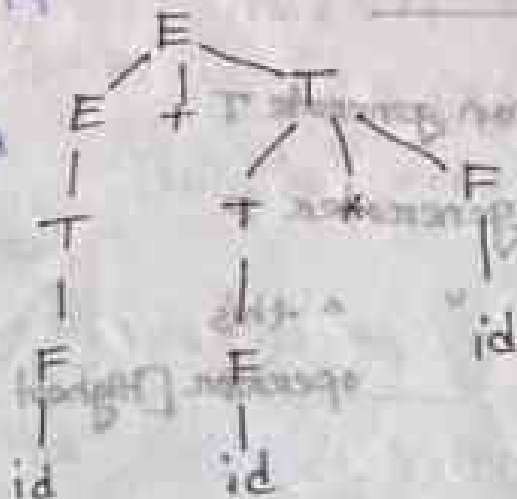
id + id * id

$$A/A = A$$

non-terminal =

$$A \rightarrow A$$

terminal =



Lec-20: Associativity & Precedence in CFGs:

$$1. E \rightarrow E + T \mid T$$

$$2. T \rightarrow T * F \mid F$$

$$3. F \rightarrow G \wedge F \mid G$$

$$4. G \rightarrow id$$

Determine Associativity and Precedence of
 these the operators:

Soln:

→ Associativity:

$$E \rightarrow E + T \mid T$$
$$T \rightarrow T \mid F \mid E$$
$$F \rightarrow G \wedge \bigvee_i F_i \mid G$$
$$b_i = b_i^* + b_i^{\dagger}$$

- Left Associative:

Right Associative:

$$b_i \in \mathbb{R}^n$$

Precedence order:

$$A = \alpha A|B$$

= Right recursive

$$A = A_0 e^{-\gamma x}$$

= Left Recursion

1. From E , we can generate T

2. From T, we can generate $\mathcal{F}$

3. From F, $\dots$ this

operator [highest priority]

orden:

High

Q1: Consider the grammar defined by the following production rules, (with two operators $*$ and $+$)



$$S \rightarrow T * P$$

$$T \rightarrow U \mid T * U$$

$$P \rightarrow Q + P \mid Q$$

$$Q \rightarrow id$$

$$U \rightarrow id$$

$$d + d \quad \text{Sol}^n$$

$$d - d$$

$$(d + d) * (d - d)$$

$$((d + d) * (d - d)) + d =$$

Solⁿ:

Associativity:

$$T \rightarrow T * P$$

Left recursive

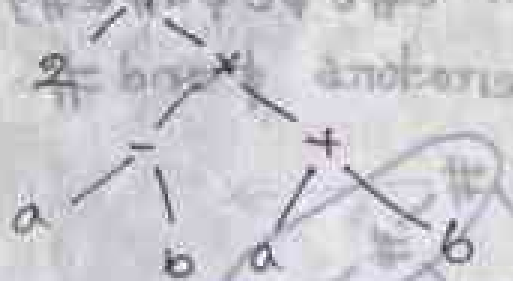
$$P \rightarrow Q + P$$

Right recursive

Precedence:



Q2: Consider the parse tree-



Assume that '+' has higher precedence than '+', and operators associate right to left (i.e. $a + b + c = (a + (b + c))$)

What is the correct expression?

Solⁿ:

Solⁿ: $a+b$

$a-b$

$(a-b) * (a+b)$

$$= 2 + ((a-b) * (a+b))$$

$q * T \leftarrow 2$

$U * T | U \leftarrow T$

$2 | 9 + 0 \leftarrow 9$

$b_i \leftarrow 0$

$b_i \leftarrow U$

Q3: Given the following expression grammar:

$E \rightarrow E * F / F + E / F$

$F \rightarrow F \circ F / id$

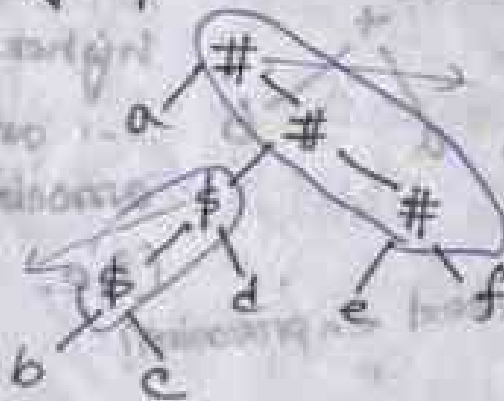
which of the following is true?

Solⁿ:

$\circ$ has higher precedence than $*$.

Lec-22:

Q1: Consider the following parse tree for the expression $a \# b \$ c \$ d \# e \# f$, involving two binary operators $\$$ and $\#$.



Soln.

\$ operators are derived in the low levels than

operators.

→ precedence:

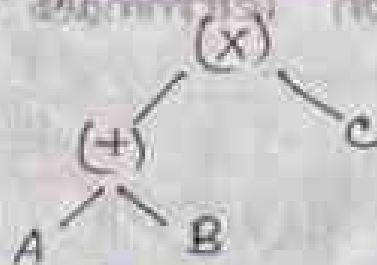


→ Associativity:

\$ - Left Associative

- Right Associative

Q2: Which of the following expression is represented by the parse tree?



Soln.

$(A + B)XC$



Q3: Given the grammar:

$$S \rightarrow T \times S / T$$

$$T \rightarrow U + T / U$$

$$U \rightarrow id$$

→ is ambiguous? **No**

→ Priority of '+' over 'x' is ensured? **Yes**

→ Right to Left evaluation of 'x' and '+' happens? (Yes) - right recursive

Soln:

$$S \rightarrow T \times S / T$$

$$T \rightarrow U + T / U$$

$$U \rightarrow id$$

non Terminals: S, T, U

Now,

$$S \rightarrow T \times S$$

$$T \rightarrow U + T$$

both are right recursive, so this is not ambiguous.

precedence:

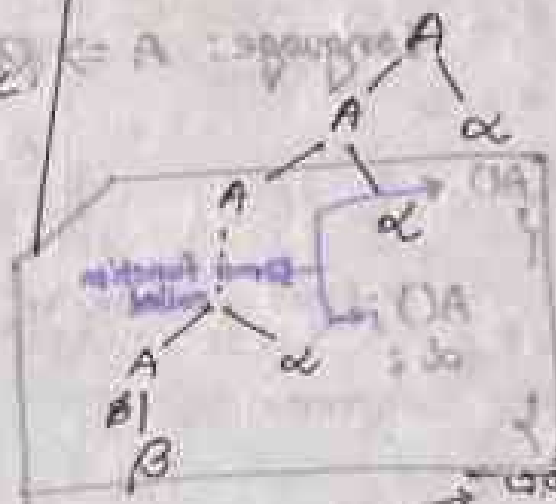
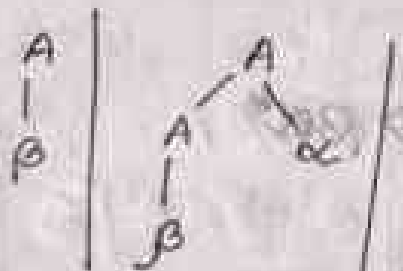


Lec-23: Recursion

Left recursion:

$$A \rightarrow A\alpha \mid \beta$$

left side α



(A)

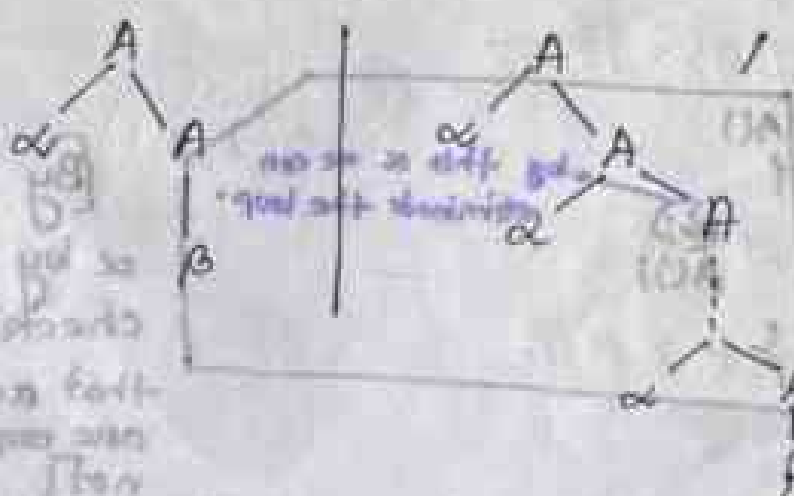
$$A = \beta\alpha^*$$

Left recursive grammar

Right Recursion:

$$A \rightarrow \alpha A \mid \beta$$

right side α



$$A = \alpha^*\beta$$

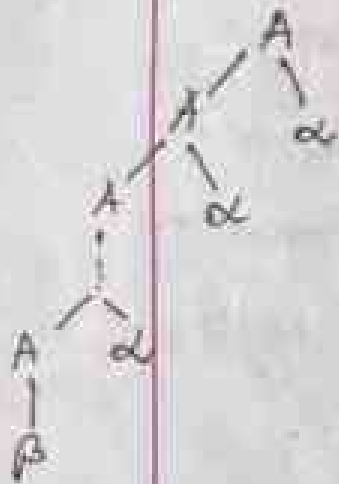
Right recursive grammar

Lec-24: Problem of Left Recursion & Solⁿ

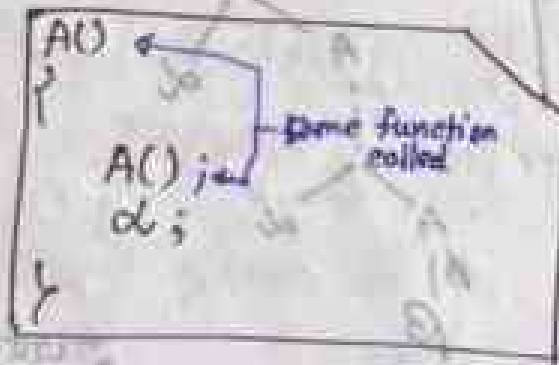
Left Recursion:

$$A \rightarrow A\alpha / \beta$$

$$A \Rightarrow \beta\alpha^*$$



Language: $A \Rightarrow \beta\alpha^*$

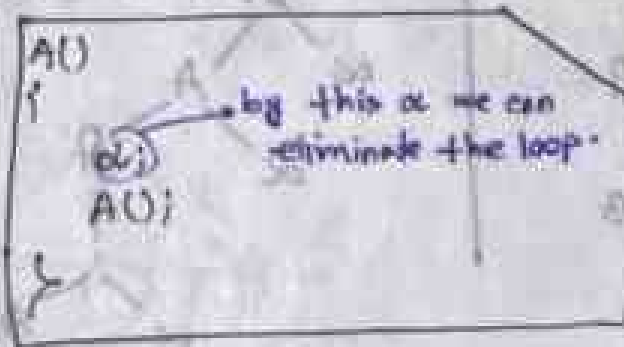


Creates infinite loop (if not handled properly)

Right Recursion:

$$A \rightarrow \alpha A / \beta$$

$$A \Rightarrow \alpha^* \beta$$

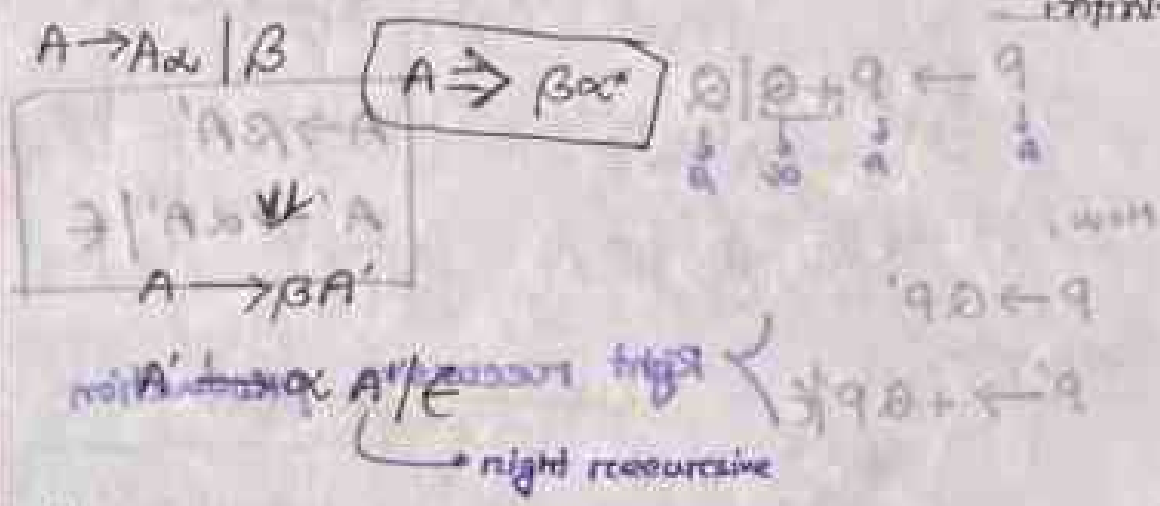


[By modify α by means of checking for that recursive call are required or not]

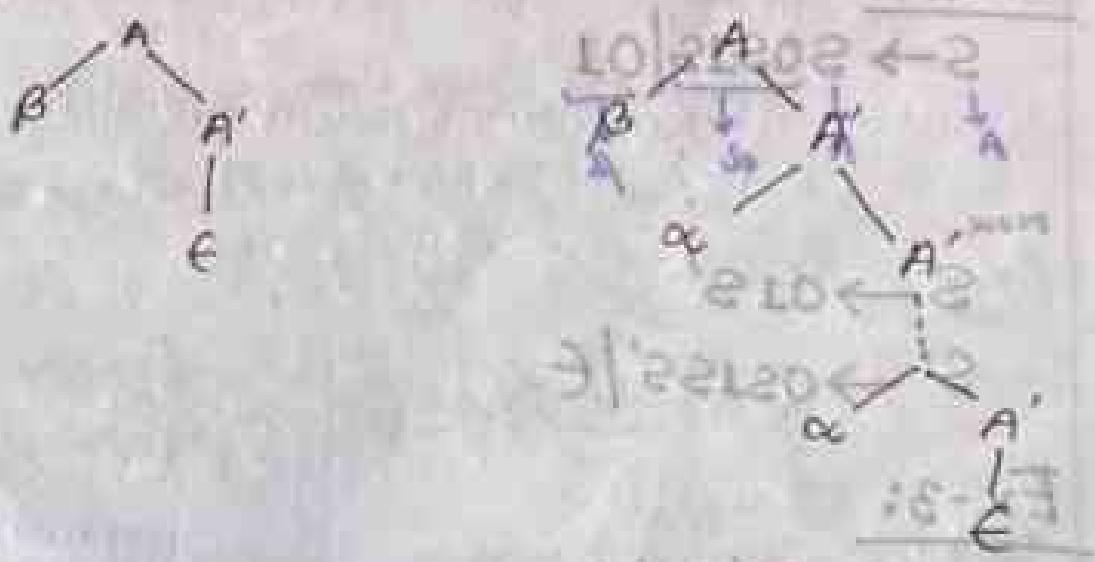
So, it is not possible to go infinite loop.

Elimination of left recursion Conversion:

Left recursive to Right recursive (to avoid falling in infinite loop)



Now,



$A \Rightarrow \beta\alpha^*$ is right re.
A is converted into same output string

$d(\alpha) \leftarrow A, \exists \beta\alpha^* \leftarrow \beta, \beta\alpha \leftarrow \beta$

Lec-25: Elimination of Left Recursion

→ Four solved examples on eliminating left recursion.

Ex-1:

$$P \rightarrow P + Q \mid Q$$

$\downarrow$ $\downarrow$ $\downarrow$ $\downarrow$
 A A α β

Now,

$$P \rightarrow Q P'$$

$$P' \rightarrow + Q P' \mid \epsilon$$

Right recursive production

$$A \rightarrow \beta A'$$

$$A' \rightarrow \alpha A' \mid \epsilon$$

Ex-2:

$$S \rightarrow S O S \mid S \mid 0 \mid 1$$

$\downarrow$ $\downarrow$ $\downarrow$ $\downarrow$
 A A α β

Now,

$$S \rightarrow 0 S'$$

$$S' \rightarrow O S \mid S S' \mid \epsilon$$

Ex-3:

$$A \rightarrow (B) \mid b$$

$$B \rightarrow B x A \mid A$$

$\downarrow$ $\downarrow$ $\downarrow$ $\downarrow$
 A A α β

Now,

$$B \rightarrow A B', \quad B' \rightarrow x A B' \mid \epsilon, \quad A \rightarrow (B) \mid b$$

Ex-4:

$$A \rightarrow A\alpha_1 | A\alpha_2 | A\alpha_3 | \dots | B_1 | B_2 | B_3 | \dots$$

Soln.

$$A \rightarrow B_1 A' | B_2 A' | B_3 A' | \dots$$

$$A' \rightarrow \alpha_1 A' | \alpha_2 A' | \alpha_3 A' | \dots | \epsilon$$

* Left Factoring : $\rightarrow$ Anisul

$$A \rightarrow \alpha B_1 | \alpha B_2 | \alpha B_3 | \dots | \alpha B_n$$

αB_2 [generate karate pura rakha]



αB_3 karate rakha

back track

In normal common prefix problem ko solve karke left factoring diya.

Kisi koi 20 karate rakha.

2003

$$A \rightarrow \alpha \beta_1 / \alpha \beta_2 / \alpha \beta_3 \dots / \alpha \beta_n / \alpha \beta_{n+1} \dots$$

left factoring

$\rightarrow \dots | A_3 a | A_2 a | A_1 a \leftarrow A$
 $A \rightarrow \alpha A'$
 $\rightarrow \dots | A_3 b | A_2 b | A_1 b \leftarrow A$
 (common)

$$A' \rightarrow \beta_1 | \beta_2 | \beta_3 | \dots | \beta_n$$

Answer: principle of Action



β_2 [common prefix तारे तारे एकत्रित मात्र
द्वारा direct प्रदान करें]

Ever married

| Age Group | Percentage |
|-----------|------------|
| 15-19 | 10% |
| 20-24 | 20% |
| 25-29 | 40% |
| 30-34 | 60% |
| 35-39 | 75% |
| 40-44 | 85% |
| 45-49 | 90% |
| 50-54 | 95% |
| 55-59 | 98% |

$$S \rightarrow \underline{if + s} \mid \underline{if + s \text{ else } s} \mid a$$

$$E \rightarrow b$$

Now

$$S \rightarrow iE | SS' | a$$

$$S \rightarrow E | eS$$

$$f \rightarrow b$$

Ex-2:

$$A \rightarrow aAB \mid aA$$

$$B \rightarrow bB \mid b$$

Left factoring:

$$A \rightarrow aA'$$

$$A' \rightarrow AB \mid A$$

$$B \rightarrow bB \mid b$$

Ex-4:

$$E \rightarrow T + E \mid T$$

↓

$$E \rightarrow T E'$$

$$E' \rightarrow + E \mid \epsilon$$

Ex-3:

$$A \rightarrow aAB \mid aA \mid a$$

↓

$$A \rightarrow aA'$$

$$A' \rightarrow AB \mid A \mid \epsilon$$

[Common prefix removed]

Ex-5:

$$S \rightarrow bSSa \mid bSSb \mid bSb \mid a$$

↓

$$S \rightarrow bSS' \mid a$$

$$S' \rightarrow SaaS' \mid SaaS'b \mid b$$

[Common prefix problem fix]

$$S \rightarrow bSS' \mid a$$

$$S' \rightarrow SaaS' \mid b$$

$$S' \rightarrow aS \mid Sb$$

12-08-2025 (Lab)

implement: $(a/b)^* b(a/b)^*$

abab

b

bbb

a b input string
 yes string accepted.
 a b input string accepted.
 no string

• Language processing

• Eliminate Left Recursion:

$$① E \rightarrow E + T \mid T$$

$$T \rightarrow T * F \mid F$$

$$F \rightarrow (E) \mid id$$

$$E \rightarrow E + T \mid T$$

$$E \rightarrow TE'$$

$$E' \rightarrow +TE' \mid \epsilon$$

$$T \rightarrow T * F \mid F$$

$$T \rightarrow FT'$$

$$T \rightarrow *FT' \mid \epsilon$$

code: implement (eliminate left recursion)

code: implement (eliminate left recursion)



$$(i) A \rightarrow A a_1 | A a_2 | A a_3 | \dots | b_1 | b_2 | b_3 | \dots$$

$$\downarrow$$

$$A \rightarrow b_1 A' | b_2 A' | b_3 A'$$

$$A' \rightarrow a_1 A' | a_2 A' | a_3 A' | \dots | \epsilon$$

code: implement (eliminate left recursion)

codeshare.io/6bn2A5

LR(0) Item Sets

| LR(0) Item Set | LR(0) Item Set | LR(0) Item Set |
|--|--|--|
| $\{A, a_1 A', a_2 A', a_3 A', \dots, \epsilon\}$ | $\{A, a_1 A', a_2 A', a_3 A', \dots, \epsilon\}$ | $\{A, a_1 A', a_2 A', a_3 A', \dots, \epsilon\}$ |
| $\{A, a_1 A', a_2 A', a_3 A', \dots, \epsilon\}$ | $\{A, a_1 A', a_2 A', a_3 A', \dots, \epsilon\}$ | $\{A, a_1 A', a_2 A', a_3 A', \dots, \epsilon\}$ |
| $\{A, a_1 A', a_2 A', a_3 A', \dots, \epsilon\}$ | $\{A, a_1 A', a_2 A', a_3 A', \dots, \epsilon\}$ | $\{A, a_1 A', a_2 A', a_3 A', \dots, \epsilon\}$ |

Symbol analysis (4)

lexical analysis
प्रारंभिक विश्लेषण

12-08-2025

* parsing :

LL1 parsing

→ Top Down parsing :

First Function :

Identify the First(), Follow(), LL1 Parse Table

$S \rightarrow aS \mid B$
 $B \rightarrow bBc \mid c$
 $c \rightarrow cc \mid d \mid \epsilon$

Soln.

| Non-Terminals | FIRST() | FOLLOW() |
|---------------|---------------------------|-------------------------------|
| S | {a, b, c, d, ϵ } | {\$, ϵ } |
| B | {b, c, d, ϵ } | {\$, ϵ , c, d, f} |
| c | {a, d, ϵ } | {f, \$, ϵ , c, d, a} |

Follow (प्रारंभिक):

Start = S $\rightarrow$ aS $\mid$ B
 $S \rightarrow aS \mid B$

B follows S
 S → a S c | B
 B → b B c f | c

Note: Capital letter are non-terminals
 c is a terminal symbol (non-terminal) & c, d, e are terminals
 Example: (A) → ...

Example:
 B → b B c f | c
 c → c C g | d | f

Example: Identify the First(), Follow(), LL1 parse table.

Example:
 $S \rightarrow (A) | \epsilon$
 $A \rightarrow T E$
 $E \rightarrow \epsilon | T E$
 $T \rightarrow (A) | a | b | c | \epsilon$

Note:
 The string ϵ is a terminal symbol
 E is a non-terminal symbol
 T is a terminal symbol
 E is a non-terminal symbol

Soln:

| Non-terminals | First() | Follow() |
|---------------|--------------------------------------|-------------------|
| S | {(, ϵ } | { ϵ ,)} |
| A | {(, a, b, c, ϵ ,)} | { ϵ ,)} |
| E | { ϵ , a, b, c, ϵ } | { ϵ ,)} |
| T | {(, a, b, c, ϵ } | { ϵ ,)} |

Ans: (b) (c) (d) (e)

$A \rightarrow T E$
 $T \rightarrow (A) | a | b | c | \epsilon$

S ko Follow: (Kadabar) karu \$

A ko Follow:

$$S \rightarrow (A) | \epsilon$$

$$T \rightarrow (A) | a | b | c | \epsilon$$

E ko Follow:

$A \rightarrow T(E) \Rightarrow E$ (Kadabar) karu A ko follow karu
 (Kadabar) karu E ko follow =)

$$E \rightarrow \epsilon | T(E)$$

$E \rightarrow \epsilon$ (Kadabar) karu

add follow

Uthas E ko follow karu karu =) karu
 (Kadabar)

T ko Follow:

$$A \rightarrow T(E)$$

E non-Terminals karu E ko
 first (Kadabar) karu follow (Kadabar)

E ko first: ϵ, ϵ

if, $E = \epsilon$

$$A \rightarrow T\epsilon = \epsilon$$

if $E = \epsilon$

$$A \rightarrow T(\epsilon)$$

add follow
 ϵ

A ko follow karu
 (Kadabar)

Again:

$$E \rightarrow \epsilon \mid TE$$

$$\text{if } E = b \Rightarrow E \rightarrow \epsilon \mid Tb = b$$

$$\text{if } E = \epsilon \Rightarrow E \rightarrow \epsilon \mid T = \epsilon \mid \text{follow}$$

$$\text{Thus, } E \text{ or follow} = \{b, \epsilon\}$$

$$\text{New, } T \text{ or follow} = \{b, \epsilon\}$$

example: Identify the FIRST(), FOLLOW()

LL1 Parse Table

$$S \rightarrow aAbc \mid Bcg$$

$$A \rightarrow Cabab \mid aBg$$

$$B \rightarrow ad \mid ef \mid \epsilon$$

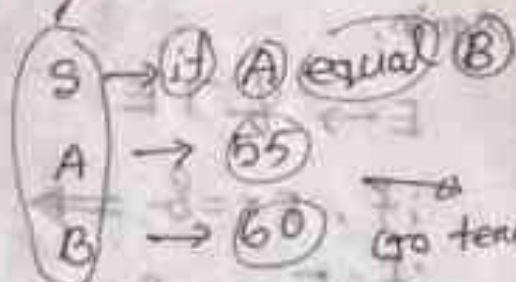
$$C \rightarrow k \mid \epsilon$$

Soln:

| Non terminals | FIRST() | FOLLOW() |
|---------------|---------------|-----------|
| S | {a, e} | {g, \$} |
| A | {k, a} | {b, \$} |
| B | {a, e, f} | {a, g, b} |
| C | {k, \epsilon} | {a} |

A-calling
is not the
call as with
@ b k a a a
FIRST

non-terminals



A's first:

if c first consider

$A \rightarrow @abB$

if, $c = k$ then $\epsilon = k$

if $c = \epsilon$ then $\epsilon = a$

Again,

$A \rightarrow aBc$

$\{k, a\}$

S's first:

$S \rightarrow aAbc / Bc$

$S \rightarrow @Abc$

Again,

$S \rightarrow @Bc$ B's first consider,

if, $B = a$ then $\epsilon = a$

if, $B = e$ then $\epsilon = e$

if, $B = \epsilon$ then $\epsilon = e$

$\{a, e\}$

| | | |
|---------------------|---------------------|---|
| $S \rightarrow @Bc$ | B's first consider, | |
| if, $B = a$ | then $\epsilon = a$ | |
| if, $B = e$ | then $\epsilon = e$ | A |
| if, $B = \epsilon$ | then $\epsilon = e$ | B |
| $\{a\}$ | $\{e, \epsilon\}$ | C |

Operating System

13-08-2025 (Lab)

(Late)

write = print करे, 058 | 05A 0 ← 2
 gcc write.c -o write

→ read system call → write system call :

nan write

nan 2 write

main = {
 printf
 scanf

#include <unistd.h>
 for add करे
 मगर,

keyboard input = 0 (fd)
 output 1 fd = 1

```
#include <unistd.h>
#include <stdio.h>
int main() {
    int n;
    n = write(1, "Hello\n", 6);
    printf("%d\n", n);
    return 0;
}
```

write(1, "Hello\n", 6)
 1 → stdout
 "Hello\n" → string
 6 → length

nan write.c

↓
 जवना C code काता
 option उभाता

```
#include <unistd.h>
int main() {
    write(1, "Hello", 6);
    return 0;
}
```

save करे फिन
 gcc write.c -o write
 ./write
 एका Hello print करे
 मारो।

$$\text{write}(a, c, \text{store}(c)) ;$$

need system call

```
return 0;
```

10 वादिल गुणवत्ता
value print कराना
इति गुरु ब्रह्म
एव गुरु गुरु
वा एव

2. $\frac{1}{2}$ second: nano read
gcc read - 0.00 read
./read

13-08-2025

LL1 $\rightarrow$ parse Table:

Left to right (scan)

LL1

Left to right scan

Left Most Derivation (LMD)

1- token (scan) $\rightarrow$ taken consider

* LL1: LL1- parse tree making

grammar:

$$e \rightarrow \underline{f} e'$$

underline mark

$$e' \rightarrow + \underline{f} e' \mid \epsilon$$

non terminal

$$+ \rightarrow \underline{f} \underline{f}'$$

प्रभावा रहे

$$f' \rightarrow * \underline{f} \underline{f}' \mid \epsilon$$

$$f \rightarrow (e) \mid x \mid y$$

Soln:

E-সম্পর্কিত Follow()

| Non-Terminals | First() | Follow() |
|---------------|-------------------|------------------------------|
| e | $\{C, x, y\}$ | $\{\$, \cdot\}$ |
| e' | $\{+, \epsilon\}$ | $\{\$, \cdot\}$ |
| t | $\{C, x, y\}$ | $\{+, \cdot, \$, \cdot\}$ |
| t' | $\{*, \epsilon\}$ | $\{+, \cdot, \$, \cdot\}$ |
| f | $\{C, x, y\}$ | $\{*, +, \cdot, \$, \cdot\}$ |

→ new,

~~table with an empty parsing table~~

creating the parse table:

Input Symbols

| Non-Terminals | x | y | $+$ | $*$ | $($ | $)$ | $\$$ |
|---------------|--------------------|--------------------|---------------------------|---------------------------|--------------------|---------------------------|---------------------------|
| e | $e \rightarrow xe$ | $e \rightarrow ye$ | | | $e \rightarrow (e$ | | |
| e' | | | $e' \rightarrow +e$ | | | $e' \rightarrow \epsilon$ | $e' \rightarrow \epsilon$ |
| t | $t \rightarrow xt$ | $t \rightarrow yt$ | | | $t \rightarrow (t$ | | |
| t' | | | $t' \rightarrow \epsilon$ | $t' \rightarrow \epsilon$ | | $t' \rightarrow \epsilon$ | $t' \rightarrow \epsilon$ |
| f | $f \rightarrow xf$ | $f \rightarrow yf$ | | | $f \rightarrow (f$ | | |



input symbols

যেই কলামে উপরে

সিগন বসবে first()

column (কলাম)

$e \rightarrow ($ sign

উপরে $e \rightarrow (e$

ফলস্বরূপ

column (কলাম)

first() column (কলাম)

ফলস্বরূপ

যখন ϵ কে $\text{follow}()$ (৩) যখন symbol considered করা হয় তখন (যখন কোনো symbol নেই) কোনো rule (ধরে নেই)। আর অন্যভাবে বোঝানো হল

DANGLING GRAMMAR। যাওজন LL1
 পিস parse table গঠানো possible না।

* Another example:

Q. Create parse table using LL1 for the grammar:

$P \rightarrow i | a | n TS$

$Q \rightarrow P | a S | b S c S T$

$R \rightarrow b | \epsilon$

$S \rightarrow c | R n | \epsilon$

$T \rightarrow R S g$

প্রদত্ত $\text{start} = G$

জান, প্রদত্ত P, S, T বাক্য
 নির্মাণ, যা T (যা R, S এর
 সাথে যুক্ত নিম্নে হল।

সুতরাং প্রদত্ত G নাকি
 LL1 G (যা LL1 নাকি)

Soln:

| Non-terminals | First() | Follow() |
|---------------|------------------------|---------------------------|
| P | {i, a, n} | {#, \$} |
| Q | {i, c, n, b} | {#} |
| R | {b, ϵ } | {n, c, b, ϵ } |
| S | {c, b, n, ϵ } | {i, a, b, n, ϵ } |
| T | {b, c, n, ϵ } | {c, b, n, #} |

Now creating the parse table:

Rough:

follow first

$P \rightarrow nTS$

$G \rightarrow aS$

$G \rightarrow b$ if $b \in ST$
if $b \in ST$ then find

$T \rightarrow RSa$



if, $R = b$
if then
b is the
first

Now, creating the parse table:

| Terminals | Input Symbol | | | | | | |
|-----------|-------------------|--|--|--------------------|---|---------------------|-------------------|
| | i | c | n | a | b | q | \$ |
| P | $P \rightarrow i$ | $P \rightarrow c$ | $P \rightarrow n$ | | | | |
| G | $G \rightarrow P$ | $G \rightarrow P$ | $G \rightarrow P$ | $G \rightarrow aS$ | $G \rightarrow bS$ | | |
| R | | $R \rightarrow c$ | $R \rightarrow e$ | | $R \rightarrow b$
$R \rightarrow e$ | $R \rightarrow e$ | |
| S | | $S \rightarrow c$
$S \rightarrow e$ | $S \rightarrow n$
$S \rightarrow e$ | | $S \rightarrow Rn$
$S \rightarrow e$ | $S \rightarrow e$ | $S \rightarrow e$ |
| T | | $T \rightarrow RSa$ | $T \rightarrow Rn$ | | $T \rightarrow RSa$ | $T \rightarrow RSa$ | |

$S \rightarrow Rn$ and b, n are

$R \rightarrow b$ or $R \rightarrow e$

if b is follow of R then symbol

consider b or e and then

$R \rightarrow e$ and i is the dangling
grammar

માર્ક સ્કેન કર્યા
બાદ આગળ આગળ
L2 / માર્ક સ્કેન કર્યા
જિલ્લો આગળ L10

Slide (parsing part III)

18-08-2025

LL1 parsing: [માર્ક સ્કેન કર્યા પોઇન્ટર ડાઉન આગળ
આગળ L1]
↳ Syntax analysis લાગુ પડે છે

Top-down Parsing:

→ Recursive Decent Parsing
(required back tracking)
(LMD follow કરે છે)

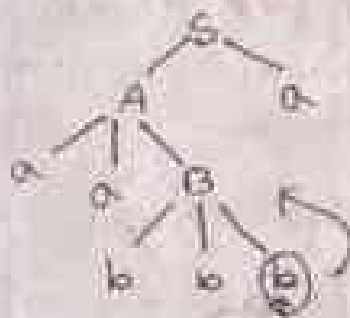
→ Predictive Parsing

Recursive Decent Parsing: (Backtracking)

| | Input → aabbde | i | |
|------|----------------|---|------------|
| Soln | | | 1. S → Aa |
| | | | 2. → ce |
| | | | 3. A → aaB |
| | | | 4. → aaba |
| | | | 5. B → bbb |
| | | | 6. C → aad |
| | | | 7. D → bbd |

Diagram showing the input string **aabbde** with a **pointer** at the first 'a'. Below it, a partial parse tree is shown with root **S** branching into **A** and **a**. **A** further branches into **a**, **a**, and **B**.

Now scan the tree.

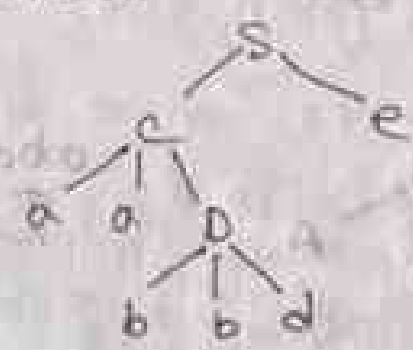


aabbde

↑
x
error
or backtrack

A को एकमात्र
विशेष गुण है
A को backtrack

backtrack:



aabbde

aabbde ↑

अब string read
successfully
completed.

Another example:

$S \rightarrow ABA \mid aaBA$

$A \rightarrow aBa \mid bA$

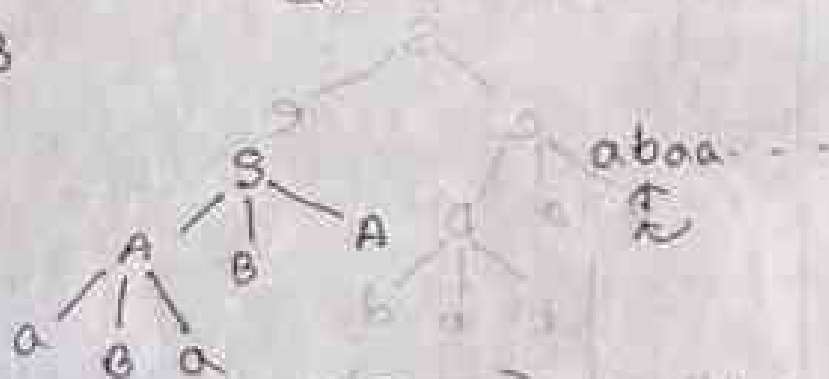
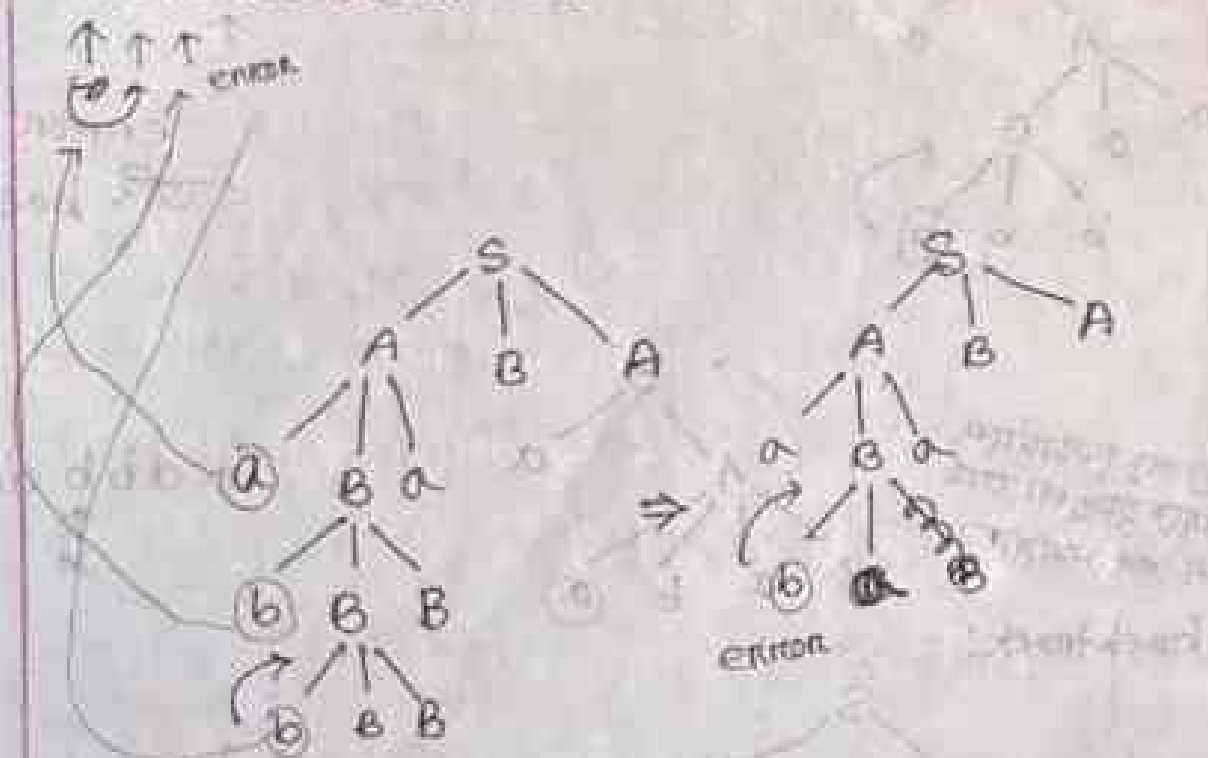
$B \rightarrow bBB \mid ba$

derive: abaabbababa
baa

Soln:

[संयोजन]

↑ ↑ ↑
↑ ↑ ↑



બસ્ટોપ બાદ વચ્ચે મિલ્ય ડાહ્યા ના મિનિમલ બેક ટ્રેક.

predictive parsing : → example 3.3.1

ex:

$(id * id) + id$

$E \rightarrow TE'$

$E \rightarrow +TE | \epsilon$

$T \rightarrow FT'$

$T \rightarrow *FT' | \epsilon$

$F \rightarrow (E) | id$

$(id * id) + id | \$$

← (id * id) + id include

53 slide 4 - প্রস্তুত করা আছে

push serial 4 বা 54 নং

slide 4 চিহ্ন আছে

stack top element = pointer element হলেই pop +
pointer right shift

Cont (a):

stack top = \$

scan pointer = \$

> terminate

LMD:

LMD(1) LMD(2)

LMD
X op

Lab 09: 09/09/2025 (08-09-2025)
10-09-2025 (Lab)

1. Find number of characters, number of alphabets, #digits, #special characters, #lines from file.
2. Validate variables
3. Find different types of variables.
4. Find comments in a given file.
5. find keywords, numbers in c file.
6. Parenthesis matching (with comments)
7. Validate string by a given regular expression.
8. Write program to remove left recursion from a grammar production set.
write a program to
9. Find First from a given grammar production set.
write a program to
10. Find Follow from a given production set.

new lab 9
evaluation

11. Generate LL(1) parser.

12. Validate a given string using a LL(1) parser.

* First set:

$E \rightarrow TE'$

$E' \rightarrow +TE' | \epsilon$

$T \rightarrow FT'$

$T' \rightarrow *FT' | \epsilon$

$F \rightarrow id | (E)$

id
+
*
(
)

id
+
*
(
)

const string



problem

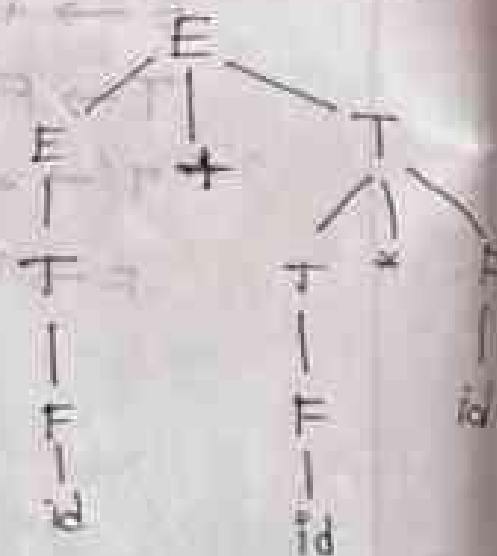
Grammar →
 $E \rightarrow E + T \mid T$
 $T \rightarrow T * F \mid F$
 $F \rightarrow (E) \mid id$

free
 for the string,
 $w = id + id * id$

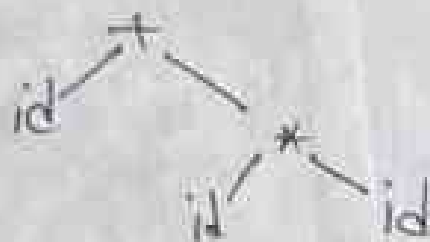
Now,

$E \rightarrow E + T$
 $\rightarrow T + T$
 $\rightarrow T + T * F$
 $\rightarrow F + F * F$
 $\rightarrow id + id * id$

parse tree



Syntax tree



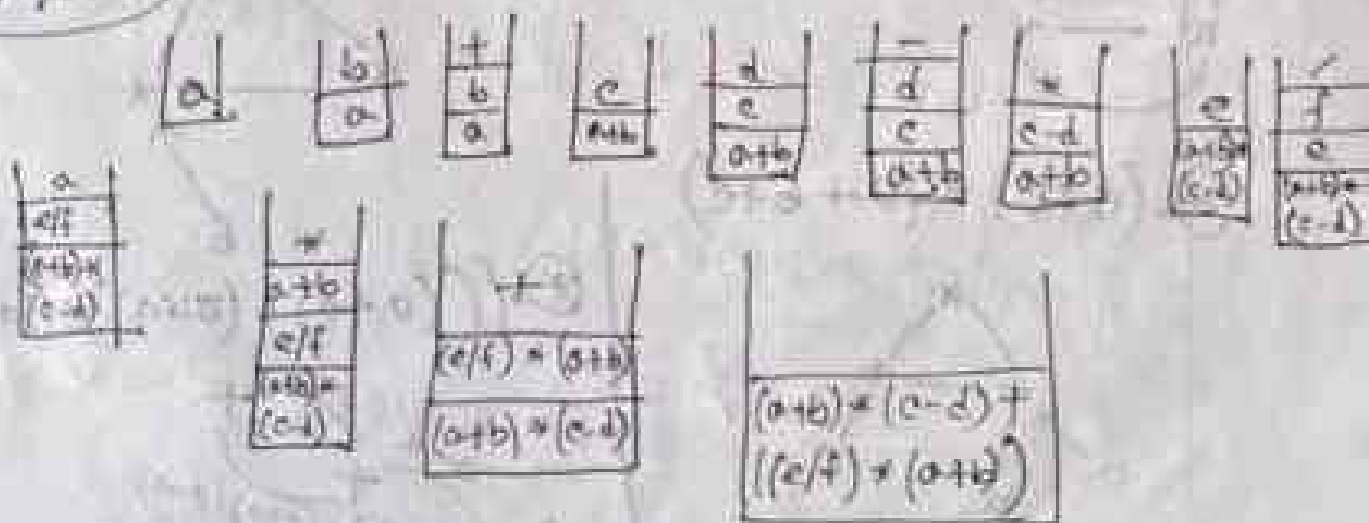
postfix conversion:

sp103

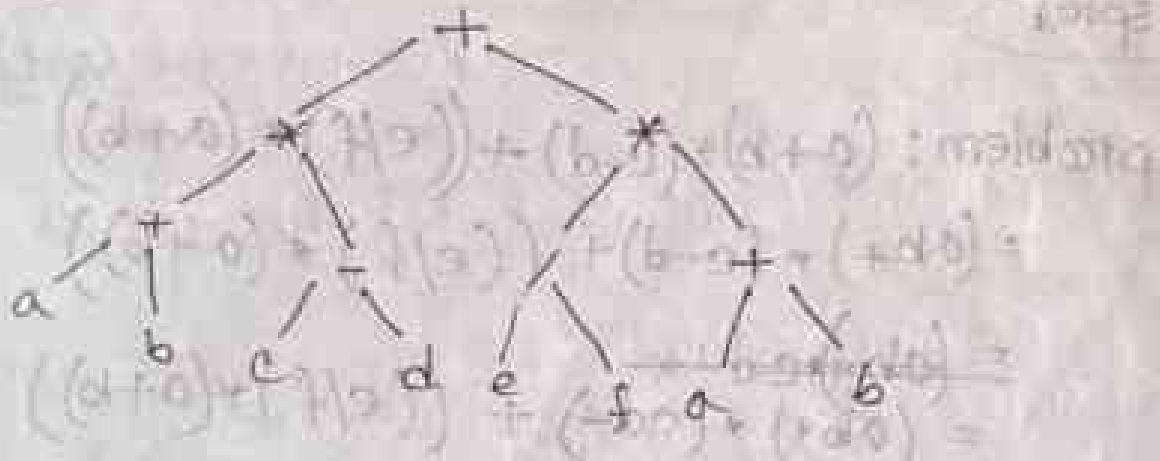
$$\begin{aligned}
 \text{problem: } & (a+b)*(c-d) + ((e/f)*(a+b)) \\
 & = (ab+) * (c-d) + ((e/f) * (a+b)) \\
 & = \cancel{(ab+)} ed - * \\
 & = (ab+) * (cd-) + ((e/f) * (a+b)) \\
 & = (ab+cd-*) + ((e/f) * (a+b)) \\
 & = (ab+cd-*) + ((\cancel{ef/}) * (a+b)) \\
 & = (ab+cd-*) + ((ef/) * (ab+)) \\
 & = (ab+cd-*) + (ef/ab+*) \\
 & = ab+cd-ef/ab+*+
 \end{aligned}$$

stack

step 2



Syntax Tree:

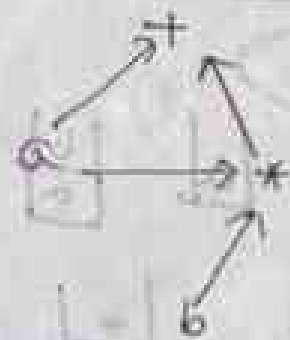
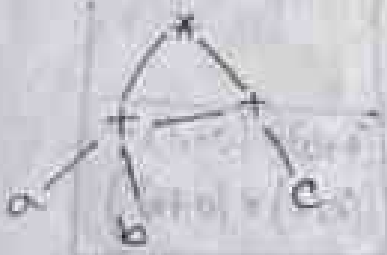


□

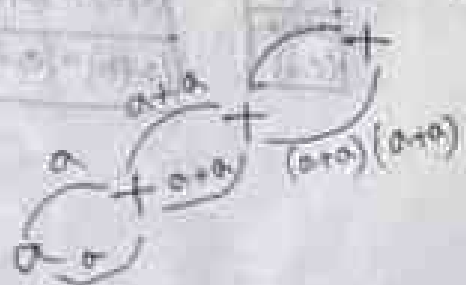
DAG



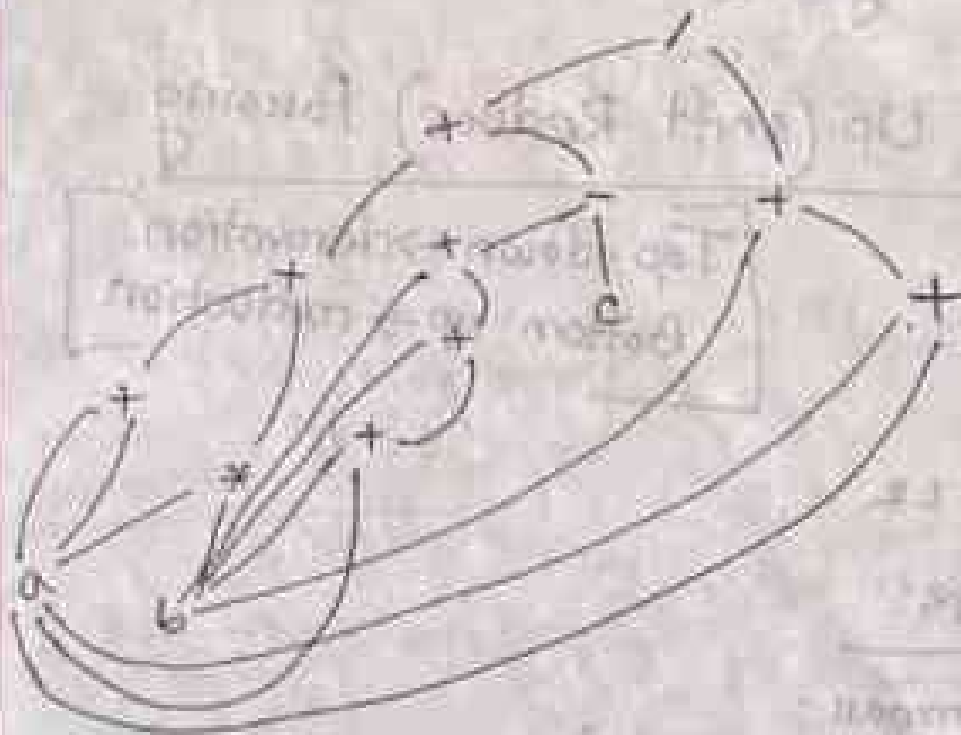
⊗. (a+b) * (a+b+c)



⊗. (((a+a)+(a+a))+((a+a)+(a+a)))



⑥ $((a+b) + (a+a) + (a+b+b+b)) / (a+a+b)$



(0+0+0) (08-09-2025) + (0+0)+(0+0) (2)

(LR(0) parsing slide)

x. Bottom Up (shift Reduce) Parsing

Top-down $\rightarrow$ derivation
Bottom up $\rightarrow$ reduction

x. LT vs LR

page-48.

Grammar:

$S \rightarrow (L)$

$S \rightarrow X$

$L \rightarrow S$

$L \rightarrow L, S$

canonical Diagram & LR(0) parse table.

Soln:

Augmented grammar:

$S' \rightarrow S \$$

$S \rightarrow (L)$

$S \rightarrow X$

$L \rightarrow S$

$L \rightarrow L, S$

var variable ko mer \$

variables same

closure:

$S' \rightarrow \cdot S \$$

scanner जिन्हें

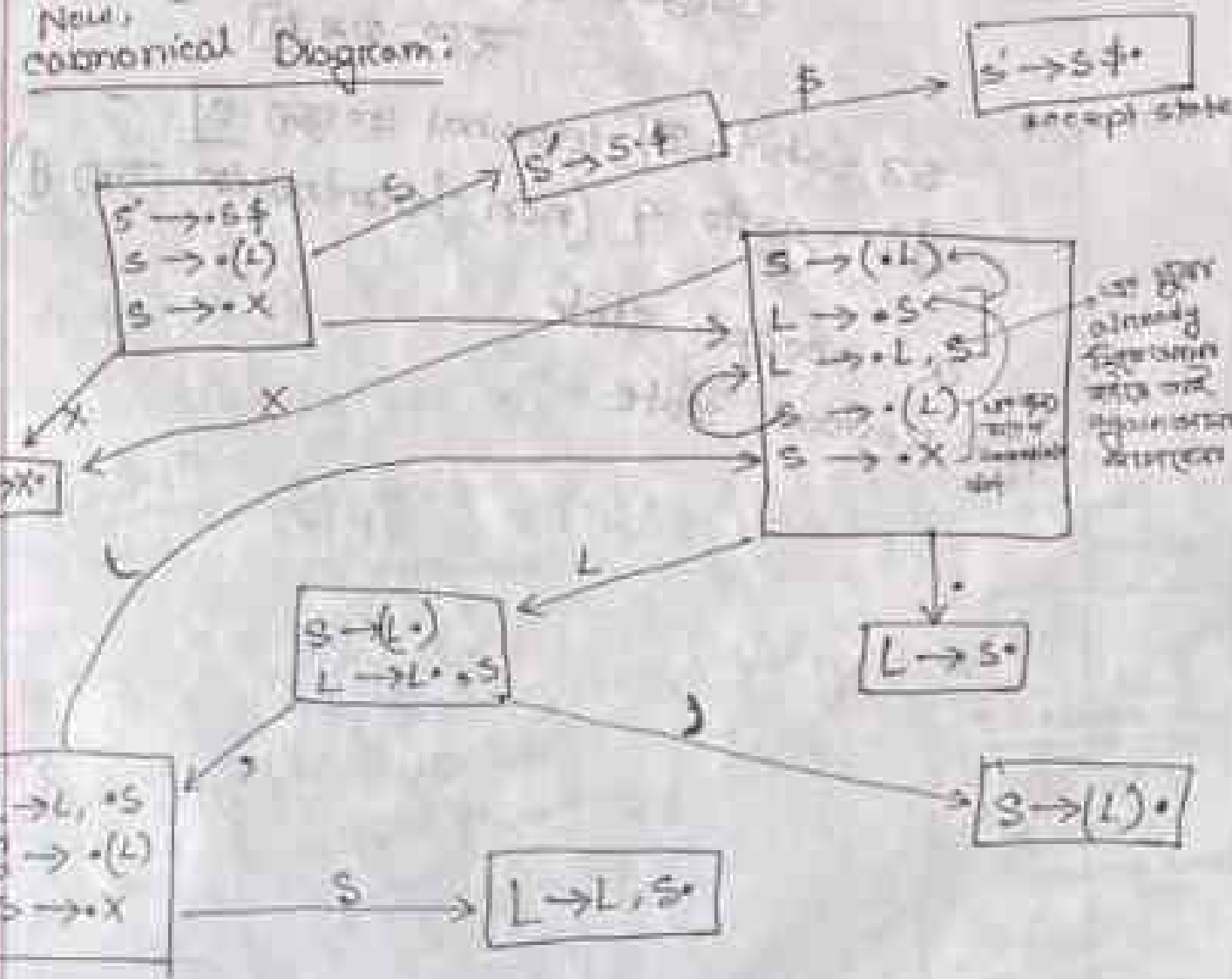
S का immediate भाग है उसे जो S का शुरू भाग।

$S \rightarrow \cdot (L)$

$S \rightarrow \cdot X$

Closure जो $\cdot$ के भाग है

New, Canonical Diagram:



Three Address Code:

આમાં 3જો add. તો જમ 2(5)
મારું સિદ્ધાંત લેવા લા.

Generate 3 address code for.

$$\cancel{a+b+c}$$

$$a = b + c + d$$

Solⁿ:

$$t1 = b + c$$

$$t2 = t1 + d$$

$$a = t2$$

→ 3 address

Another:

$$-(a+b) + (c+d) = (a+b+c+d)$$

Solⁿ:

$$t1 = a + b$$

$$t2 = \text{minus } t1$$

$$t3 = c + d$$

$$t4 = t2 + t3$$

$$t5 = a + b$$

$$t6 = t5 + t3$$

$$t7 = t4 - t6$$

- S
Unary
operation

variable નામ
ઠાણમાં સિદ્ધાંત
લુટિ line 6
3જો variable ના
મારું ઠાણમાં,
કેવું 3 address
code

3 address code

લુટિ line 6

3જો લુટિ

variable નામ

મા,

અહીં 3 જમ

મા અસહિત

Lee-18 (Farhan)

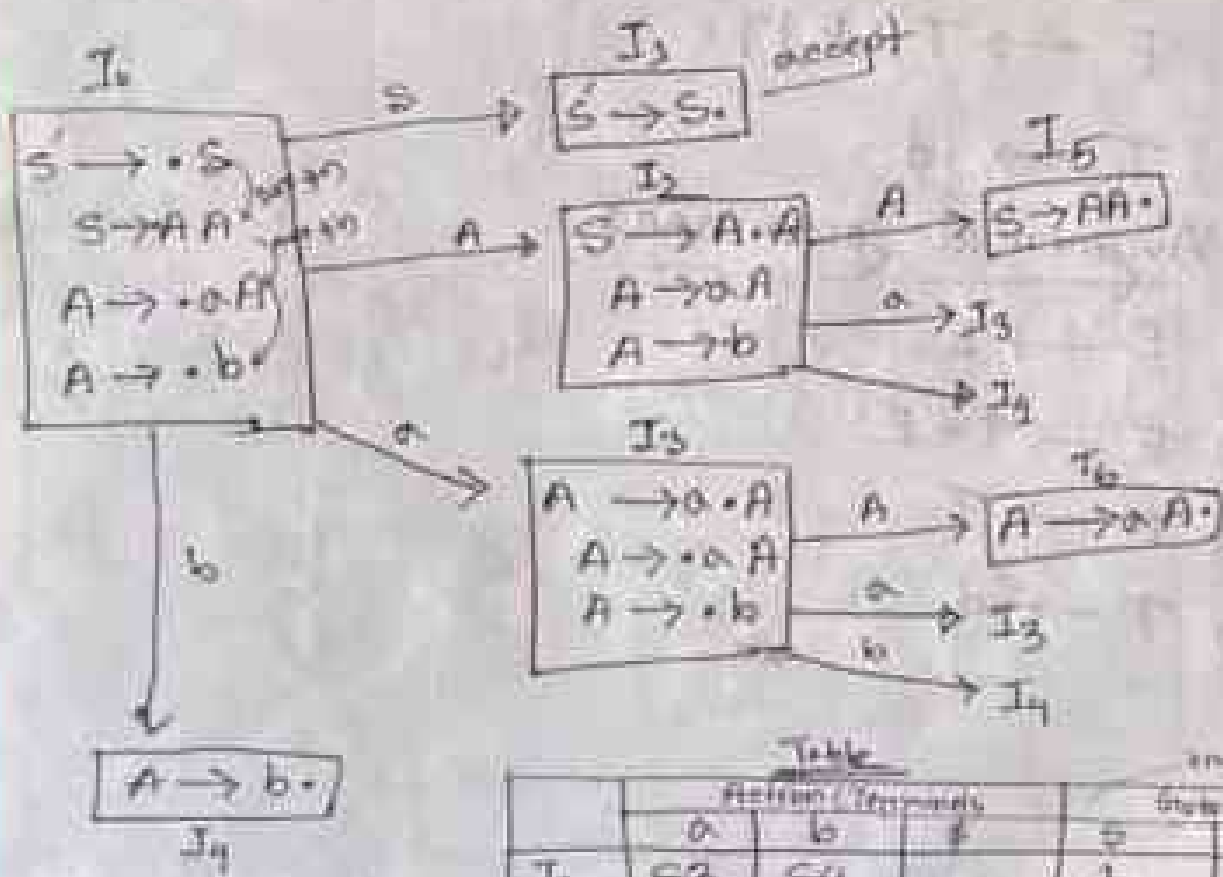
LR(0) parsing Table

Grammar

$S \rightarrow AA$
 $A \rightarrow aA | b$

Augmented Grammar:

$S' \rightarrow S$
 $S \rightarrow AA$
 $A \rightarrow aA | b$



Item sets:

1. $S' \rightarrow \cdot S$
2. $S \rightarrow \cdot AA$
3. $A \rightarrow \cdot aA$
4. $A \rightarrow \cdot b$

Table

| | Action / Transitions | | | State Transitions | |
|----------------|----------------------|----|--------|-------------------|---|
| | a | b | \$ | S | A |
| I ₀ | S3 | S4 | | 1 | 2 |
| I ₁ | | | accept | 0 | 0 |
| I ₂ | S3 | S4 | | 0 | 5 |
| I ₃ | S3 | S4 | | | 6 |
| I ₄ | R3 | R3 | R3 | | |
| I ₅ | R1 | R1 | R1 | | |

8. Reduction

→ Scanner लांछ (साह आह) ।
अमीकरणक numbering का R वर आह काह ।

हीन (ज I वरकन) अरि काहण A काहण (अमीकरणक)

*. Example of Not LR(0) Grammar:

$$E \rightarrow T + E \mid T$$

$$T \rightarrow id$$

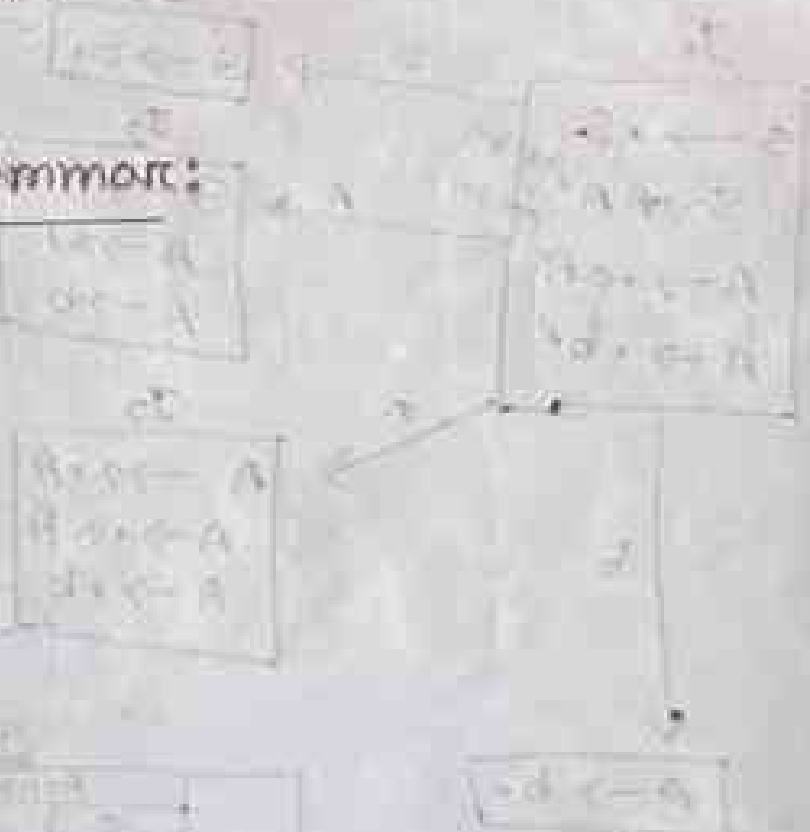
Augmented grammar:

① $E \rightarrow E$

② $E \rightarrow T + E$

③ $E \rightarrow T$

④ $T \rightarrow id$



| State | id | + | E |
|-------|----|---|---|
| 0 | 1 | 3 | 2 |
| 1 | | | |
| 2 | | | 0 |
| 3 | | | 0 |
| 4 | | | 0 |
| 5 | | | |

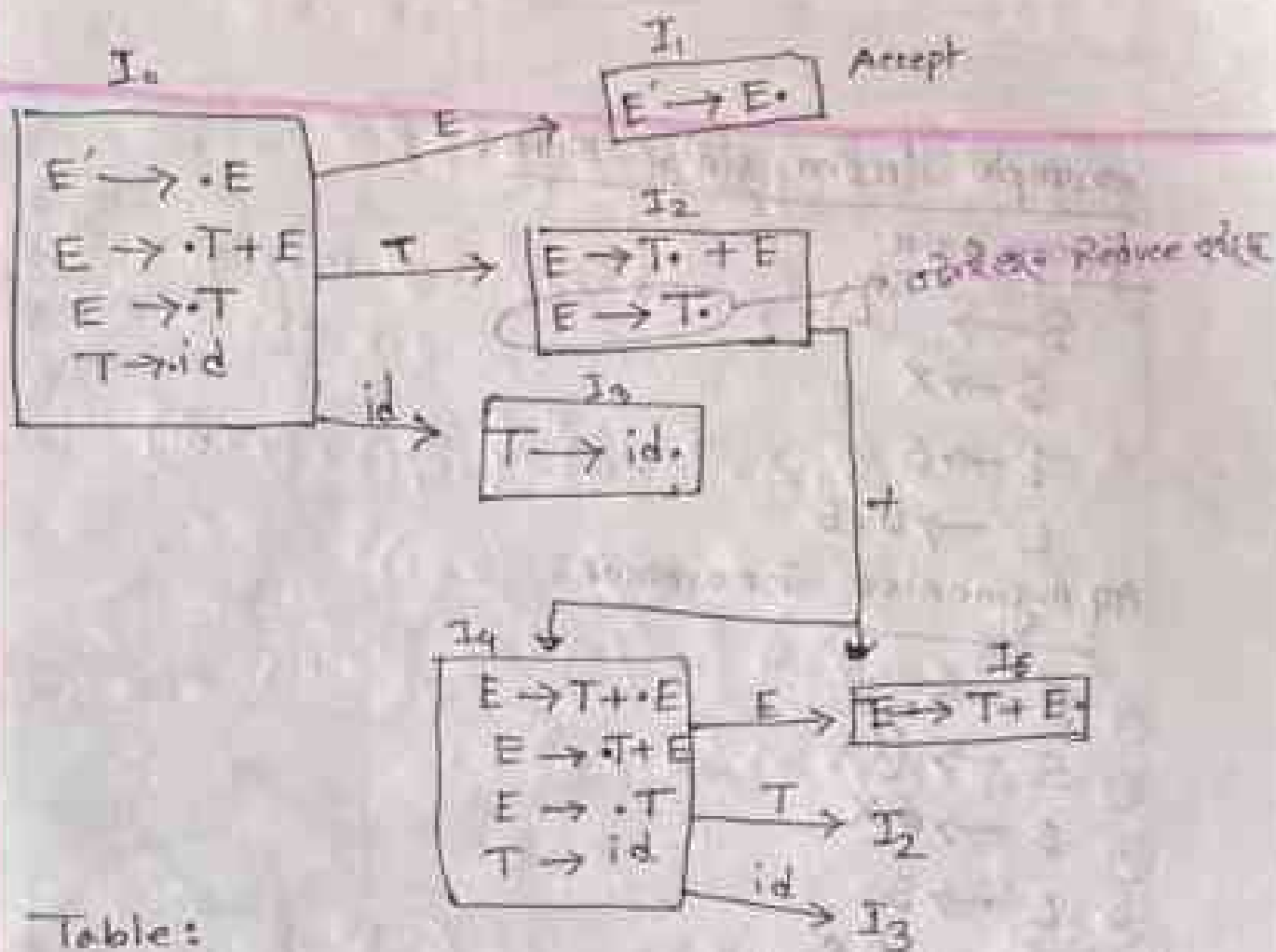


Table:

| | Action | | | Goto | |
|----------------|--------|-------|--------|------|---|
| | id | + | * | E | T |
| I ₀ | S3 | | | 1 | 2 |
| I ₁ | | | Accept | | |
| I ₂ | R3 | S3/R3 | R2 | | |
| I ₃ | R4 | R4 | R4 | | |
| I ₄ | S3 | | | 5 | 2 |
| I ₅ | R2 | R2 | R2 | | |

For that reason it is not LR(0) parser

example from sin's slide:

Grammar:

$$S \rightarrow (L)$$

$$S \rightarrow X$$

$$L \rightarrow S$$

$$L \rightarrow L, S$$

Augmented Grammar:

$$S' \rightarrow S \#$$

$$S \rightarrow (L)$$

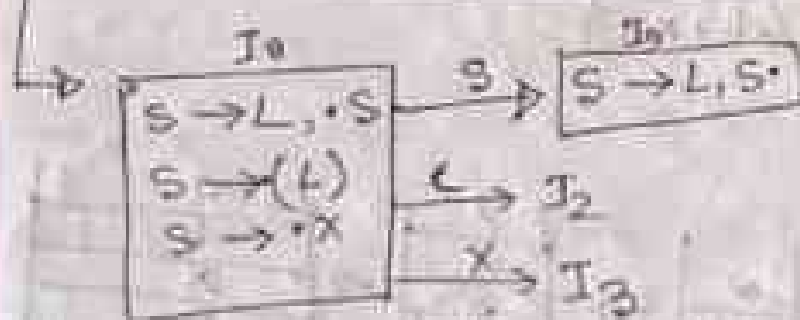
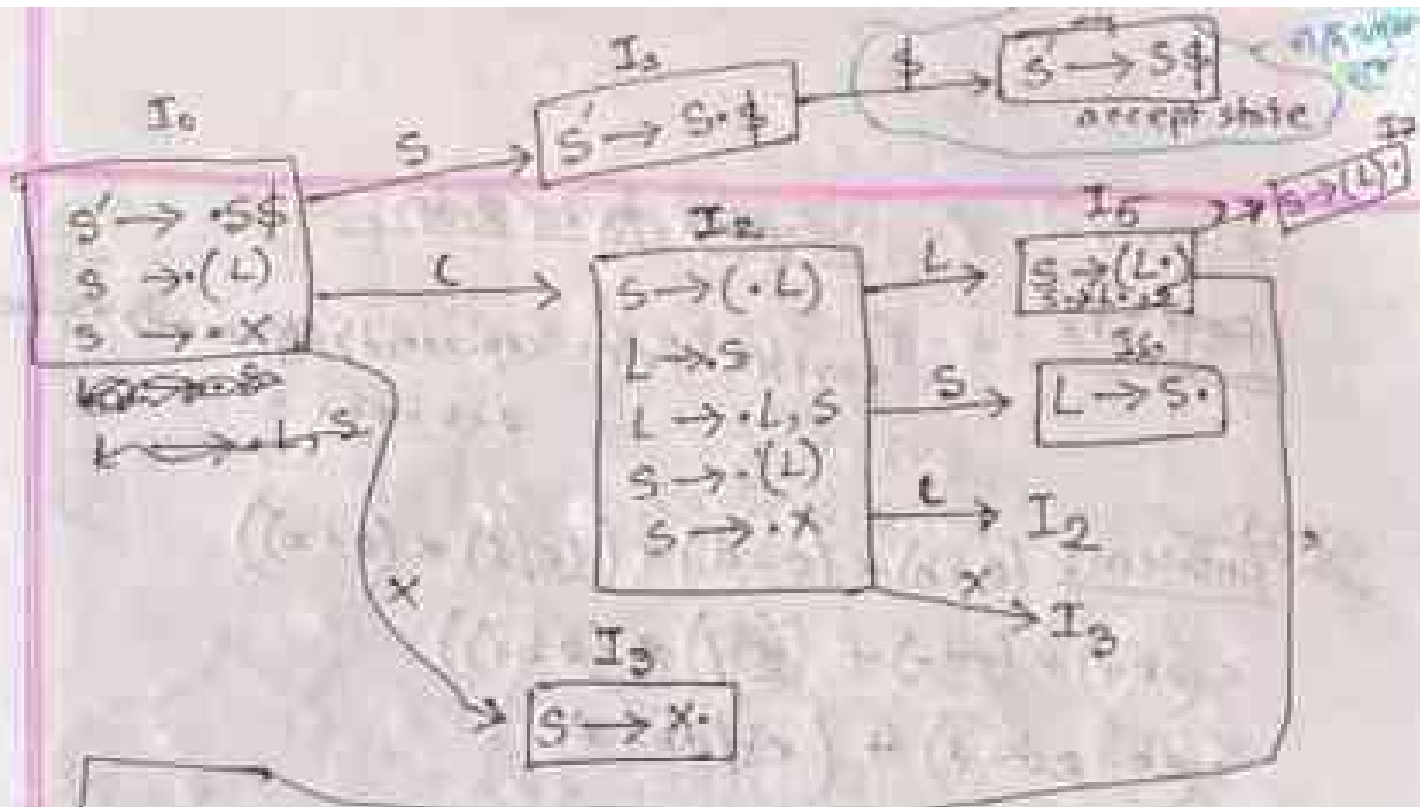
$$S \rightarrow X$$

$$L \rightarrow S$$

$$L \rightarrow L, S$$

Now,

Canonical Diagram:



LR(0) parse Table

| States | Action | | | | | goto | |
|--------|--------|----|----|----|----|------|----|
| | L |) | X | , | \$ | S | L |
| I0 | S2 | | S3 | | | S1 | |
| I1 | R2 | R2 | R2 | R2 | S2 | | |
| I2 | S2 | | S3 | | | S6 | S5 |
| I3 | R3 | R3 | R3 | R3 | R3 | | |
| I4 | | | | | | | |
| I5 | | S7 | | S8 | | | |
| I6 | R4 | R4 | R4 | R4 | R4 | | |
| I7 | R2 | R2 | R2 | R2 | R2 | | |
| I8 | S2 | | S3 | | | S9 | |
| I9 | R5 | R5 | R5 | R5 | R5 | | |

Date: 15-09-2025

Three Address Code:

આઈએસએસ 3-addr. કોડ: ત્રણ સ્થાન 2(ક) આપવાનું છે.

Generate 3 address code for,

$$a = b + c + d$$

Soln:

$$\begin{aligned} t1 &= b + c \\ t2 &= t1 + d \\ a &= t2 \end{aligned}$$

→ 3 address

Another:

$$-(a+b) + (c+d) = (a+b+c+d)$$

Soln:

$$\begin{aligned} t1 &= a + b \\ t2 &= \text{uminus } t1 \\ t3 &= c + d \\ t4 &= t2 + t3 \\ t5 &= a + b \\ t6 &= t5 + t3 \\ t7 &= t4 - t6 \end{aligned}$$

→ unary operation

variable નામ
આપવાનું છે
જો line 4
માં variable ના
નામ આપવાનું
છે તો 3 address
code

3-address code
જો line 4
માં જોઈ
variable નામ
આપવાનું
છે તો 3-addr
કોડ બનશે

* Directed Acyclic Graph (DAG) for the following block -

$$a = b * c$$

$$d = b$$

$$e = d * e$$

$$b = e$$

$$f = b * c$$

$$g = f * d$$

Graph:



* Implementation of Three address code:

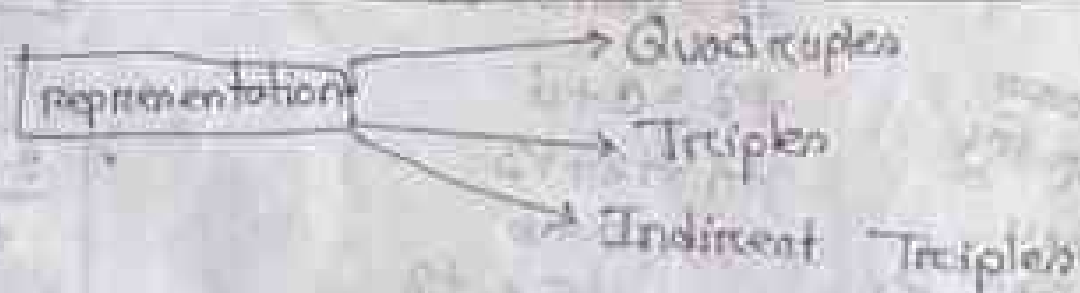


Illustration-01:

$$a + b * c / e + f + b * a$$

Solⁿ:

TS = e, f

priority for power

Indirect triplet → memory index

quasi-notation

sub-notation

Another example:

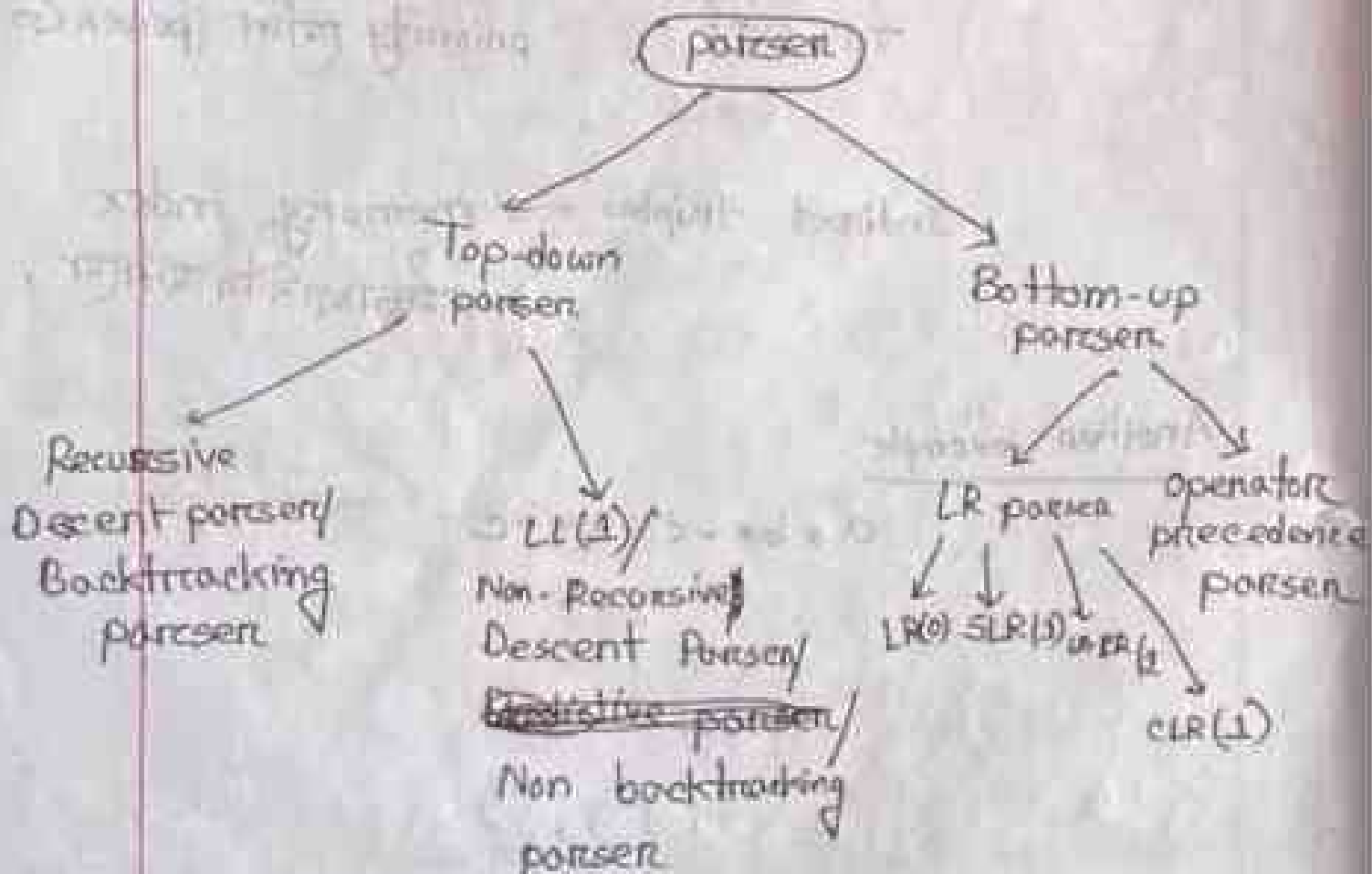
$$a * b * c + b * c$$

memory

memory index

Youtube

Types of parser:



→ Function

* Lec. 13: find First() & Follow()

First(): The first set of nonterminal symbol represents the set of terminal symbols that can appear as the first token in any string derived from non-terminal.

① $S \rightarrow Bbc$
 $B \rightarrow aB$
 $C \rightarrow c/d$

First:
 $C = \{c, d\}$
 $B = \{a\}$
 $S = \{a\}$

② $S \rightarrow Bbc/cd$
 $B \rightarrow aB/\epsilon$
 $C \rightarrow cC/\epsilon$

First:
 $C = \{c, \epsilon\}$
 $B = \{a, \epsilon\}$
 $S = \{a, b, c, d\}$

③ $S \rightarrow AB$
 $A \rightarrow aA/\epsilon$
 $B \rightarrow bB/c/d$

First:
 $B = \{b, c\}$
 $A = \{a, \epsilon\}$
 $S = \{a, b, c\}$

Diagram showing derivation for $S \rightarrow AB$:
 $S \rightarrow AB \xrightarrow{a, \epsilon} aAB \xrightarrow{a, \epsilon} aaAB \dots$
 $S \rightarrow AB \xrightarrow{b, c, d} aB \xrightarrow{b, c, d} abB \dots$

④ $S \rightarrow AB$ First()

$A \rightarrow a|b|c$

$B \rightarrow b|c|d$

$B = \{b, c, d\}$

$A = \{a, b, c\}$

$S \rightarrow AB$

$\rightarrow aAB$

$\rightarrow \epsilon \cdot B$

$\rightarrow B$

$\rightarrow bB$

$\rightarrow cB$

$\rightarrow d$

⑤ $E \rightarrow TE'$

$E' \rightarrow +TE' | \epsilon$

$T \rightarrow FT' | \epsilon$

$T' \rightarrow *FT' | \epsilon$

$F \rightarrow id | (E)$

First

$F = \{id, (\}$

$T' = \{*, \epsilon\}$

$T = \{id, (\}$

$E' = \{+, \epsilon\}$

$E = \{id, (\}$

⑥ $S \rightarrow ACB | CbB | Ba$

$A \rightarrow d|g|c$

$B \rightarrow g|c$

$C \rightarrow h|c$

First

$C = \{h, c\}$

$B = \{g, c\}$

$A = \{d, g, c, h, \epsilon\}$

$S \rightarrow ACB | CbB | Ba$

$A \rightarrow ACB$

$\rightarrow dCB$

$\rightarrow gCB$

$\rightarrow cCB$

$\rightarrow hCB$

$\rightarrow \epsilon CB$

$\rightarrow CB$

$\rightarrow CbB$

$\rightarrow Ba$

$S \rightarrow CbB$ $C = \{c, k, f\}$
 $\rightarrow h.b.B$ h already on stack
 $\rightarrow f.b.B$ f already on stack
 $\rightarrow b.B$ b add on stack

$S \rightarrow Ba$ $B = \{g, f\}$
 $\rightarrow g.a$ g already on stack
 $\rightarrow f.a$
 $\rightarrow a$ a add on stack

* Follow()

1. या Follow एव वसता है, जो immediate right side के एक था वह जो pick करता है।

$\Rightarrow$ Right side के यदि terminal symbol था तो उसका (अर्थात्) नाम बताना।

$\Rightarrow$ Right side के यदि Non-terminal था तो उसका जो First() दिया जाए, First() का Value बताना।

$\Rightarrow$ Right side के यदि सिर्फ का था तो उसका नाम

Follow of L.H.S = Follow of R.H.S

④

$S \rightarrow ABd$

$B \rightarrow b$

एक terminal symbol के लिए कुछ नामकरण।

FOLLOW

$S = \{ \$ \}$

$B = \{ d \}$

$\rightarrow ABd$

$$(2) S \rightarrow ABD$$

$$B \rightarrow b$$

$$D \rightarrow d$$

Follow

$$S \rightarrow \{ \$ \}$$

$$B = \{ d \}$$

$$D \rightarrow \{ d \}$$

$$S \rightarrow ABD$$

non-terminal
D is Final for empty

$$D \rightarrow d$$

$$D = \{ \$ \}$$

(1) empty set

$$S \rightarrow ABD$$

To start with
D is Follow (d)
Follow(D)

$$(3) S \rightarrow ABDC$$

$$B \rightarrow b | \epsilon$$

$$D \rightarrow d | \epsilon$$

Follow

$$S \rightarrow \{ \$ \}$$

$$B = \{ d, c \}$$

$$S \rightarrow ABDC$$

First

$$D = \{ d, \epsilon \}$$

$$\rightarrow ABDC$$

$$\rightarrow ABCE$$

$$\rightarrow AB@$$

$$D = \{ c \}$$

$$(4) S \rightarrow ABD$$

$$B \rightarrow b | \epsilon$$

$$D \rightarrow d | \epsilon$$

Follow

$$S \rightarrow \{ \$ \}$$

$$F(B) = \{ d, \$ \}$$

$$S \rightarrow ABCE$$

$$F(D) = \{ \$ \}$$

First

$$D = \{ d, \epsilon \}$$

Example:

| | |
|----------------------------------|-------------------------|
| $E \rightarrow TE'$ | $\{ \epsilon, \cdot \}$ |
| $E' \rightarrow +TE' \epsilon$ | $\{ \epsilon, \cdot \}$ |
| $T \rightarrow FT'$ | $\{ \epsilon, \cdot \}$ |
| $T' \rightarrow *FT' \epsilon$ | $\{ \epsilon, \cdot \}$ |
| $F \rightarrow id (E)$ | $\{ \epsilon, \cdot \}$ |

Soln:

| | |
|------------------------------|--------------------------|
| <u>Follow</u> | <u>First</u> |
| $E = \{ \epsilon, \cdot \}$ | $E' = \{ +, \epsilon \}$ |
| $E' = \{ \epsilon, \cdot \}$ | $T' = \{ *, \epsilon \}$ |

$E \rightarrow TE'$
 $\rightarrow$ right a sign and
 $F(E) = F(T)$
 $E' \rightarrow +TE' | \epsilon$
 $F(E) = F(E)$
 $T = \{ +, \epsilon, \cdot \}$
 $F = \{ id, (, \cdot \}$
 $T' = \{ *, \epsilon, \cdot \}$
 $F(T) = F(T)$
 $F(T') = F(T')$
 $F = \{ \epsilon, +, \cdot \}$

$E \rightarrow TE'$
 $\rightarrow$ right a sign and
 $F(E) = F(T)$
 $E' \rightarrow +TE' | \epsilon$
 $\rightarrow$ right a sign and
 $F(E) = F(T)$

Ex-15: First() & Follow()

| | First | Follow |
|-----------------------|-------------------|-----------------|
| $S \rightarrow ABCDE$ | $\{a, b, c\}$ | $\{\$ \}$ |
| $A \rightarrow a E$ | $\{a, \epsilon\}$ | $\{b, c\}$ |
| $B \rightarrow b E$ | $\{b, \epsilon\}$ | $\{c\}$ |
| $C \rightarrow c$ | $\{c\}$ | $\{d, e, \$ \}$ |
| $D \rightarrow d E$ | $\{d, \epsilon\}$ | $\{e, \$ \}$ |
| $E \rightarrow e E$ | $\{e, \epsilon\}$ | $\{\$ \}$ |

Ex-16

Another example

| | First | Follow |
|-----------------------------|-------------------------------|-------------------|
| $S \rightarrow ACB CBA BCA$ | $\{a, b, h, \epsilon, b, a\}$ | $\{\$ \}$ |
| $A \rightarrow da BC$ | $\{d, a, h, \epsilon\}$ | $\{h, g, \$ \}$ |
| $B \rightarrow g E$ | $\{g, \epsilon\}$ | $\{\$, a, h, g\}$ |
| $C \rightarrow h E$ | $\{h, \epsilon\}$ | $\{g, \$, b, h\}$ |

→ Lec-16

$A \rightarrow a$
A derives a

* LL(1) Parsing Table:

22-11-2020

example-1:

| Initial | Final | First | Follow |
|--------------------------|---------|---------|------------|
| $S \rightarrow AaB$ | $\{a\}$ | $\{a\}$ | $\{ \$ \}$ |
| $A \rightarrow a$ | $\{a\}$ | $\{a\}$ | $\{a\}$ |
| $B \rightarrow \epsilon$ | $\{ \}$ | $\{ \}$ | $\{ \$ \}$ |

| Non-Terminal | a | \$ |
|--------------|---------------------|--------------------------|
| S | $S \rightarrow AaB$ | |
| A | $A \rightarrow a$ | |
| B | | $B \rightarrow \epsilon$ |

condition
Follow(B)
consideration during
shift.

Ans A
A correct A

Ans B

example-2:

| Non-terminal | Terminal | First | Follow |
|--------------|----------------------------------|-----------------------|-------------------|
| E | $E \rightarrow TE'$ | $\{id, (, \epsilon\}$ | $\{\$,)\}$ |
| E' | $E' \rightarrow +TE' \epsilon$ | $\{+, \epsilon\}$ | $\{\$,)\}$ |
| T | $T \rightarrow FT'$ | $\{[, id\}$ | $\{+, \$,)\}$ |
| T' | $T' \rightarrow *FT' \epsilon$ | $\{*, \epsilon\}$ | $\{+, \$,)\}$ |
| F | $F \rightarrow id (E)$ | $\{id, (\}$ | $\{*, +, \$,)\}$ |

Solⁿ:

| Non-terminal | id | (| * | + | \$ |
|--------------|---------------------|---------------------|---------------------------|---|---------------------------|
| E | $E \rightarrow TE'$ | $E \rightarrow TE'$ | | $E \rightarrow TE'$ | |
| E' | | | $E' \rightarrow \epsilon$ | $E' \rightarrow +TE'$ | $E' \rightarrow \epsilon$ |
| T | $T \rightarrow FT'$ | $T \rightarrow FT'$ | | | |
| T' | | | $T' \rightarrow *FT'$ | $T' \rightarrow \epsilon$ | $T' \rightarrow \epsilon$ |
| F | $F \rightarrow id$ | $F \rightarrow (E)$ | | | |

Ans:

$$E \rightarrow TE'$$

$$T \rightarrow \{[, id\}$$

Ans:

$$E' \rightarrow +TE' | \epsilon$$

$$E' \rightarrow \epsilon$$

Follow(E')
= $\{\$,)\}$

or:

$$T \rightarrow FT'$$

$$F \rightarrow id | (E)$$

$\rightarrow \{id, (\} \rightarrow \text{First}$

Ans:

$$T' \rightarrow *FT'$$

$$T' \rightarrow \epsilon$$

$$= \{+, \$,)\}$$

Example: $F \rightarrow id$
 $F \rightarrow (E)$

$F \rightarrow id$
 $F \rightarrow (E)$

Ques-17: Check the given grammar is LL(1) or not?

Ans:

| | First | Follow |
|-----------------------|-------|--------|
| $S \rightarrow A B$ | {a} | { \$ } |
| $A \rightarrow aC$ | {a} | { \$ } |
| $B \rightarrow aD$ | {a} | { \$ } |
| $C \rightarrow c$ | {c} | { + } |
| $D \rightarrow d$ | {d} | { \$ } |

Soln:

| Non-Terminal | a | c | d | \$ |
|--------------|---|-------------------|-------------------|----|
| S | $S \rightarrow A$ | | | |
| A | $A \rightarrow aC$ | | | |
| B | $B \rightarrow aD$ | | | |
| C | $C \rightarrow c$ | $C \rightarrow c$ | | |
| D | $D \rightarrow d$ | | $D \rightarrow d$ | |

Conflict situation for LL(1) parser as
 same non-terminal has multiple productions

So, not LL(1) parser.

Example (sin's slide (1st page) → Page 36)

$S \rightarrow aSe/B$
 $B \rightarrow bBcf/c$
 $c \rightarrow ccg/d/\epsilon$

First & Follow:

| | First() | Follow() |
|--------------------------------|------------------------|-----------------------------|
| $S \rightarrow aSe/B$ | {a, b, c, d, \epsilon} | {\\$, \epsilon} |
| $B \rightarrow bBcf/c$ | {b, c, d, \epsilon} | {c, d, f, \\$, \epsilon} |
| $c \rightarrow ccg/d/\epsilon$ | {c, d, \epsilon} | {g, c, d, f, \\$, \epsilon} |

First in

$B \rightarrow bBcf/c$
 $B \rightarrow bBcf$
 $\rightarrow bBcf$
 $\rightarrow bBcf$
 $\rightarrow bBcf-bcf$
 $B \rightarrow B\Box$

$B \rightarrow bBcf/c$
 $B \rightarrow bBcf$
 $\rightarrow bBcf$
 $\rightarrow bBcf$
 $\rightarrow bBcf-bcf$
 $B \rightarrow B\Box$

First & Follow:
 Follow(c) = {d, g, \\$}

$B \rightarrow c$
 (1) (2) (3) (4) (5) (6) (7) (8) (9) (10) (11) (12) (13) (14) (15) (16) (17) (18) (19) (20) (21) (22) (23) (24) (25) (26) (27) (28) (29) (30) (31) (32) (33) (34) (35) (36) (37) (38) (39) (40) (41) (42) (43) (44) (45) (46) (47) (48) (49) (50) (51) (52) (53) (54) (55) (56) (57) (58) (59) (60) (61) (62) (63) (64) (65) (66) (67) (68) (69) (70) (71) (72) (73) (74) (75) (76) (77) (78) (79) (80) (81) (82) (83) (84) (85) (86) (87) (88) (89) (90) (91) (92) (93) (94) (95) (96) (97) (98) (99) (100)

Examples (sin's lecture)

$S \rightarrow (A) | \epsilon$
 $A \rightarrow TE$
 $E \rightarrow bTE | \epsilon$
 $T \rightarrow (A) | a | b | c$

First & Follow:

| | First() | Follow() |
|---------------------------------|---------------|-----------------|
| $S \rightarrow (A) \epsilon$ | {(, \epsilon} | {\\$, \epsilon} |
| $A \rightarrow TE$ | {(, a, b, c} | {\\$, \epsilon} |
| $E \rightarrow bTE \epsilon$ | {b, \epsilon} | {\\$, \epsilon} |
| $T \rightarrow (A) a b c$ | {(, a, b, c} | {\\$, \epsilon} |

Example:

$S \rightarrow (A) | \epsilon$
 $A \rightarrow TE$
 $E \rightarrow bTE | \epsilon$
 $T \rightarrow (A) | a | b | c | \epsilon$

First & Follow:

| | First() | Follow() |
|--|------------------------|-----------------|
| $S \rightarrow (A) \epsilon$ | {(, \epsilon} | {\\$, \epsilon} |
| $A \rightarrow TE$ | {(, a, b, c, \epsilon} | {\\$, \epsilon} |
| $E \rightarrow bTE \epsilon$ | {b, \epsilon} | {\\$, \epsilon} |
| $T \rightarrow (A) a b c \epsilon$ | {(, a, b, c, \epsilon} | {\\$, \epsilon} |

$A \rightarrow TE$
 $E \rightarrow bTE | \epsilon$
 $T \rightarrow (A) | a | b | c | \epsilon$

Examples (sing's lecture)

$$S \rightarrow (A) | \epsilon$$

$$A \rightarrow TE$$

$$E \rightarrow bTE | \epsilon$$

$$T \rightarrow (A) | a | b | c$$

First & Follow:

| Production | First() | Follow() |
|---------------------------------|---------------------|--------------|
| $S \rightarrow (A) \epsilon$ | $\{ (, \epsilon \}$ | $\{ \$ \}$ |
| $A \rightarrow TE$ | $\{ (, a, b, c \}$ | $\{) \}$ |
| $E \rightarrow bTE \epsilon$ | $\{ b, \epsilon \}$ | $\{) \}$ |
| $T \rightarrow (A) a b c$ | $\{ (, a, b, c \}$ | $\{ b,) \}$ |

Example:

$$S \rightarrow (A) | \epsilon$$

$$A \rightarrow TE$$

$$E \rightarrow bTE | \epsilon$$

$$T \rightarrow (A) | a | b | c | \epsilon$$

First & Follow:

| | First() | Follow() |
|--|------------------------------|--------------|
| $S \rightarrow (A) \epsilon$ | $\{ (, \epsilon \}$ | $\{ \$ \}$ |
| $A \rightarrow TE$ | $\{ (, a, b, c, \epsilon \}$ | $\{) \}$ |
| $E \rightarrow bTE \epsilon$ | $\{ b, \epsilon \}$ | $\{) \}$ |
| $T \rightarrow (A) a b c \epsilon$ | $\{ (, a, b, c, \epsilon \}$ | $\{ b,) \}$ |

Example: Find the First and Follow sets for the grammar

$S \rightarrow aAbc/Bac$

$A \rightarrow CabdB/aBc$

$B \rightarrow ad/ef/\epsilon$

$C \rightarrow k/\epsilon$

Solⁿ:

| | First() | Follow() |
|--------------------------------|----------------------|---------------|
| $S \rightarrow aAbc/Bac$ | $\{a, \epsilon\}$ | $\{ \$ \}$ |
| $A \rightarrow CabdB/aBc$ | $\{k, a\}$ | $\{b\}$ |
| $B \rightarrow ad/ef/\epsilon$ | $\{a, e, \epsilon\}$ | $\{a, b, c\}$ |
| $C \rightarrow k/\epsilon$ | $\{k, \epsilon\}$ | $\{a\}$ |

Table 2

| Follow() | First() |
|---------------|----------------------|
| $\{ \$ \}$ | $\{a, \epsilon\}$ |
| $\{b\}$ | $\{a, \epsilon, k\}$ |
| $\{a, b, c\}$ | $\{a, \epsilon\}$ |
| $\{a\}$ | $\{a, \epsilon, k\}$ |

example: (3 page slide to page-2)

$\epsilon \rightarrow \underline{t} \epsilon'$
 $\epsilon \rightarrow \underline{+} \underline{t} \epsilon' / \epsilon$
 $\underline{t} \rightarrow \underline{f} \underline{f}'$
 $\underline{t} \rightarrow \underline{x} \underline{f} \underline{f}' / \epsilon$
 $\underline{f} \rightarrow (\epsilon) | x | y$

underline = non-terminal

Soln:

| Production | Follow | Follow |
|---|----------------------|----------------------------|
| $\epsilon \rightarrow \underline{t} \epsilon'$ | $\{x, y, \epsilon\}$ | $\{x, y\}$ |
| $\epsilon' \rightarrow \underline{+} \underline{t} \epsilon' / \epsilon$ | $\{+, \epsilon\}$ | $\{+, \epsilon\}$ |
| $\underline{t} \rightarrow \underline{f} \underline{f}'$ | $\{x, y, \epsilon\}$ | $\{+, x, y, \epsilon\}$ |
| $\underline{t} \rightarrow \underline{x} \underline{f} \underline{f}' / \epsilon$ | $\{x, \epsilon\}$ | $\{+, x, y, \epsilon\}$ |
| $\underline{f} \rightarrow (\epsilon) x y$ | $\{x, y, \epsilon\}$ | $\{x, +, x, y, \epsilon\}$ |

Follow

turning

$\epsilon \rightarrow \underline{+} \epsilon'$

$\underline{t} \rightarrow \underline{f} \underline{f}'$

$f = \text{non-terminal}$
 $f \in \{x, y\}$

$f \neq \{x, y\}$

$f \in \{x, y\}$

$\epsilon' = \text{non-terminal}$

$\text{First}(\epsilon') = \{+, \epsilon\}$

$\epsilon \rightarrow \underline{+} \underline{t} \epsilon' / \epsilon$

$\epsilon \rightarrow \underline{+} \underline{t} \epsilon' / \epsilon$

$\underline{t} \rightarrow \underline{f} \underline{f}'$

non-terminal

$\underline{t} \rightarrow \underline{f} \underline{f}'$
 $\underline{t} \rightarrow \underline{f} \underline{f}'$
 $\underline{t} \rightarrow \underline{f} \underline{f}'$

→ First find the input symbol that is not final state
 Follow the transition on that symbol to the state which is final state

| Non terminal | input symbol | | | | | | |
|--------------|--------------------|--------------------|----------------------|----------------------|----------------------|----------------------|---------------------------|
| | x | y | + | * | (|) | \$ |
| <u>e</u> | $e \rightarrow xe$ | $e \rightarrow ye$ | | | $e \rightarrow (e$ | | |
| <u>e'</u> | | | $e' \rightarrow +e'$ | | $e' \rightarrow *e'$ | $e' \rightarrow)e'$ | $e' \rightarrow \epsilon$ |
| <u>t</u> | $t \rightarrow xt$ | $t \rightarrow yt$ | | | $t \rightarrow (t$ | | |
| <u>t'</u> | | | $t' \rightarrow +t'$ | $t' \rightarrow *t'$ | | $t' \rightarrow)t'$ | $t' \rightarrow \epsilon$ |
| <u>f</u> | $f \rightarrow xf$ | $f \rightarrow yf$ | | $f \rightarrow *f$ | $f \rightarrow (f$ | | |
| | | | | | | | |

DANGLE LINE Grammar (Not LL1) = grammar in which some non-terminal has more than one production with the same left-hand side.
 → Not possible to make parse table.

$t \rightarrow f t$
 $t \rightarrow (x, y)$

$e \rightarrow te$
 $e \rightarrow +te$
 $e \rightarrow te$

$e \rightarrow \epsilon$
 $t \rightarrow (x, y)$

$f \rightarrow (x) | y$

$t \rightarrow +t t' | \epsilon$
 $t' \rightarrow t$ (because Follow(t') is not empty)
 Follow(t') = {+, \$,)}

$e \rightarrow \epsilon$
 $\$,)$

Follow(t') = {+, \$,)}

Follow(t') = {+, \$,)}

Q. Construct LR(0) items and LR(0) automaton for the following grammar.

Create a parse table using LR(0) items.

$$P \rightarrow i | c | n | T S$$

$$Q \rightarrow P | c | S | b S c S T$$

$$R \rightarrow b | \epsilon$$

$$S \rightarrow c | R n | \epsilon$$

$$T \rightarrow R S q$$

LR(0) items and LR(0) automaton for the following grammar.

LR(0) items and LR(0) automaton for the following grammar.

Soln: LR(0) items and LR(0) automaton for the following grammar.

| Non-terminals | Final() | Follow() |
|---------------|---------------------|-----------------|
| P | {i, c, n, T} | {#} |
| Q | {c, n, o, b} | {#} |
| R | {b, \epsilon} | {n, b, q} |
| S | {c, b, n, \epsilon} | {c, b, n, q} |
| T | {c, b, n, q} | {c, b, n, q, #} |

Table:

| Non-terminals | i | c | n | a | b | q | \$ |
|---------------|--------------|-------|-------|-------|-------|-------|--------------|
| P | P → i | P → c | P → n | | | | |
| Q | Q → P | Q → c | Q → n | Q → a | Q → b | Q → q | |
| R | R → \epsilon | R → b | R → q | | R → b | R → q | R → \epsilon |
| S | S → c | S → b | S → n | | S → c | S → b | S → \epsilon |
| T | T → c | T → b | T → n | | T → c | T → b | T → q |

→ predictive Parsing (Programming After 2)

→ One type of Top Down Parsing

→ Only works on LL(1) Grammars

→ Leftmost Derivation

Before Predictive Parsing

→ Grammar needs to have no left recursion.

→ Grammar needs left-factored

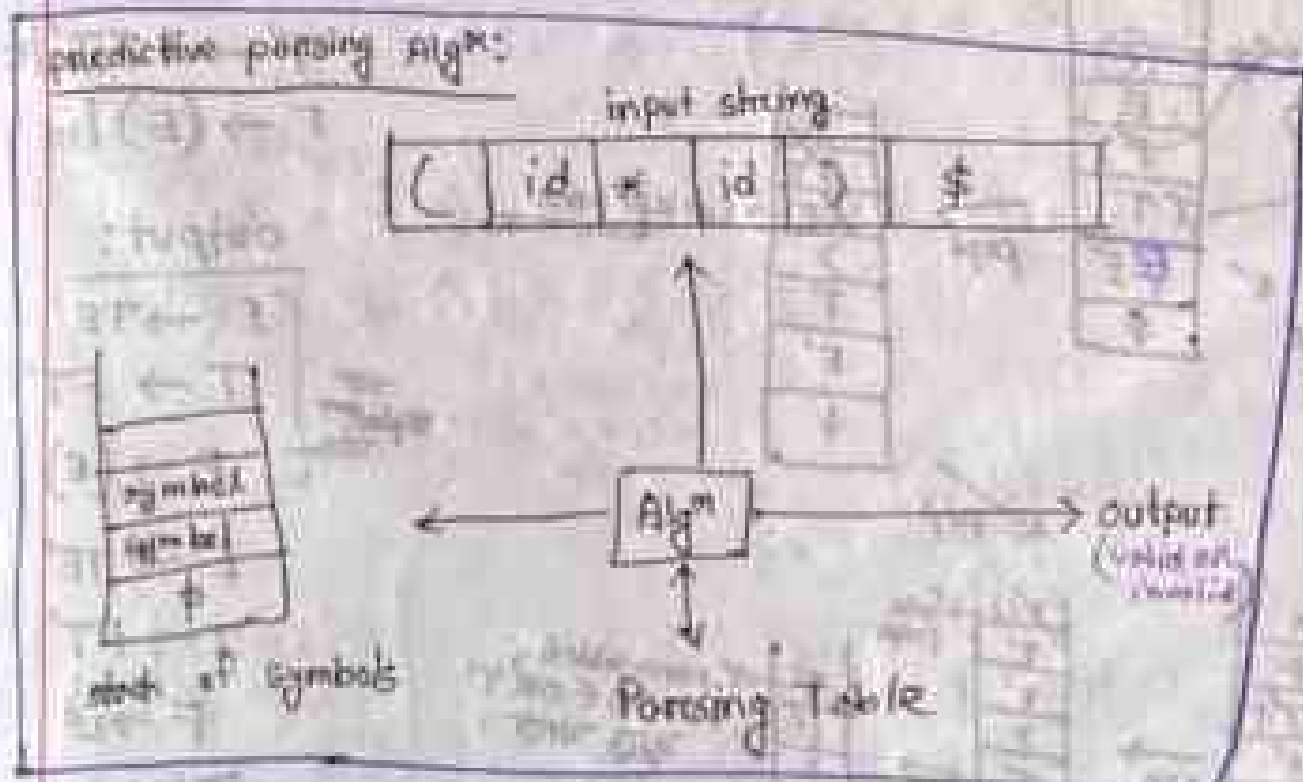
Step 1: Find FirstPOS and FollowPOS

| production | FirstPOS | FollowPOS |
|----------------------------------|---------------------|------------------|
| $E \rightarrow TE'$ | $\{ (, id \}$ | $\{), \$ \}$ |
| $E' \rightarrow +TE' / \epsilon$ | $\{ +, \epsilon \}$ | $\{), \$ \}$ |
| $T \rightarrow FT' / \epsilon$ | $\{ +, \epsilon \}$ | $\{ +,), \$ \}$ |
| $T' \rightarrow *ET' / \epsilon$ | $\{ *, \epsilon \}$ | $\{ +,), \$ \}$ |
| $F \rightarrow (E)id$ | $\{ (, id \}$ | $\{ +,), \$ \}$ |

Step 2: Find Parsing Table

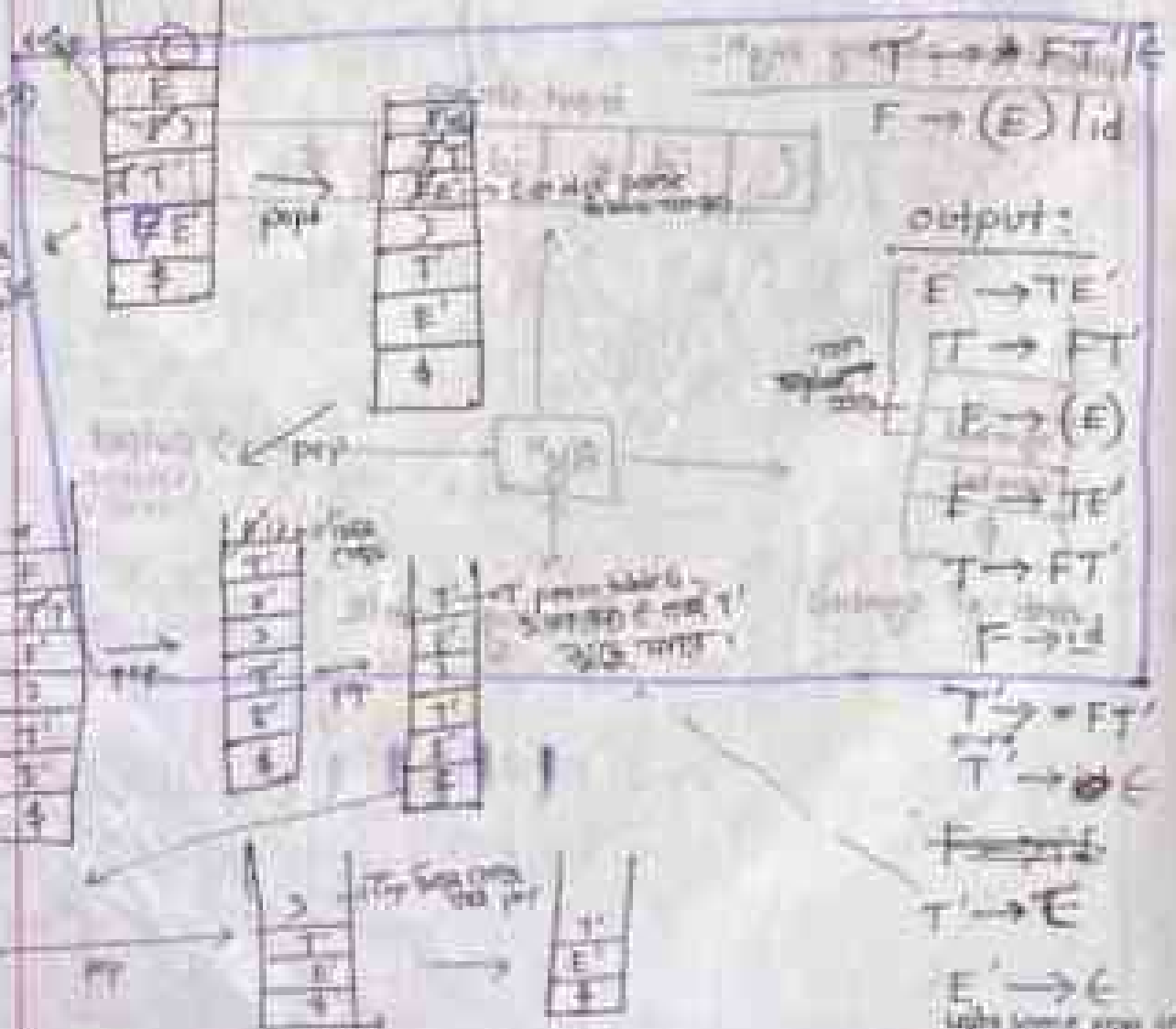
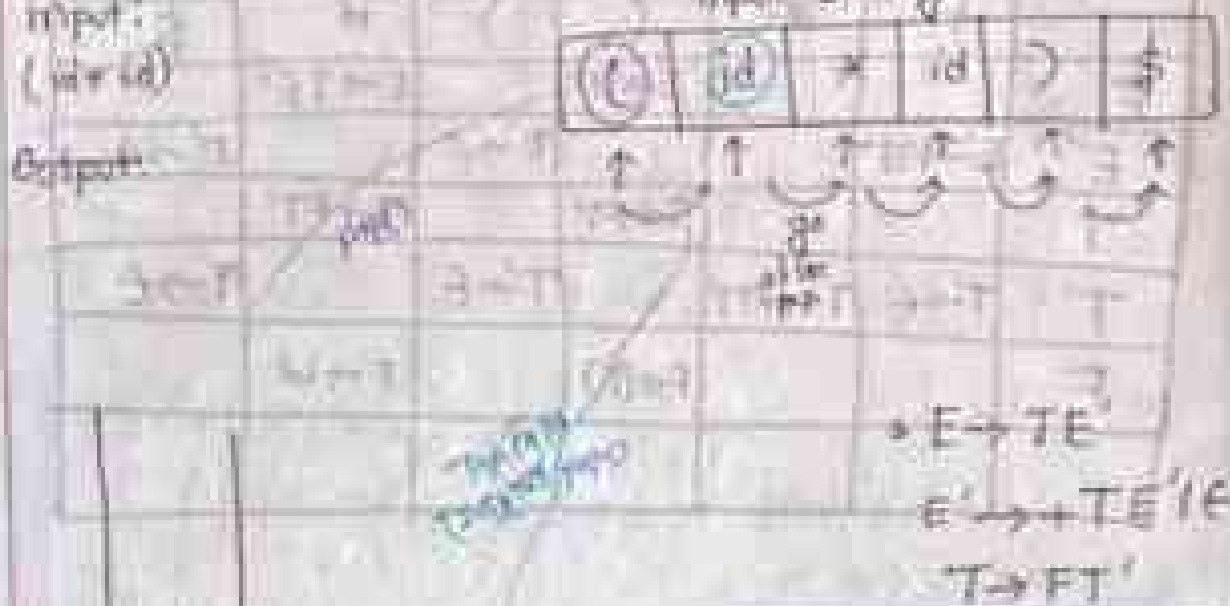
| | A | * | (|) | id | \$ |
|----|---------------------------|---------------------------|---------------------|---------------------------|---------------------|---------------------------|
| E | | | $E \rightarrow TE'$ | | $E \rightarrow TE'$ | |
| E' | $E' \rightarrow TE$ | | | $E' \rightarrow \epsilon$ | | $E' \rightarrow \epsilon$ |
| T | | | $T \rightarrow FT'$ | | $T \rightarrow FT'$ | |
| T' | $T' \rightarrow \epsilon$ | $T' \rightarrow \epsilon$ | | $T' \rightarrow \epsilon$ | | $T' \rightarrow \epsilon$ |
| F | | | $F \rightarrow (E)$ | | $F \rightarrow id$ | |
| | | | | | | |

predictive parsing Algⁿ:



input string

Output:





$\gamma \rightarrow \text{e}^+ \text{e}^-$ or $\mu^+ \mu^-$
or only $\mu^+ \mu^-$
etc.

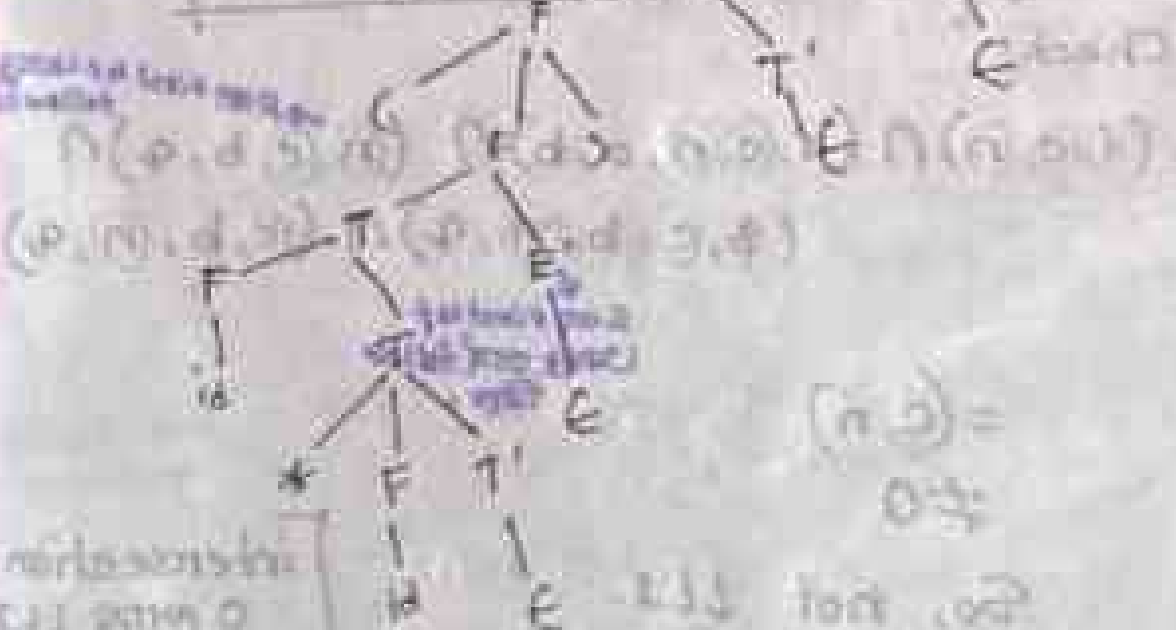
There is no clutch.

$$E' \rightarrow E$$

So, solid string

Now, please free from

| So, valid string | Count | Answer |
|------------------------|-------|--------|
| once free from output: | | 9 |
| | | 0 |
| | | 8 |
| | | 2 |
| | | 1 |


$$\begin{pmatrix} 1 & 2 \\ 0 & 1 \end{pmatrix} =$$

0 WHITE (13.1%)

Top-Down Parsers with backtracking = Brute Force

LL(1) or not check:

| Non-terminals | First() | Follow() |
|---------------|-----------------|-----------------|
| P | {i, c, n} | {} |
| Q | {i, c, n, a, b} | {} |
| R | {b, e} | {n, c, b, a} |
| S | {c, b, n, e} | {i, c, b, n, a} |
| T | {c, b, n, a} | {c, b, n, a} |

Check:

$$(i, c, n) \cap (i, c, n, a, b) \cap (n, c, b, a) \cap (c, b, n, e) \cap (c, b, n, a)$$

$$= (c, n)$$

$$\neq \emptyset$$

So, not LL1

intersection of 0 or more LL1 is LL1

Unit 3

22-09-2025 (Date)

Basic block:

Rules:

Rule 1: Determine the leader

1. First stat is leader

2. goto or break statement is leader

block 1

1. a = b
2. d = a + b
3. goto (3)

3rd leader
1st leader
2nd leader
block 1

4. c = f + g

5. h = i + j
6. goto (10)

block 2

7. m = n + o
8. k = 1 - n
9. k2 = 12 + n

block 3

10. a = 0
11. goto (4)

block 4

12. e = f + 5
13. g = e + 10

block 5

Follow diagram:

Block 1

Block 2

Block 3

Code optimization: 6 basic technique are

→ Compile Time Evaluation:

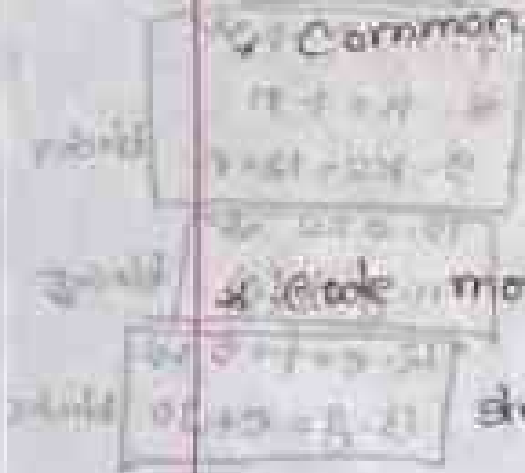
→ constant Folding

→ constant propagation



→ Common sub-expression elimination:

same type sub-expression दुबारा नही



→ Code movement:

code ko move krna

$$\begin{matrix} y = 5 \\ z = 6 \end{matrix} \quad \left. \begin{matrix} \\ \end{matrix} \right\} \text{value}$$

memory me store krna

→ Dead Code Elimination:

if (i < 1) (code ko execute krna)

→ Strength Reduction:

⇒ Code (या) थापाई आहे. (म्हणजे) compiler time evaluation करू शकतो आहे. (म्हणजे) ही step उल्लेखित आहे. 'code optimize करू शकतो' आहे.

LL(1), LR(0) = Home work

LL(1), LR(0) (या) आहे already



A Product of
Mgi

Fresh Exercise Book

OS → 5

Compiler → 3 (anor)

~~21101~~ → 17 (198)



Name

Institute

Roll/ID Year

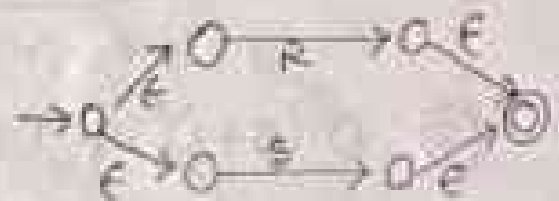
Section/Dept. Subject

* Regular Expression to NFA using Thompson's Rule :

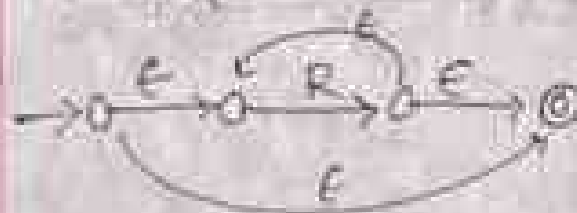
① $\pi = RS$



② $\pi = R + S$ (Union)
 $\Rightarrow \pi = R | S$



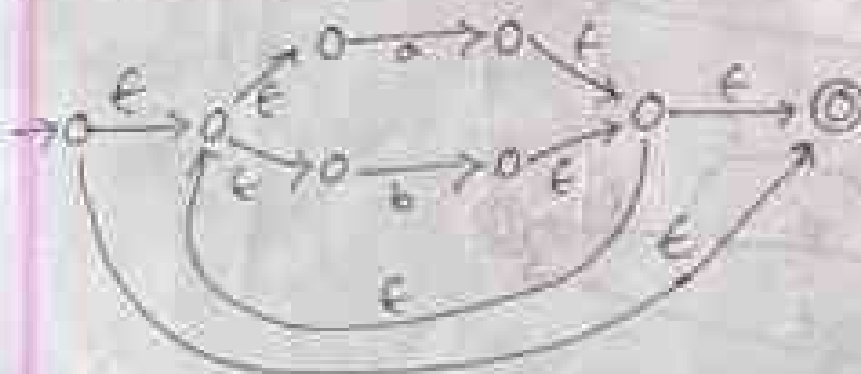
③ $\pi = (R)^*$ (R - closure)



Ex:

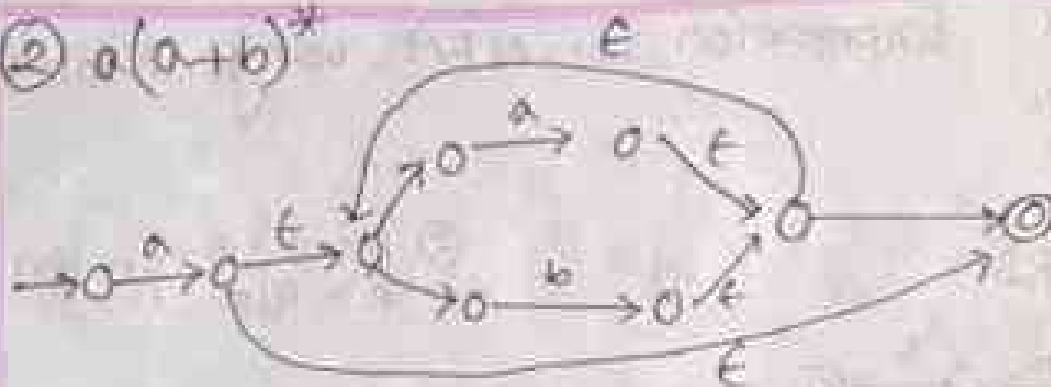
③ $RE = (a+b)^*$

$\Rightarrow RE = (a|b)^*$

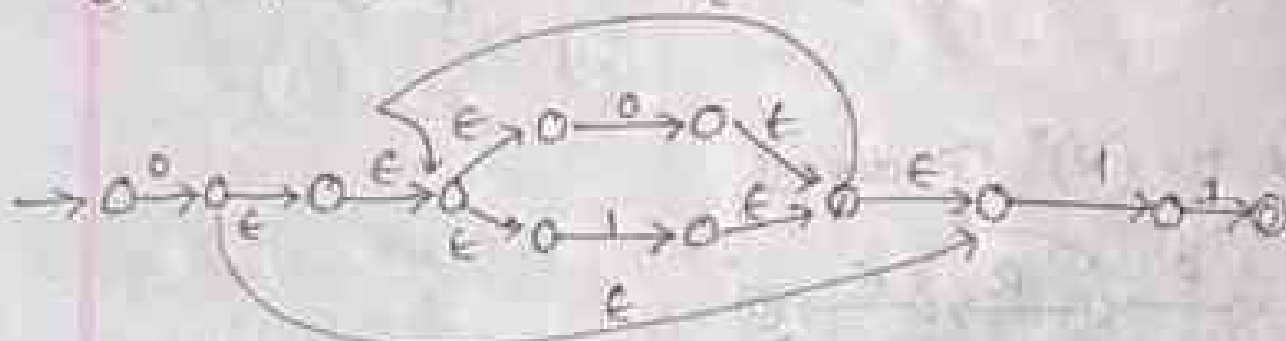


study with sourmdeep

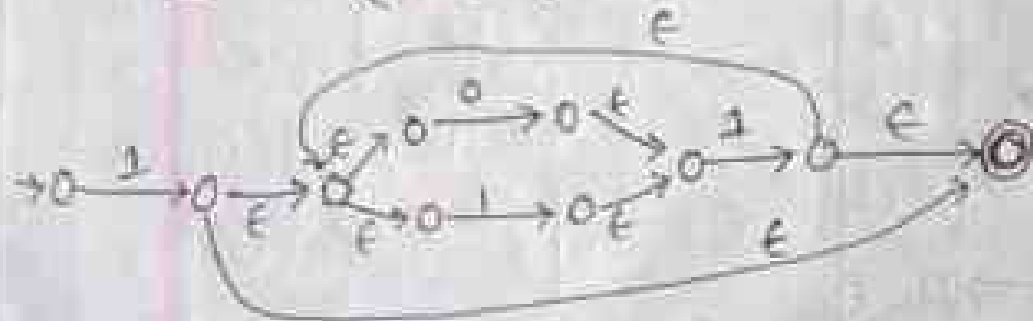
② $a(a+b)^*$



③ $0(0+1)^*11$



④ $1((0+1)1)^*$



⑤ a^+



7/5/2020 207

⑥ ab^*



⑦ $(ab^* | a^*b)^*$

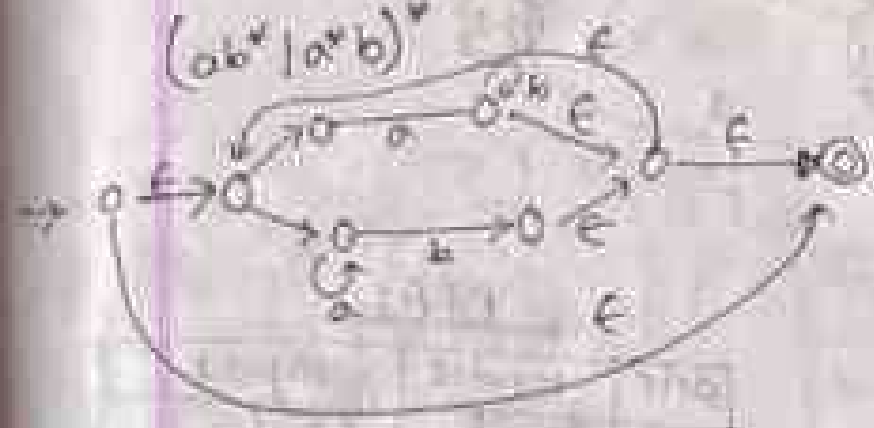
ab^* :



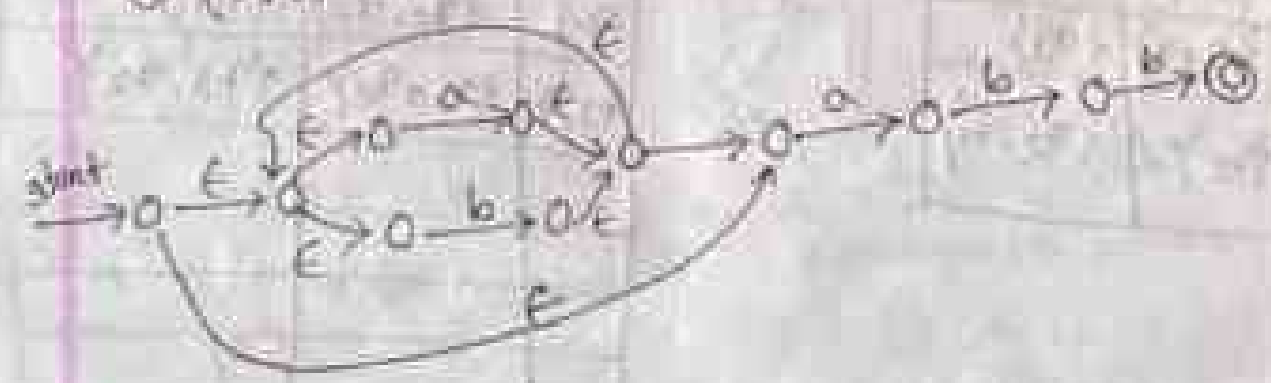
a^*b :



$(ab^* | a^*b)^*$



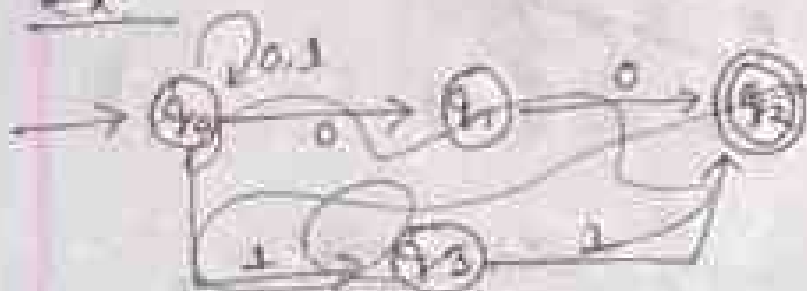
⑧ $(a|b)^* a b b$



→ TOC note (20%)

* NFA to DFA convert:

ex:



Transition table:

NFA:

| state | input 0 | input 1 |
|----------------|------------------------------------|------------------------------------|
| q ₀ | {q ₀ } | {q ₁ } |
| q ₁ | {q ₁ , q ₂ } | {q ₂ } |
| q ₂ | {q ₂ } | {q ₂ , q ₁ } |

DFA:

| state | input 0 | input 1 |
|---------------------------------|------------------------------------|------------------------------------|
| q ₀ | {q ₀ } | {q ₁ } |
| q ₁ | {q ₁ , q ₂ } | {q ₂ } |
| q ₁ , q ₂ | {q ₁ , q ₂ } | {q ₁ , q ₂ } |
| | | |
| | | |
| | | |
| | | |

DFA:



4. ϵ -NFA to NFA:



ϵ -closure:

$$\epsilon\text{-closure}(q_0) = \{q_0, q_1, q_2\}$$

$$\epsilon\text{-closure}(q_1) = \{q_1, q_2\}$$

$$\epsilon\text{-closure}(q_2) = \{q_2\}$$

ϵ -NFA:

| | a | b | c | ϵ |
|-------|-------------|-------------|-------------|-------------|
| q_0 | q_0 | $\emptyset$ | $\emptyset$ | q_1 |
| q_1 | $\emptyset$ | q_1 | $\emptyset$ | q_2 |
| q_2 | $\emptyset$ | $\emptyset$ | q_2 | $\emptyset$ |

NFA table:

| | a | b | c |
|-------|---------------------|----------------|-----------|
| q_0 | $\{q_0, q_1, q_2\}$ | $\{q_1, q_2\}$ | $\{q_2\}$ |
| q_1 | $\emptyset$ | $\{q_1, q_2\}$ | $\{q_2\}$ |
| q_2 | $\emptyset$ | $\emptyset$ | $\{q_2\}$ |

For (q_0, a) :

$$\emptyset = \epsilon\text{-closure}(\epsilon\text{-closure}(q_0))$$

$$= \epsilon(\epsilon(q_0, a, q_2))$$

$$= \epsilon(q_0, a) \cup \epsilon(q_1, a) \cup \epsilon(q_2, a)$$

$$= \emptyset \cup \emptyset \cup \emptyset$$

$$= \epsilon(q_0) = \{q_0, q_1, q_2\}$$

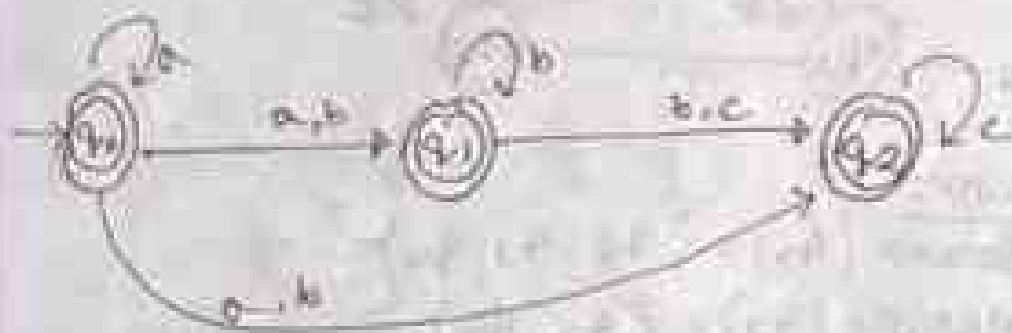
For (q_0, b) : $\epsilon\text{-closure}(\epsilon(q_0, b, q_1))$

$$= \epsilon(q_1, b) \cup \epsilon(q_2, b)$$

$$= \emptyset \cup \emptyset \cup \emptyset$$

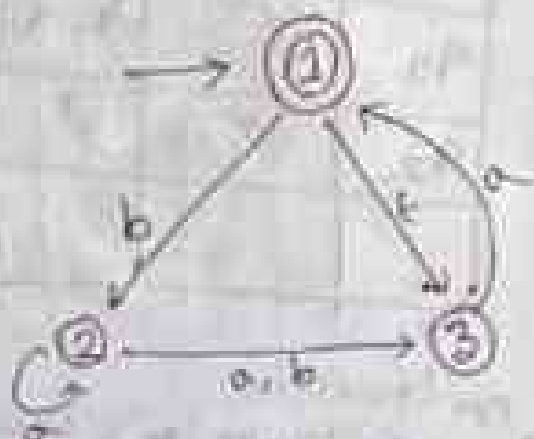
$$= \{q_1, q_2\}$$

Equivalent NFA:



यहाँ final state का भी भरो closure is q2
उपलब्ध।

* E-NFA to DFA:



$$E\text{-closure}(1) = \{1, 3\}$$

$$E\text{-closure}(2) = \{2\}$$

$$E\text{-closure}(3) = \{3\}$$

E-NFA:

| | a | b |
|---|-------------|-------------|
| 1 | $\emptyset$ | $\{2\}$ |
| 2 | $\{1, 3\}$ | $\{3\}$ |
| 3 | $\{1\}$ | $\emptyset$ |

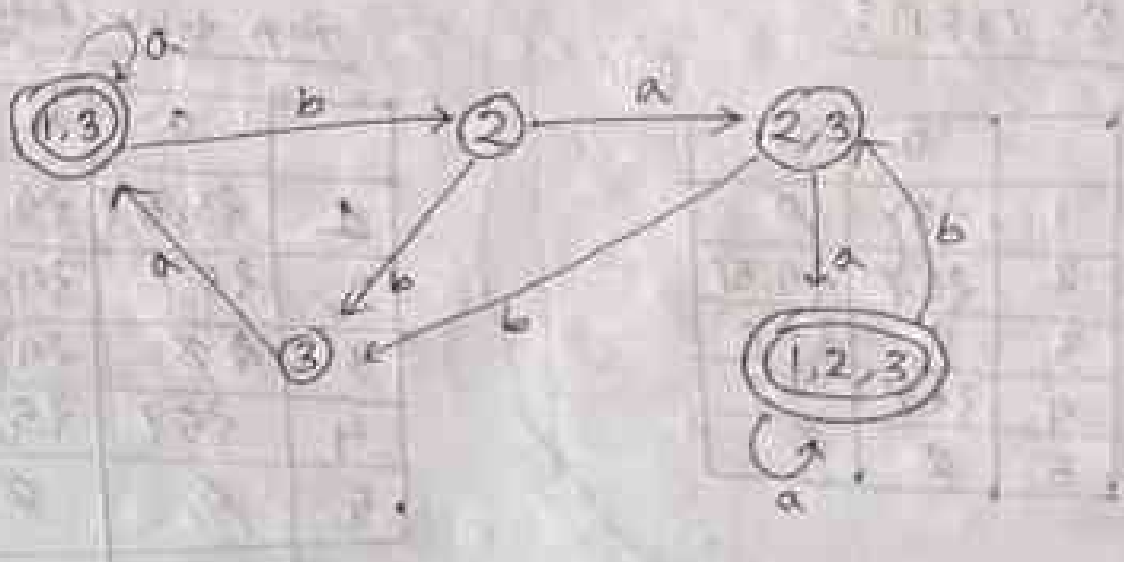
initial state of E-NFA = $\{1\}$

initial state of DFA = $E(\{1\}) = \{1, 3\}$

DFA:

| | a | b |
|-------------|-------------|-------------|
| $\{1,3\}$ | $\{1,3\}$ | $\{2\}$ |
| $\{2\}$ | $\{2,3\}$ | $\{3\}$ |
| $\{2,3\}$ | $\{1,2,3\}$ | $\{3\}$ |
| $\{3\}$ | $\{1,3\}$ | $\emptyset$ |
| $\{1,2,3\}$ | $\{1,2,3\}$ | $\{2,3\}$ |

DFA:



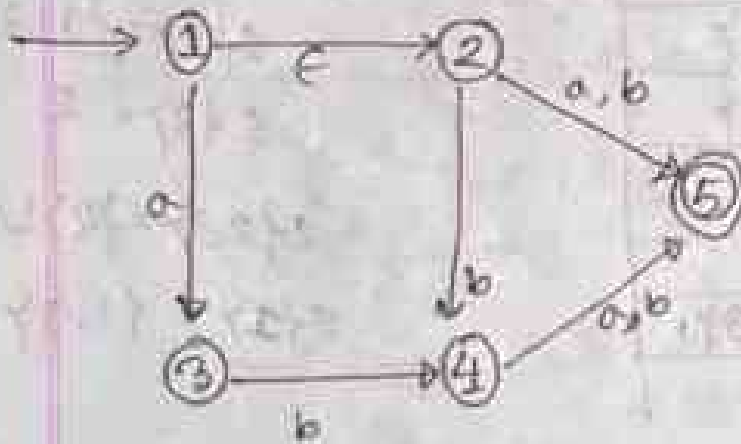
$$\delta(\{1,3\}) = \{1,3\}$$

$$\delta(\{2\}) = \{2\}$$

$$\delta(\{2,3\}) = \{2\} \cup \{3\}$$

$$\delta(\{1\}) = \{1,3\}$$

example from slide:



ϵ -clone(1) = {1, 2}
 ϵ -clone(2) = {2}
 ϵ -clone(3) = {3}
 ϵ -clone(4) = {4}
 ϵ -clone(5) = {5}

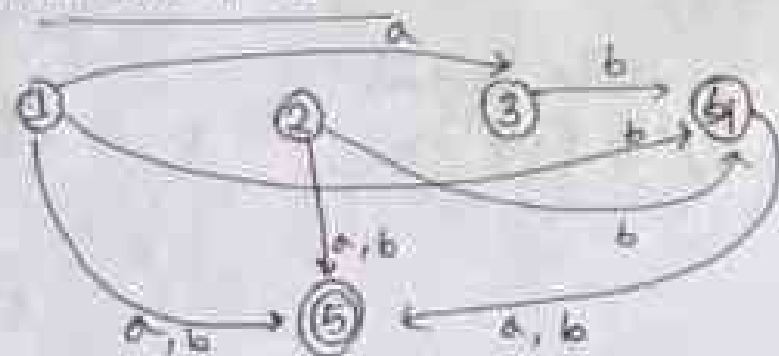
ϵ -NFA:

| | a | b |
|---|-------------|-------------|
| 1 | {3} | $\emptyset$ |
| 2 | {5} | {4, 5} |
| 3 | $\emptyset$ | {4} |
| 4 | {5} | {5} |
| 5 | $\emptyset$ | $\emptyset$ |

NFA table (Additional state)

| | a | b |
|---|-------------|-------------|
| 1 | {3, 5} | {4, 5} |
| 2 | {5} | {4, 5} |
| 3 | $\emptyset$ | {4} |
| 4 | {5} | {5} |
| 5 | $\emptyset$ | $\emptyset$ |

NFA without ϵ



initial for DFA = ϵ -closure(1) = $\{1, 2\}$

DFA:

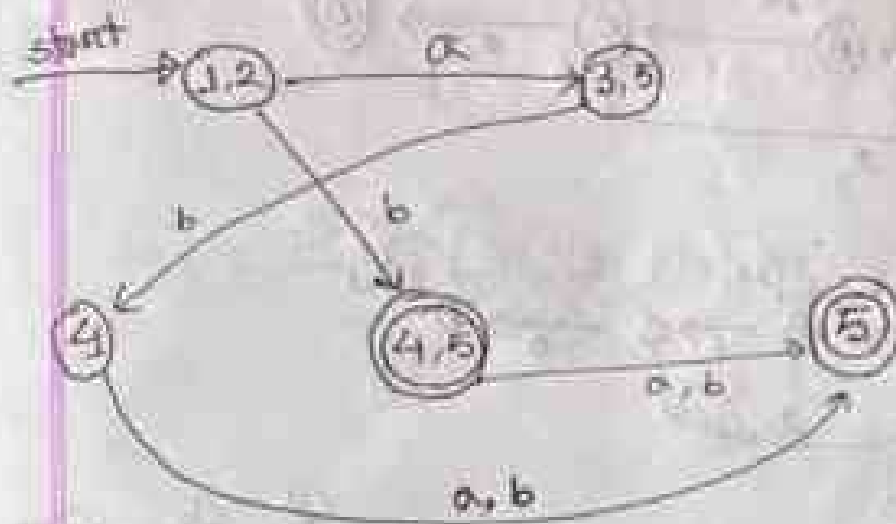
| | a | b |
|------------|-------------|-------------|
| $\{1, 2\}$ | $\{3, 5\}$ | $\{4, 5\}$ |
| $\{3, 5\}$ | $\emptyset$ | $\{4\}$ |
| $\{4, 5\}$ | $\{5\}$ | $\{5\}$ |
| $\{4\}$ | $\{5\}$ | $\{5\}$ |
| $\{5\}$ | $\emptyset$ | $\emptyset$ |

ϵ -cl($\{3, 5\}$)

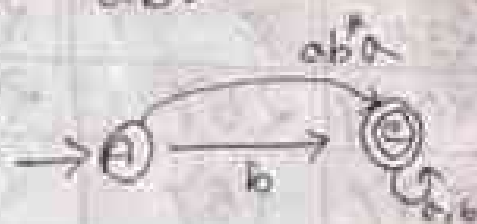
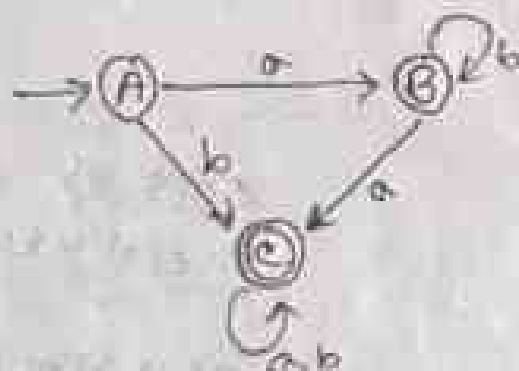
$= \{3\} \cup \{5\}$

অর্থাৎ $\{3, 5\}$ মডি
মাধ্যম আছে তাই
 ϵ -closure নিল।

Diagram of DFA:-

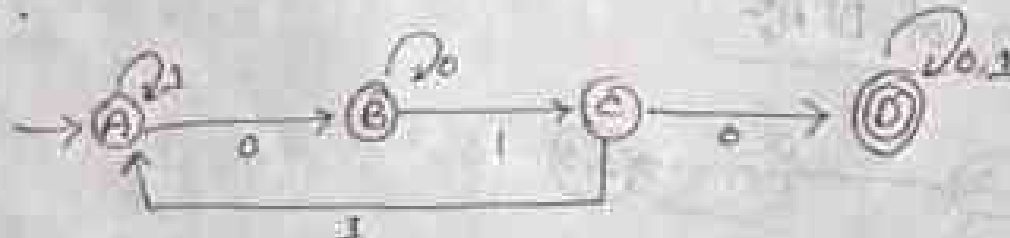


* State elimination method:

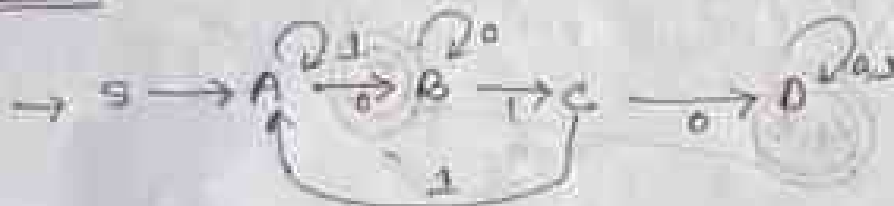


$$= (ab^*a|b)(a|b)^*$$

4.



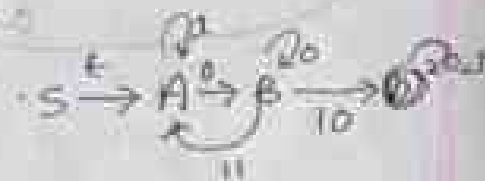
Split



Eliminate C:

$$B \xrightarrow{1} C \xrightarrow{1} A = 1A$$

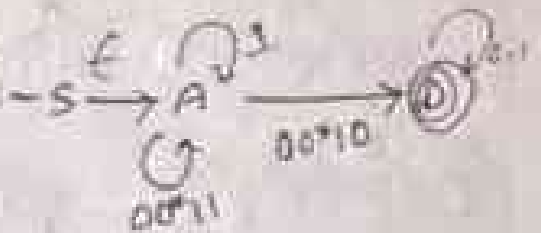
$$B \xrightarrow{1} C \xrightarrow{0} D = 1D$$



Eliminate B:

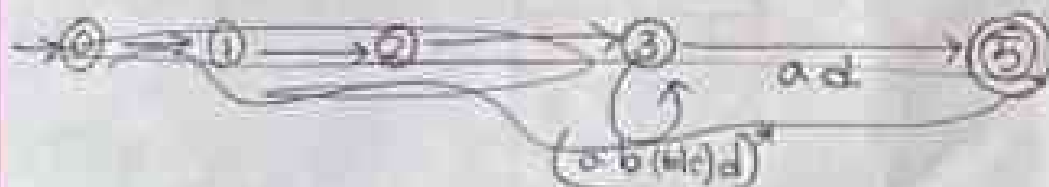
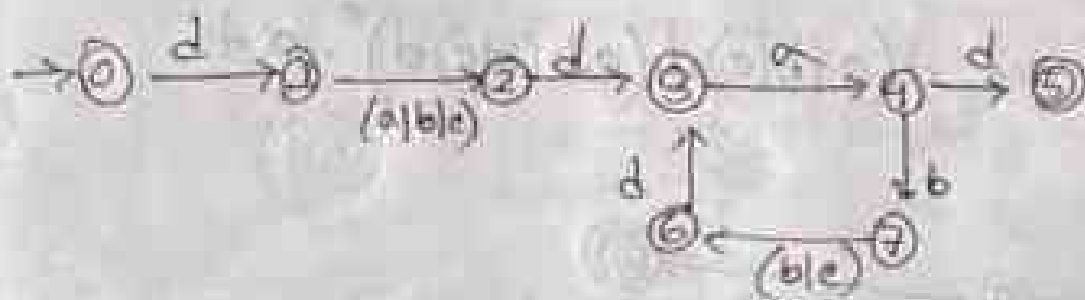
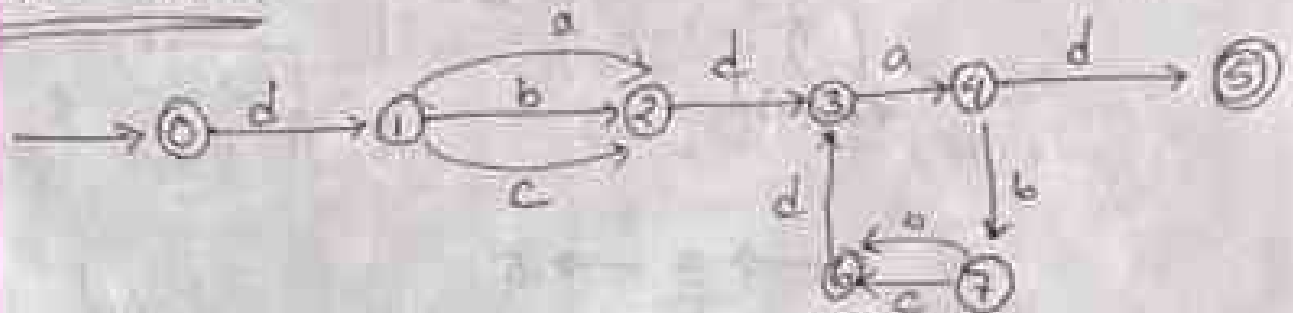
$$A \xrightarrow{0} B \xrightarrow{11} A = 00^*11$$

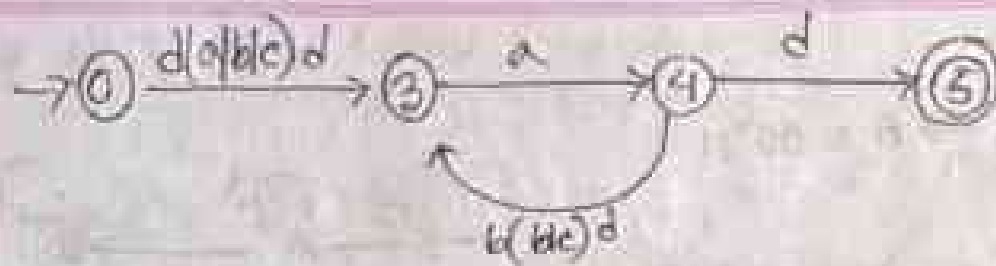
$$A \xrightarrow{0} B \xrightarrow{10} \text{end} = 00^*10$$



$$= (00^*11|1)^* (00^*10) (0|1)^+$$

Sin's slide:

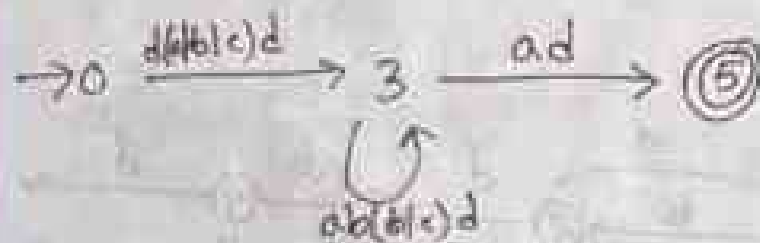




$$3 \xrightarrow{a} 4 \xrightarrow{d} 5 = ad$$

or

$$3 \xrightarrow{d(a|b|c)d} 4 \xrightarrow{a} 3 = ab(b|c)d$$



$$0 \rightarrow 3 \rightarrow 5$$

$$= d(a|b|c)d \cdot (ab(b|c)d)^* \cdot ad$$



lec-18 (Forhan)

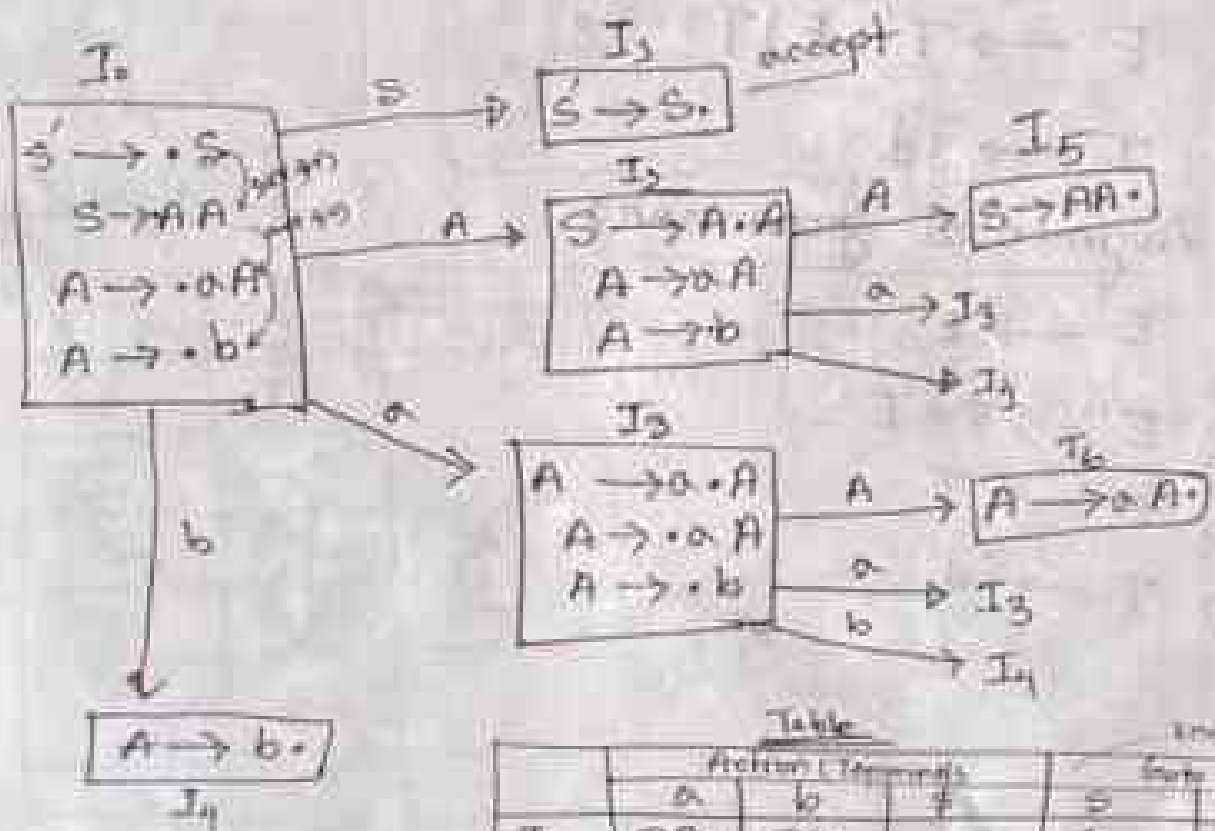
LR(0) parsing Table

Grammar:

$S \rightarrow AA$
 $A \rightarrow aA|b$

Augmented Grammar:

$S' \rightarrow S$
 $S \rightarrow AA$
 $A \rightarrow aA|b$



Table

| | Action / Transitions | | | Look ahead | |
|----------------|----------------------|----|--------|------------|---|
| | a | b | # | S | A |
| I ₀ | S3 | S4 | | 1 | 2 |
| I ₁ | | | accept | 0 | 0 |
| I ₂ | S3 | S4 | | 0 | 5 |
| I ₃ | S3 | S4 | | | 6 |
| I ₄ | R3 | R3 | R3 | | |
| I ₅ | A1 | P1 | P1 | | |

Now,
 0. $S' \rightarrow S$
 1. $S \rightarrow AA$
 2. $A \rightarrow aA$
 3. $A \rightarrow b$

Reduction

→ scanner looks for the
numbering - case R is made.

For (if) condition (if) - case R is made.

Example of Not LR(0) Grammar:

$$E \rightarrow T + E \mid T$$

$$T \rightarrow id$$

Augmented grammar:

① $E' \rightarrow E$

② $E \rightarrow T + E$

③ $E \rightarrow T$

④ $T \rightarrow id$

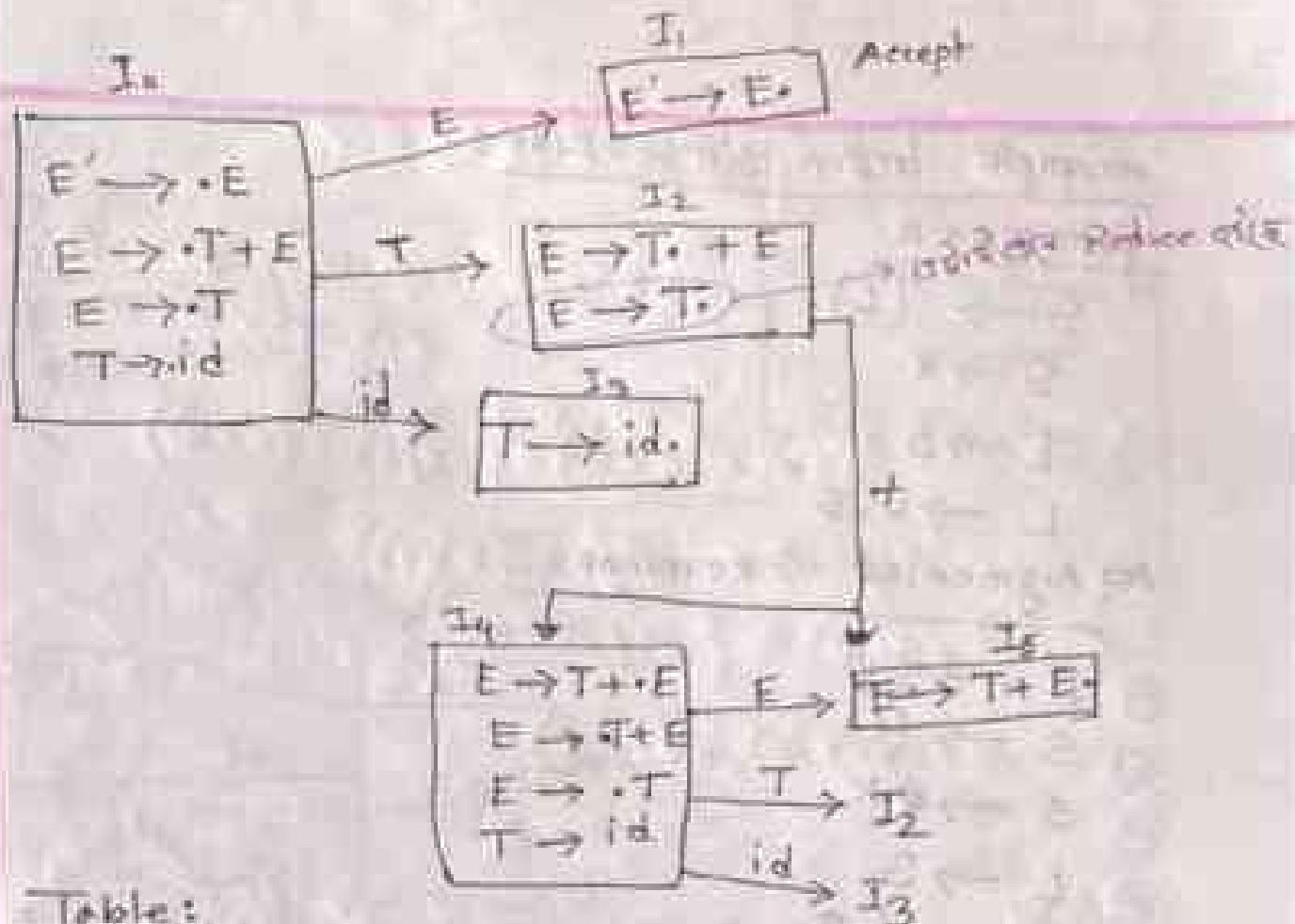
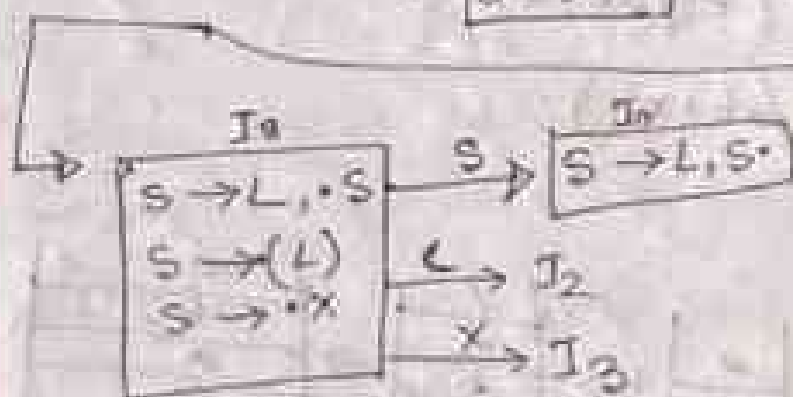
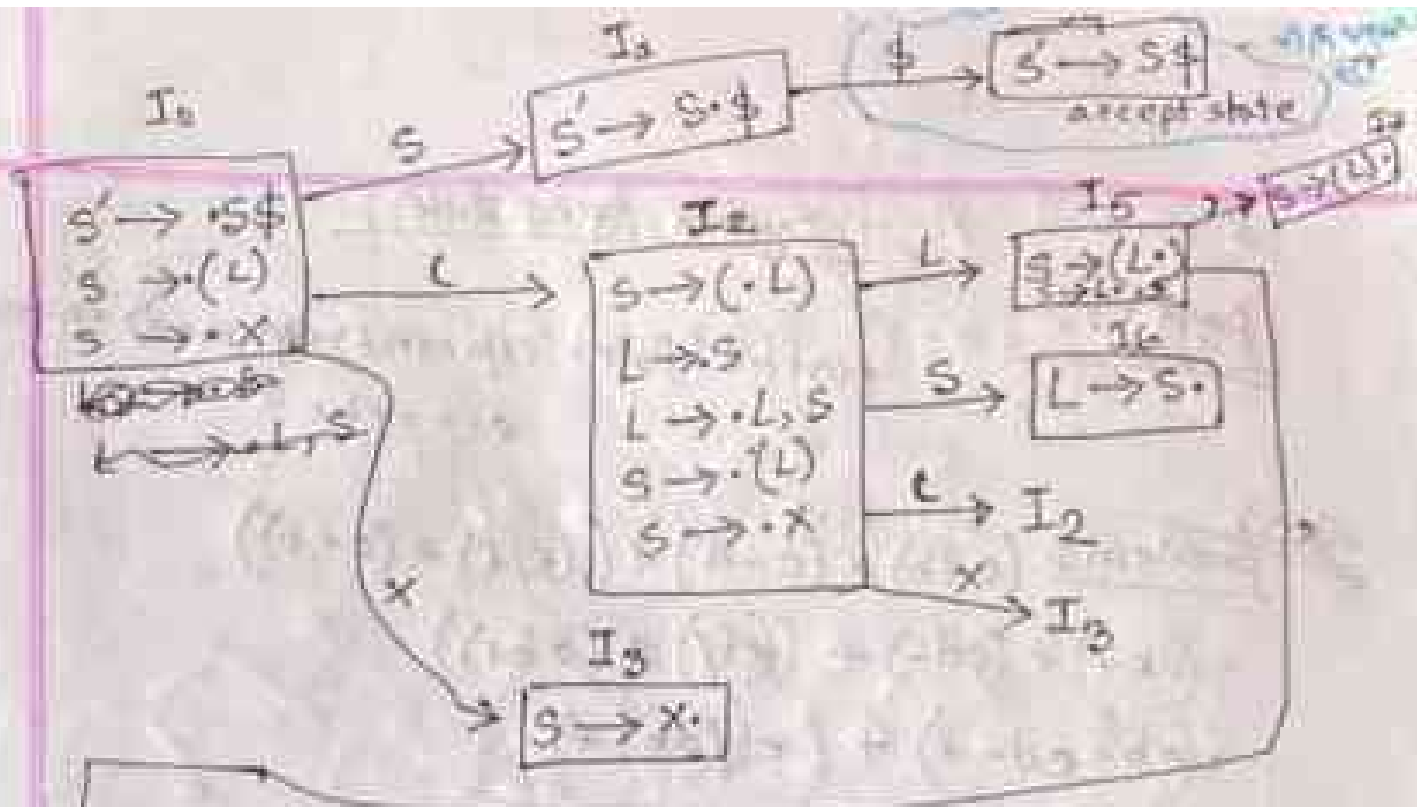


Table:

| | Action | | | Goto | |
|-------|--------|-------|--------|------|---|
| | id | + | # | E | T |
| I_0 | S3 | | Error | 1 | 2 |
| I_1 | | | Accept | | |
| I_2 | R3 | S3/R3 | R3 | | |
| I_3 | R4 | R4 | R4 | | |
| I_4 | S3 | | | 5 | 2 |
| I_5 | R2 | R2 | R2 | | |

For this grammar, LR(0) is not LR(1) because

| | |
|----|----|
| 姓名 | 学号 |
| 性别 | 年龄 |



LR(0) parsing Table

| States | Action | | | | | Goto | |
|--------|--------|----|----|----|----|------|----|
| | (| , | X | S | \$ | S | L |
| I_0 | S2 | | S3 | | | 81 | |
| I_1 | R2 | R2 | R2 | | | | |
| I_2 | S2 | | S3 | | | 86 | 85 |
| I_3 | R3 | R3 | R3 | R3 | R3 | | |
| I_4 | | | | | | | |
| I_5 | | S7 | | S8 | | | |
| I_6 | R4 | R4 | R4 | R4 | R4 | | |
| I_7 | R2 | R2 | R2 | R2 | R2 | | |
| I_8 | S2 | | S3 | | | 89 | |
| I_9 | R5 | R5 | R5 | R5 | R5 | | |

Syntax tree slide

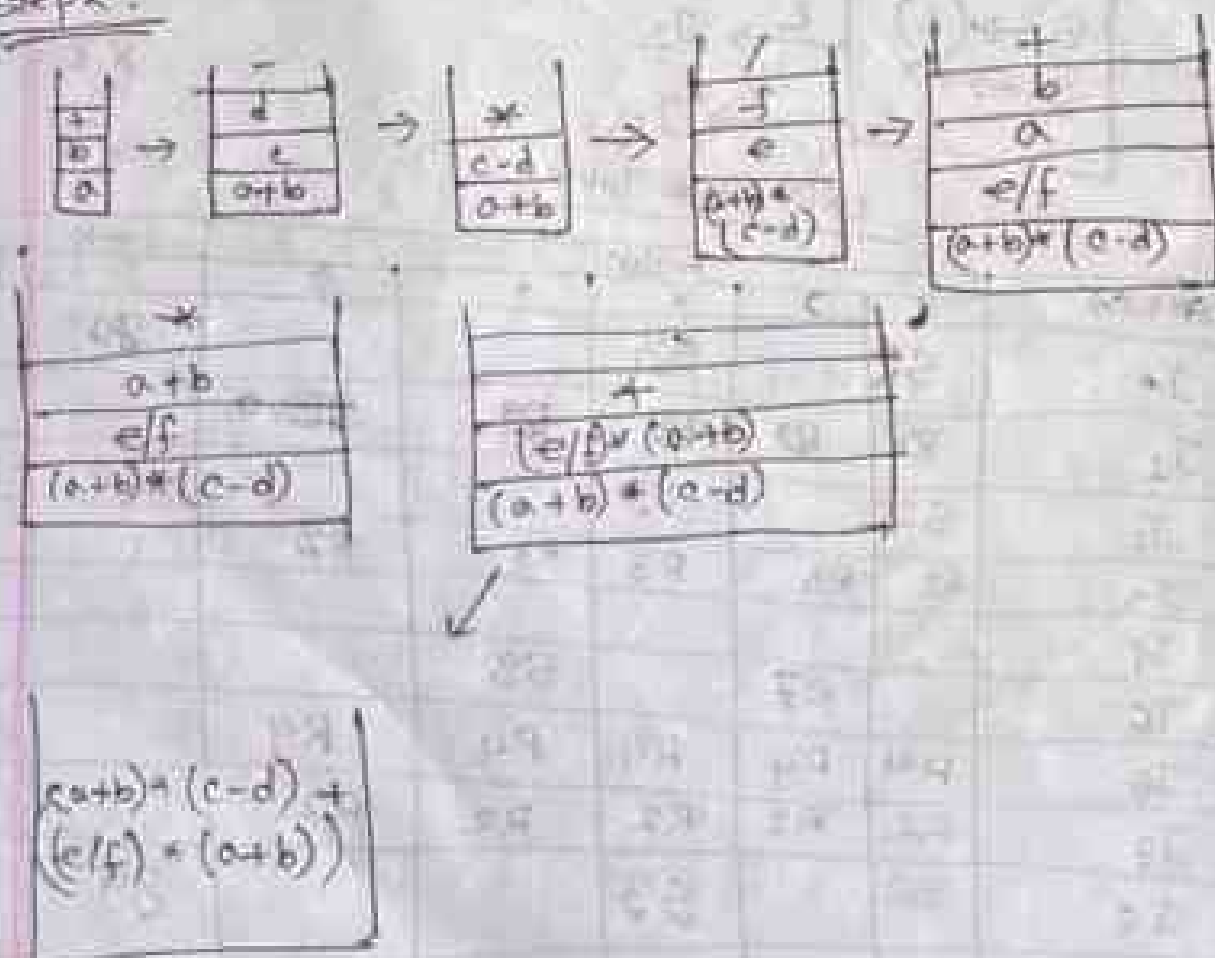
page - 7:

postfix Rule = $\langle \text{operand} \rangle \langle \text{operand} \rangle \langle \text{operator} \rangle$
 $a + b = ab +$

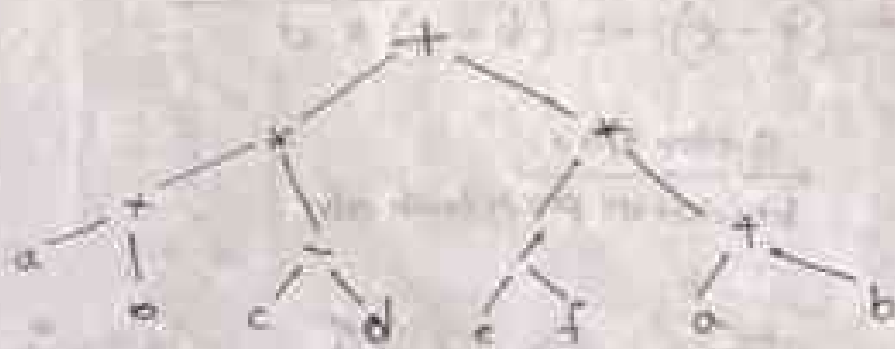
step 1:

problem: $(a+b) * (c-d) + ((e/f) * (a+b))$
 $= (ab+) * (cd-) + ((ef/) * (ab+))$
 $= (ab+cd-* + (ef/ab+*))$
 $= (ab+cd-* ef/ab+*+)$

step 2:



Syntax Tree:



* Parse Tree: (detailed grammatical structure)

- shows complete derivation of a string according to grammar rules.
- Includes all grammar symbols (both terminals & non-terminals)
- Represents syntactic structure

* Syntax Tree: (simplified, abstract representation)

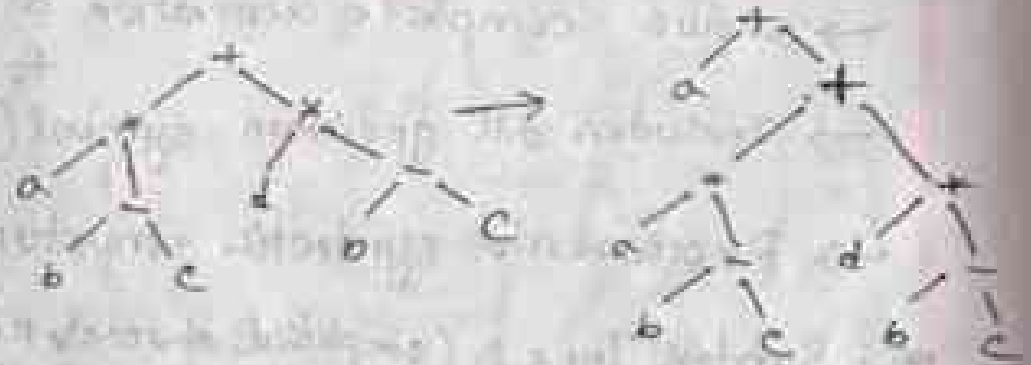
- shows abstract structure of the source code.
- Omits unnecessary non-terminals & focuses on meaningful constructs.
- Represents hierarchical, semantic relationship between elements.

Directed Acyclic Graph (DAG):

$$a + a * (b - c) + (b - c) * d$$

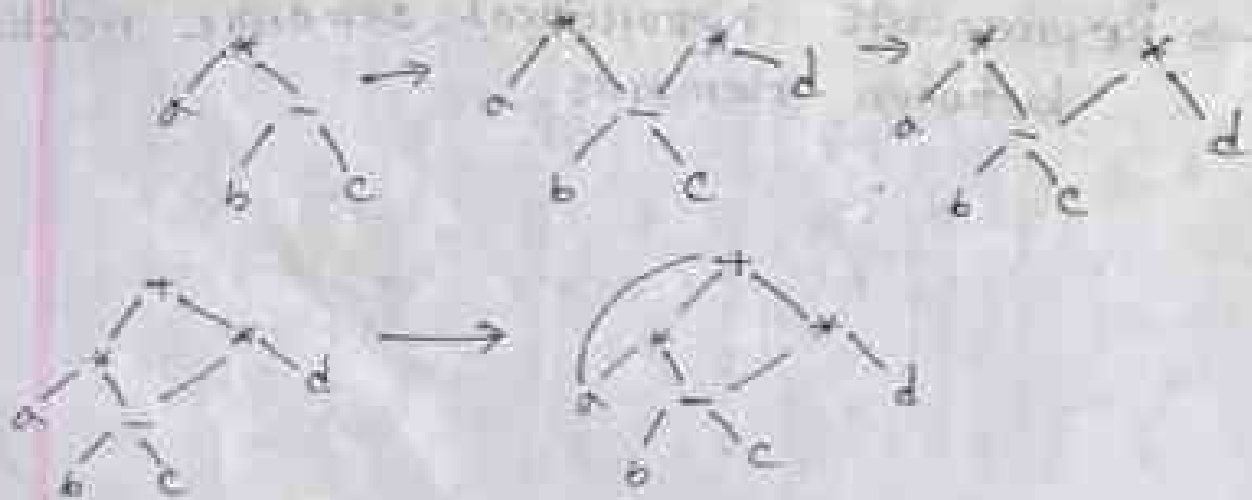
Symbol tree:

bracket use precedence (or)



DAG:

[common sub-expression sharing]



3. $id + id * id$



DAG



4. $a + a * b$



DAG

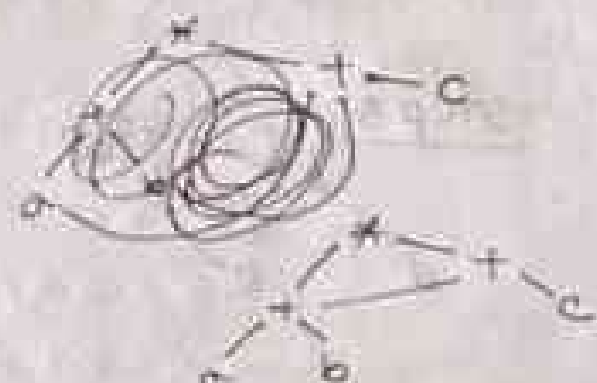


5. Slide:

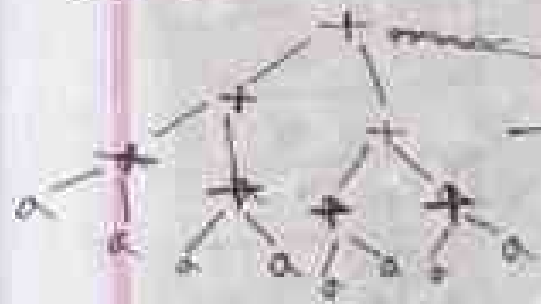
⊙ $(a + b) * (a + b + c)$



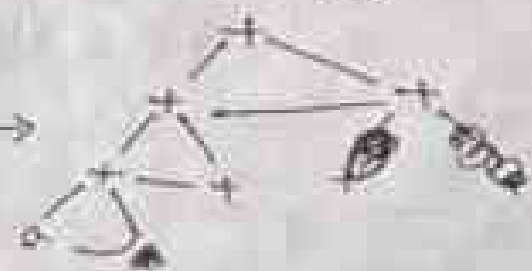
DAG



⊙ $((a + a) + (a + a)) + ((a + a) + (a + a))$



DAG





AND



prob-3:

step 1:



step 2:



step 3:



step 4:



step 5:



step 6:



$$(((a+b)*c)+d) = ((a+b)*c)+d$$



* Quadruples & Triples:

Ex: $a + (b * c + b * c + d)$

Three Address Code (TAC):

$$\begin{aligned} T_1 &= b * c \\ T_2 &= T_1 + T_1 \\ T_3 &= T_2 + d \\ T_4 &= a + T_3 \end{aligned} \Rightarrow$$

$$\begin{aligned} +1 &= b * c \\ +2 &= +1 + +1 \\ +3 &= +2 + d \\ +4 &= a + +3 \end{aligned} \left. \vphantom{\begin{aligned} +1 &= b * c \\ +2 &= +1 + +1 \\ +3 &= +2 + d \\ +4 &= a + +3 \end{aligned}} \right\} \text{instructions}$$

Quadruples In Quadruples each instruction is represented on a record with four fields:

- Operator
- Argument 1 & 2 (two operands 1 & 2)
- Result

| Operator | Argument 1 | Argument 2 | Result |
|----------|------------|------------|--------|
| * | b | c | +1 |
| + | +1 | +1 | +2 |
| + | +2 | d | +3 |
| + | a | +3 | +4 |

Triples: each instruction is represented on a record with three fields:

- Operator
- Argument 1 & 2: use previous reference using index

| | index | operator | Argument 1 | Argument 2 |
|----|-------|----------|------------|------------|
| +2 | 0 | * | b | c |
| +2 | 1 | + | 0 | 0 |
| +3 | 2 | + | 1 | d |
| +4 | 3 | + | a | 2 |

QA slide:

page 6

E2: $a + b * c / e + f + b * a$

TAC:

$$T1 = e + f$$

$$T2 = b * c$$

$$T3 = T2 / T1$$

$$T4 = b * a$$

$$T5 = a + T3$$

$$T6 = T5 + T4$$

Quadruples:

| Location | operator | Argument 1 | Argument 2 | Result |
|----------|----------|------------|------------|--------|
| 0 | + | e | f | T1 |
| 1 | * | b | c | T2 |
| 2 | / | T2 | T1 | T3 |
| 3 | * | b | a | T4 |
| 4 | + | a | T3 | T5 |
| 5 | + | T5 | T4 | T6 |

Tuple :-

| | Location | operation | Argument 1 | Argument 2 |
|----|----------|-----------|------------|------------|
| T1 | 0 | ↑ | e | f |
| T2 | 1 | * | b | c |
| T3 | 2 | / | 1 | 0 |
| T4 | 3 | * | b | a |
| T5 | 4 | + | a | 2 |
| T6 | 5 | + | 4 | 3 |

Index of Tuple:

| | statement |
|----|-----------|
| 35 | 0 |
| 36 | 1 |
| 37 | 2 |
| 38 | 3 |
| 39 | 4 |
| 40 | 5 |

| Location | operation | Argument 1 | Argument 2 |
|----------|-----------|------------|------------|
| 0 | ↑ | e | f |
| 1 | * | b | c |
| 2 | / | 1 | 0 |
| 3 | * | b | a |
| 4 | + | a | 2 |
| 5 | + | 4 | 3 |

Page-10 $a = b * -c + b * -c$

TAC:

$$T1 = \text{uminus } c$$

$$T2 = b * T1$$

$$T3 = \text{uminus } c$$

$$T4 = b * T3$$

$$T5 = T2 + T4$$

$$a = T5 \text{ } T5 = a$$

| Op | Chang | Value | Result |
|----|-------|-------|--------|
| 1 | 2 | 1 | 2 |
| 0 | 0 | 0 | 0 |
| 0 | 1 | 1 | 1 |
| 0 | 0 | 0 | 0 |
| 0 | 0 | 0 | 0 |
| 0 | 0 | 0 | 0 |

| Op | Value |
|----|-------|
| 0 | 0 |
| 1 | 1 |
| 2 | 2 |
| 3 | 3 |
| 4 | 4 |
| 5 | 5 |

* Basic Blocks and Flow Graph:

TAC:

1. $A = 1$ (L)

2. $T1 = 5$

3. $T2 = A + 3$ (L)

4. $B = T1 + T2$

5. $R = 5 - T1$

6. if $A > R$ go to (3)

7. $B1 = T1 + A$ (L)

8. $E = 5 + B$

jump at next statement

* Leader selection:

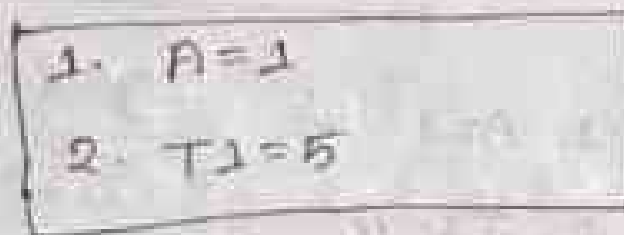
1. The first instruction is a leader

2. The target of a conditional or an unconditional goto/jump instruction is a leader

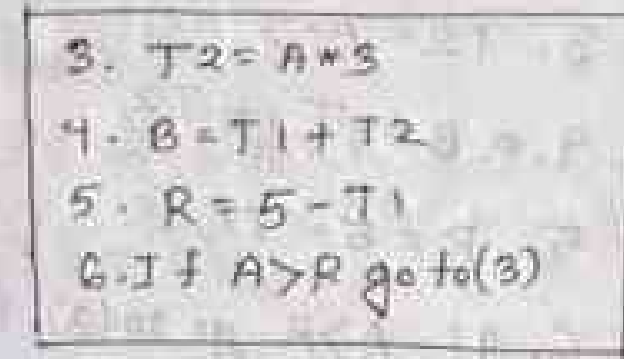
unconditional
□ go to (3)
□ if $a = 5$ go to (6) → conditional
□ return (3)

3. The instruction that immediately follows a conditional or an unconditional goto/jump instruction is a leader
(jump is next statement)

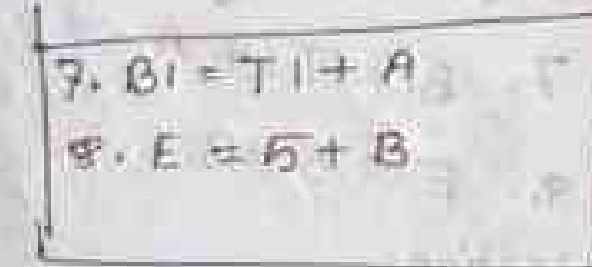
Basic Blocks



B1

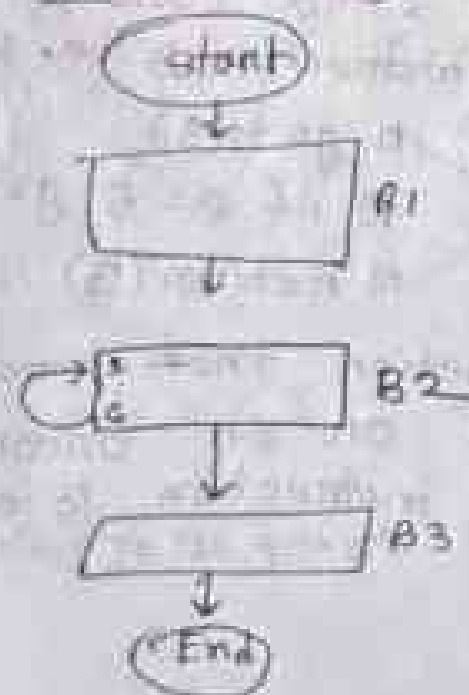


B2

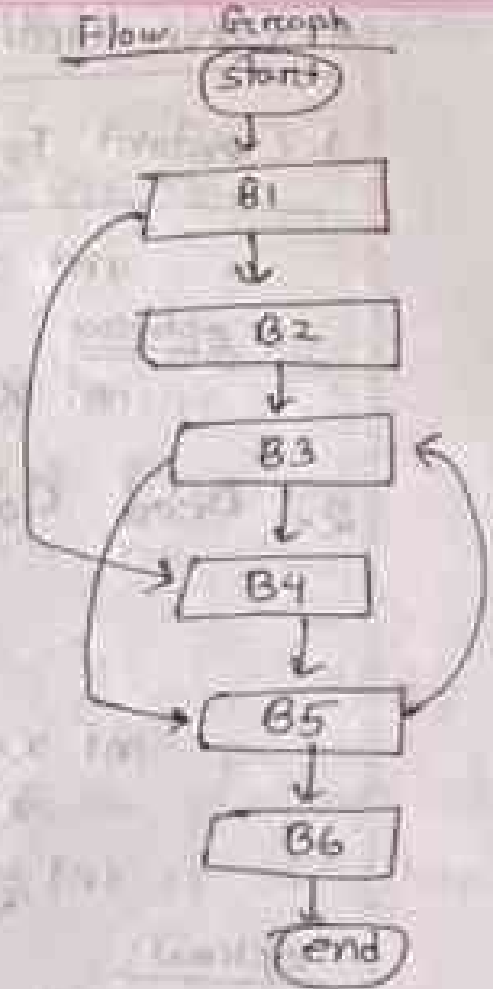
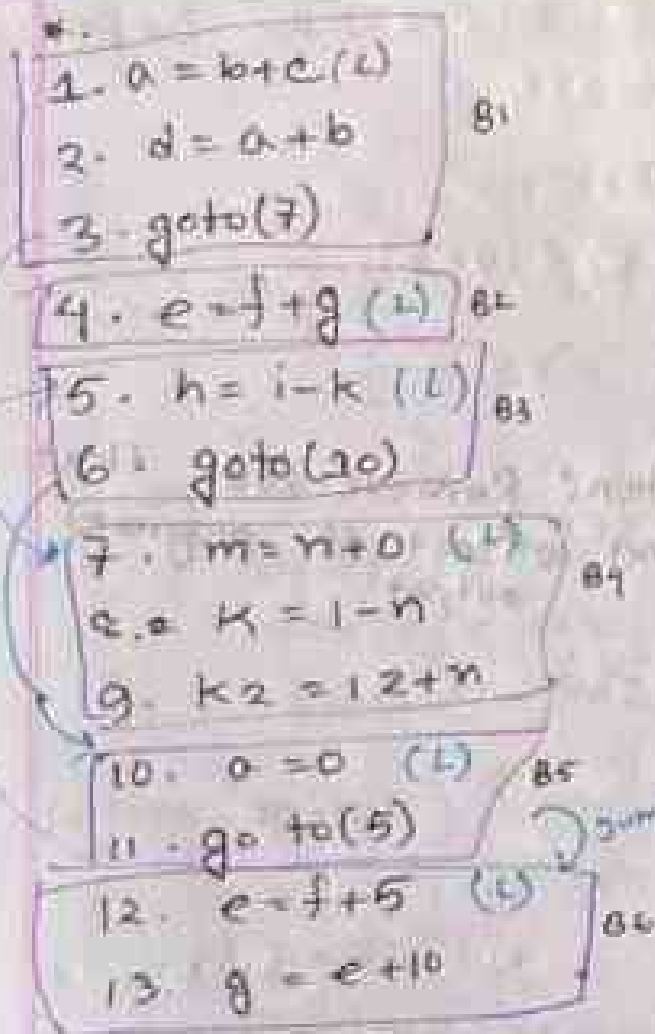


B3

Flow Graph



Flow Graph



* Code Optimization:

1. Constant Folding:

```
int x = 3 * 5
```

optimized

```
int x = 15
```

2. Dead Code Elimination: Remove code that does not affect the program's output.

```
int x = 10;
```

```
x = 20;
```

```
int y = x + 5;
```

Optimized:

```
int x = 20
```

```
int y = x + 5
```

3. Strength Reduction: Replaces expensive operations with cheaper ones.

```
int y = x * 4;
```

OP
int y = x << 2 [bitwise shift]
2²

4. Loop Unrolling: Expands the loop body to reduce the overhead of looping, often improving performance.

```
for (int i = 0; i < 3; i++) {
```

```
    arr[i] = i * 2;
```

```
}
```

op

```
arr[0] = 0 * 2;
```

```
arr[1] = 1 * 2;
```

```
arr[2] = 2 * 2;
```

5. Code Motion: Moves code outside of loops if it does not depend on the loop.

```
int y = 10;
```

```
for (int i = 0; i < 5; i++) {
```

```
    int z = y * 2;
```

```
    arr[i] = z + i;
```

opti

```
int y = 10;
```

```
int z = y * 2;
```

```
for (int i = 0; i < 5; i++) {
```

```
    arr[i] = z + i;
```

6. Common Subexpression Elimination: Remove duplicate

~~int~~ int a = 5, b = 10;

int res = (a * b) + (a * b);

op

int a = 5, b = 10;

int temp = a * b;

int res = temp + temp.

100